

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



BÁO CÁO ASSIGNMENT 02

MÔN HỌC: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn : PGS.TS. Trần Đình Quế

Nhóm học phần : 05

Sinh viên thực hiện : Vũ Thùy Dung

Mã sinh viên : B23DCCN194

Hà Nội, 9/2026

Mục lục

Mã nguồn dự án.....	1
Chương I. Giới thiệu bài toán.....	1
1.1. Bối cảnh.....	1
1.2. Mục tiêu của bài tập.....	1
1.3. Quy trình phát triển hệ thống thông minh.....	3
1.4. Tổng quan về ba ứng dụng.....	4
1.4.1. Ứng dụng dự đoán bệnh tiểu đường.....	4
1.4.2. Ứng dụng dự đoán giá nhà.....	4
1.4.3. Ứng dụng phân tích hành vi và khám phá sở thích khách hàng thương mại điện tử.....	5
Chương II. Cơ sở lý thuyết về biểu diễn dữ liệu.....	6
2.1. Tổng quan về học máy và hệ thống thông minh.....	6
2.2. Bài toán phân loại và hồi quy.....	6
2.2.1. Bài toán phân loại.....	6
2.2.2. Bài toán hồi quy.....	6
2.3. Khái niệm biểu diễn dữ liệu.....	7
2.4. Vectơ đặc trưng.....	7
2.5. Ma trận đặc trưng.....	8
2.6. Tensor và biểu diễn dữ liệu nhiều chiều.....	8
2.7. Biểu diễn dữ liệu văn bản.....	9
2.8. Mã hóa dữ liệu phân loại.....	10
2.9. Kiểu dữ liệu và miền giá trị.....	10
2.10. Tính nhất quán của biểu diễn dữ liệu.....	11
2.11. Rò rỉ dữ liệu.....	11
Chương III. Phương pháp tiền xử lý dữ liệu.....	12
3.1. Tìm hiểu dữ liệu.....	12
3.2. Làm sạch dữ liệu.....	13
3.2.1. Xử lý giá trị thiếu.....	13
3.2.2. Xử lý dữ liệu trùng lặp.....	13
3.2.3. Xử lý giá trị không hợp lệ.....	13
3.2.4. Phân tích ngoại lệ.....	14
3.3. Mã hóa biến phân loại.....	14
3.3.1. Label Encoding.....	14
3.3.2. One-Hot Encoding.....	14
3.4. Chuẩn hóa dữ liệu số.....	15

3.5. Chia tập dữ liệu.....	15
3.6. Ngăn ngừa rò rỉ dữ liệu.....	16
3.7. Pipeline tiền xử lý.....	17
3.8. Nguyên tắc tái lập quá trình tiền xử lý.....	17
Chương IV. Các độ đo đánh giá mô hình.....	18
4.1. Đối với bài toán Hồi quy.....	18
4.2. Đối với bài toán Phân lớp.....	18
Chương V. Phương pháp xây dựng mô hình.....	20
5.1. Tổng quan về xây dựng mô hình.....	20
5.2. Mô hình Baseline.....	20
5.3. Các mô hình cho bài toán hồi quy.....	20
5.4. Các mô hình cho bài toán phân loại.....	22
Chương VI. Phương pháp lưu trữ và triển khai mô hình.....	25
6.1. Tổng quan.....	25
6.2. Lưu trữ mô hình.....	25
6.3. Tính nhất quán giữa huấn luyện và dự đoán.....	25
6.4. Triển khai mô hình dưới dạng Web Service.....	26
6.5. Web Application.....	26
6.6. Triển khai trên ứng dụng mobile.....	27
6.7. Kiến trúc hoàn chỉnh của hệ thống.....	27
Chương VII. Ứng dụng 1 - Dự đoán bệnh tiểu đường.....	29
7.1. Mô tả bài toán.....	29
7.2. Giới thiệu tập dữ liệu.....	29
7.3. Khảo sát và tìm hiểu dữ liệu.....	31
7.4. Làm sạch dữ liệu.....	32
7.5. Biểu diễn dữ liệu.....	34
7.6. Phân tích khám phá dữ liệu (EDA).....	37
7.7. Xây dựng mô hình.....	41
7.7.1. Chia tập Train, Validation và Test.....	41
7.7.2. Xây dựng các mô hình phân loại.....	43
7.7.3. Baseline và huấn luyện mô hình.....	43
7.8. Đánh giá mô hình.....	44
7.9. Lựa chọn mô hình.....	46
7.10. Triển khai hệ thống.....	47
Chương VIII. Ứng dụng 2 - Dự đoán giá nhà.....	49
8.1. Mô tả bài toán.....	49

8.2. Giới thiệu tập dữ liệu.....	49
8.4. Làm sạch dữ liệu.....	54
8.5. Biểu diễn dữ liệu.....	55
8.6. Phân tích khám phá dữ liệu (EDA).....	58
8.7. Xây dựng mô hình.....	63
8.7.1. Chia tập Train, Validation và Test.....	63
8.7.2. Xây dựng các mô hình hồi quy.....	64
8.7.3. Huấn luyện mô hình.....	65
8.8. Đánh giá mô hình.....	65
8.9. Lựa chọn mô hình.....	65
8.10. Triển khai hệ thống.....	66
Chương IX. Ứng dụng 3 - Phân tích hành vi khách hàng và dự đoán khả năng đề xuất sản phẩm.....	68
9.1. Mô tả bài toán.....	68
9.2. Giới thiệu tập dữ liệu.....	68
9.3. Biểu diễn khách hàng.....	71
9.4. Làm sạch dữ liệu.....	72
9.5. Khám phá sở thích và hành vi khách hàng.....	74
9.6. Phân tích khám phá dữ liệu (EDA).....	78
9.7. Xây dựng mô hình.....	81
9.7.1. Chia tập Train, Validation và Test.....	81
9.7.2. Tiền xử lý dữ liệu.....	82
9.7.3. Baseline và huấn luyện mô hình.....	83
9.7.4. Xây dựng mô hình dựa trên nội dung đánh giá.....	84
9.8. Đánh giá mô hình.....	86
9.9. Lựa chọn mô hình.....	86
9.10. Phân tích ý nghĩa kinh doanh.....	89
9.10.1. Hành vi khách hàng được phát hiện.....	89
9.10.2. Ứng dụng trong thương mại điện tử.....	89
9.10.3. Hạn chế về dữ liệu.....	89
9.11. Triển khai hệ thống.....	90
Chương X. So sánh ba hệ thống thông minh.....	92
10.1. So sánh dữ liệu và bài toán.....	92
10.2. So sánh biểu diễn dữ liệu.....	92
10.3. So sánh chất lượng dữ liệu và tiền xử lý.....	93
10.3.1. Các bước tiền xử lý chung.....	93

10.3.2. Các bước tiền xử lý riêng.....	93
10.4. So sánh mô hình và đánh giá.....	94
10.5. So sánh triển khai.....	94
10.6. Trả lời các câu hỏi thảo luận.....	95
10.6.1. Ứng dụng dự đoán bệnh tiểu đường.....	95
10.6.2. Ứng dụng dự đoán giá nhà.....	97
10.6.3. Ứng dụng E-commerce Customer Behavior and Interest Discovery.....	100
10.6.4. Câu hỏi bổ sung cho ứng dụng E-commerce.....	103
Chương XI. Mở rộng - chatbot với neo4j.....	105
11.1. Luồng hoạt động.....	105
Luồng xử lý chi tiết:.....	105
11.2. Kết quả thực nghiệm.....	107

Mã nguồn dự án

<https://github.com/dthyfu/Assignment02.git>

Chương I. Giới thiệu bài toán

1.1. Bối cảnh

Trong thực tế, dữ liệu được tạo ra dưới nhiều hình thức khác nhau như dữ liệu dạng bảng, dữ liệu giao dịch, văn bản và các thông tin mô tả. Tuy nhiên, dữ liệu ở dạng ban đầu chưa thể được đưa trực tiếp vào hầu hết các mô hình học máy. Dữ liệu cần được tìm hiểu, làm sạch, chuyển đổi và biểu diễn dưới dạng số phù hợp để máy tính có thể xử lý và mô hình học máy có thể khai thác.

Assignment 02 với chủ đề *From Data Representation to a Deployable Intelligent System* tập trung vào quá trình chuyển đổi từ dữ liệu thực tế thành một hệ thống thông minh có khả năng dự đoán và triển khai trong thực tế. Theo yêu cầu của bài tập, quá trình phát triển không chỉ dừng lại ở việc huấn luyện một mô hình học máy mà phải thể hiện đầy đủ chuỗi xử lý:

Dữ liệu thô → Tìm hiểu → Làm sạch → Biểu diễn → Học → Đánh giá → Lưu trữ → Triển khai.

Điều này nhấn mạnh rằng chất lượng của một hệ thống thông minh phụ thuộc không chỉ vào thuật toán được sử dụng mà còn phụ thuộc vào chất lượng dữ liệu, cách dữ liệu được biểu diễn và phương pháp đánh giá mô hình. Assignment yêu cầu sinh viên chứng minh cách dữ liệu thực tế được chuyển đổi thành biểu diễn số để mô hình học máy có thể xử lý.

Bên cạnh quá trình xây dựng mô hình, hệ thống sau khi huấn luyện cần có khả năng được sử dụng lại mà không phải huấn luyện lại từ đầu. Vì vậy, mô hình và quy trình tiền xử lý cần được lưu trữ và tích hợp vào các dịch vụ dự đoán. Assignment yêu cầu xây dựng cả dịch vụ Web và giao diện Mobile cho ba ứng dụng, qua đó minh họa quá trình đưa mô hình học máy từ môi trường thực nghiệm vào một hệ thống có khả năng tương tác với người dùng.

1.2. Mục tiêu của bài tập

Mục tiêu chính của Assignment 02 là xây dựng và triển khai ba hệ thống thông minh hoàn chỉnh, bắt đầu từ dữ liệu thực tế và kết thúc bằng một dịch vụ có khả năng tương tác với người dùng.

Cụ thể, bài tập hướng tới các mục tiêu sau:

Thứ nhất, hiểu và phân tích dữ liệu thực tế.

Đối với mỗi bộ dữ liệu, cần xác định dữ liệu ban đầu có dạng gì, một quan sát đại diện

cho đối tượng nào, các cột dữ liệu có ý nghĩa gì, biến nào được sử dụng làm đặc trưng đầu vào và biến nào là biến mục tiêu.

Thứ hai, xây dựng biểu diễn dữ liệu phù hợp cho mô hình học máy.

Dữ liệu thực tế cần được chuyển thành dạng số phù hợp với mô hình. Đối với dữ liệu dạng bảng, biểu diễn có thể được xây dựng dưới dạng vectơ đặc trưng và ma trận đặc trưng. Đối với dữ liệu văn bản trong ứng dụng thương mại điện tử, quá trình biểu diễn có thể bao gồm chuyển đổi từ văn bản thành token, token ID và embedding. Assignment yêu cầu phải giải thích rõ biểu diễn dữ liệu trước khi tiến hành huấn luyện mô hình.

Thứ ba, thực hiện tiền xử lý dữ liệu một cách có hệ thống.

Các vấn đề như giá trị thiếu, dữ liệu trùng lặp, giá trị không hợp lệ, biến phân loại và sự khác biệt về miền giá trị của các đặc trưng cần được kiểm tra và xử lý phù hợp. Đồng thời, quá trình tiền xử lý phải được thực hiện theo cách có thể tái sử dụng trong giai đoạn dự đoán.

Thứ tư, xây dựng và so sánh nhiều mô hình học máy.

Mỗi ứng dụng cần được xây dựng với nhiều mô hình phù hợp với loại bài toán. Các mô hình được đánh giá trên các tập dữ liệu độc lập và được so sánh dựa trên những chỉ số phù hợp. Assignment yêu cầu năm mô hình được so sánh cho bài toán tiêu đường, năm mô hình cho bài toán giá nhà và sáu mô hình cho ứng dụng thương mại điện tử.

Thứ năm, đánh giá và lựa chọn mô hình phù hợp.

Đối với bài toán phân loại, các chỉ số như Accuracy, Precision, Recall, F1-score, Confusion Matrix và ROC-AUC được sử dụng khi phù hợp. Đối với bài toán hồi quy giá nhà, các chỉ số MAE, MSE, RMSE và R^2 được sử dụng. Kết quả đánh giá cần được phân tích để lựa chọn mô hình cuối cùng thay vì chỉ dựa vào một giá trị điểm số.

Thứ sáu, lưu trữ mô hình và quy trình tiền xử lý.

Mô hình cuối cùng phải được lưu lại để có thể sử dụng cho những lần dự đoán tiếp theo mà không cần huấn luyện lại. Đồng thời, quy trình tiền xử lý và biến đổi đặc trưng cũng cần được lưu trữ để đảm bảo dữ liệu khi triển khai được xử lý giống với dữ liệu trong quá trình huấn luyện.

Thứ bảy, triển khai mô hình thành hệ thống có khả năng sử dụng.

Mỗi ứng dụng phải được triển khai dưới dạng một dịch vụ Web và được minh họa thông qua một giao diện Mobile. Kiến trúc được đề xuất là:

Giao diện Mobile → REST API → Tiền xử lý → Mô hình học máy → Kết quả dự đoán → Giao diện Mobile.

Thông qua kiến trúc này, người dùng có thể nhập hoặc lựa chọn dữ liệu đầu vào, gửi yêu cầu dự đoán và nhận kết quả từ mô hình.

1.3. Quy trình phát triển hệ thống thông minh

Trong Assignment 02, ba ứng dụng được xây dựng theo cùng một quy trình phát triển:

Dữ liệu thô → Tìm hiểu → Làm sạch → Biểu diễn → Học → Đánh giá → Lưu trữ → Triển khai.

Quy trình này có thể được chia thành các giai đoạn chính.

Dữ liệu thô (Raw Data):

Thu thập và sử dụng các bộ dữ liệu thực tế từ Kaggle làm nguồn dữ liệu đầu vào cho hệ thống.

Tìm hiểu dữ liệu (Understand):

Kiểm tra cấu trúc dữ liệu, kích thước tập dữ liệu, kiểu dữ liệu, giá trị thiếu, dữ liệu trùng lặp, giá trị không hợp lệ và các đặc điểm quan trọng của dữ liệu.

Làm sạch dữ liệu (Clean):

Xử lý các vấn đề về chất lượng dữ liệu nhằm tạo ra dữ liệu phù hợp hơn cho các bước tiếp theo.

Biểu diễn dữ liệu (Represent):

Chuyển dữ liệu thực tế thành biểu diễn số phù hợp với mô hình. Đây là bước đặc biệt quan trọng của Assignment 02 vì cùng một dữ liệu được lưu trong tệp CSV có thể cần những cách biểu diễn khác nhau trước khi được đưa vào mô hình.

Học (Learn):

Sử dụng dữ liệu đã được tiền xử lý và biểu diễn để huấn luyện nhiều mô hình học máy phù hợp.

Đánh giá (Evaluate):

Đánh giá các mô hình trên dữ liệu kiểm tra và sử dụng các chỉ số phù hợp với từng loại bài toán.

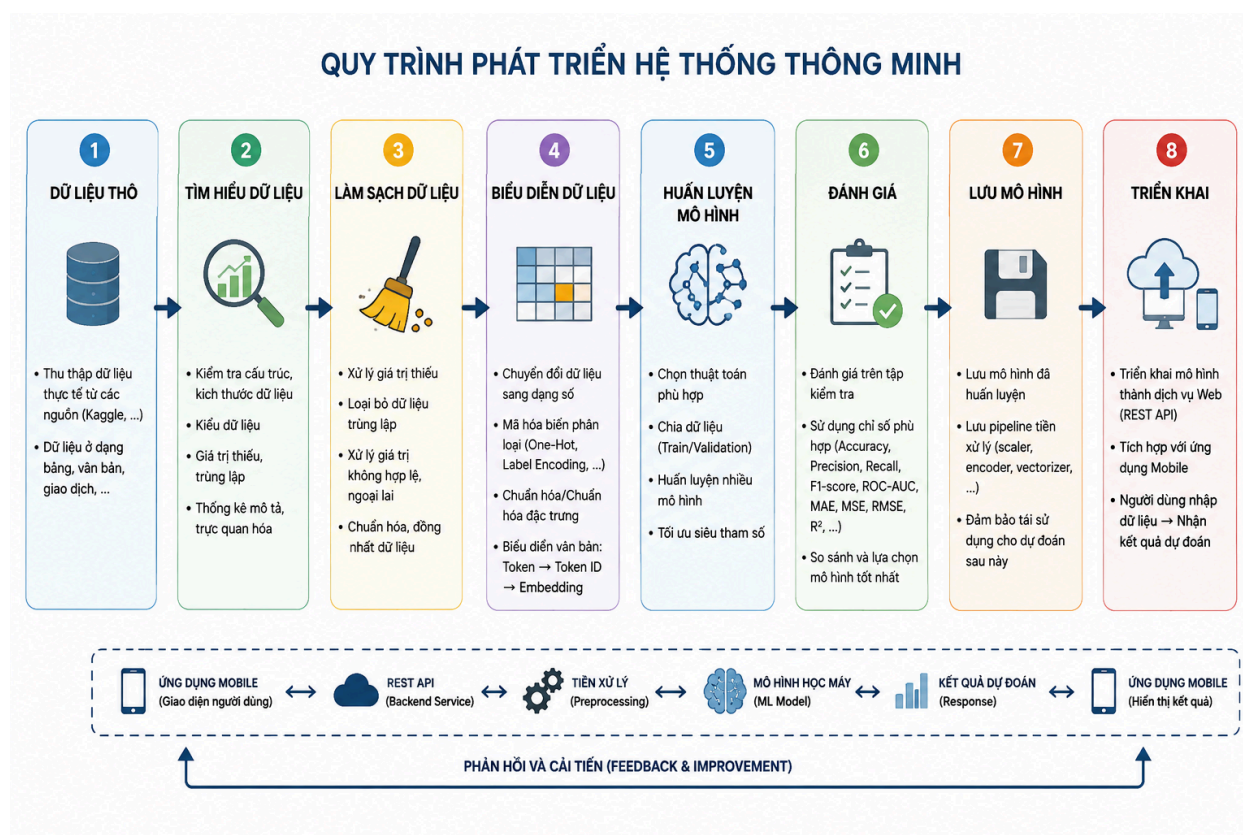
Lưu trữ (Persist):

Lưu mô hình cuối cùng cùng với các thành phần tiền xử lý cần thiết để có thể tái sử dụng mà không cần huấn luyện lại.

Triển khai (Deploy):

Đưa mô hình đã lưu vào dịch vụ Web và ứng dụng Mobile để thực hiện dự đoán trên dữ liệu mới.

Assignment nhấn mạnh rằng mục tiêu không đơn giản là “huấn luyện một mô hình”, mà là xây dựng một hệ thống thông minh nhỏ, có khả năng tái lập và triển khai được.



Hình 1.1. Quy trình phát triển hệ thống thông minh

1.4. Tổng quan về ba ứng dụng

Assignment 02 yêu cầu phát triển ba ứng dụng thông minh sử dụng ba bộ dữ liệu Kaggle khác nhau. Mặc dù cùng tuân theo một quy trình phát triển, mỗi ứng dụng có đặc điểm riêng về đối tượng quan sát, biến mục tiêu, dạng dữ liệu, biểu diễn và mô hình học máy.

1.4.1. Ứng dụng dự đoán bệnh tiểu đường

Ứng dụng thứ nhất tập trung vào bài toán dự đoán lớp liên quan đến bệnh tiểu đường dựa trên các đặc trưng của bệnh nhân. Các đặc trưng có thể bao gồm tuổi, chỉ số BMI, huyết áp, các chỉ số liên quan đến glucose, insulin và những thông tin lâm sàng khác tùy thuộc vào bộ dữ liệu được lựa chọn.

Đây là bài toán phân loại, trong đó tập đặc trưng đầu vào được ký hiệu là $\mathbf{X}$, còn biến mục tiêu đại diện cho lớp tiểu đường được ký hiệu là y . Dữ liệu dạng bảng sau khi được tiền xử lý sẽ được chuyển thành các vector đặc trưng và ma trận đặc trưng để đưa vào các mô hình học máy.

1.4.2. Ứng dụng dự đoán giá nhà

Ứng dụng thứ hai tập trung vào việc dự đoán giá của một căn nhà dựa trên các đặc điểm

của bất động sản. Các đặc trưng có thể bao gồm diện tích, số phòng ngủ, số phòng tắm, vị trí, số tầng, năm xây dựng, thông tin về chỗ đỗ xe hoặc các thuộc tính khác có trong bộ dữ liệu.

Đây là bài toán hồi quy. Trong đó, $\mathbf{X}$ biểu diễn các đặc trưng của căn nhà và y biểu diễn giá nhà cần dự đoán. Một yêu cầu quan trọng của ứng dụng là xác định các biến số và biến phân loại, sau đó chuyển chúng thành một vector đặc trưng thống nhất để mô hình có thể xử lý.

1.4.3. Ứng dụng phân tích hành vi và khám phá sở thích khách hàng thương mại điện tử

Ứng dụng thứ ba tập trung vào dữ liệu thương mại điện tử nhằm phân tích hành vi khách hàng và khám phá sở thích hoặc hành vi cần dự đoán. Bài toán cụ thể cần được xác định dựa trên bộ dữ liệu được lựa chọn, chẳng hạn như dự đoán khách hàng có mua sản phẩm hay không, dự đoán khả năng khách hàng quay lại, phân loại khách hàng hoặc dự đoán sự quan tâm của khách hàng đối với một nhóm sản phẩm.

Đối với dữ liệu giao dịch, nhiều giao dịch có thể được tổng hợp thành hồ sơ của một khách hàng, sau đó chuyển thành một vector đặc trưng biểu diễn hành vi. Ví dụ, các đặc trưng có thể bao gồm mức độ gần đây của giao dịch, tần suất mua hàng, giá trị mua hàng và số lượng hoạt động liên quan đến từng nhóm sản phẩm. Nếu bộ dữ liệu có chứa bình luận hoặc đánh giá của khách hàng, thông tin văn bản có thể tiếp tục được chuyển đổi thành token, token ID và embedding để kết hợp với các đặc trưng dạng bảng.

Chương II. Cơ sở lý thuyết về biểu diễn dữ liệu

2.1. Tổng quan về học máy và hệ thống thông minh

Học máy (Machine Learning) là phương pháp cho phép máy tính học từ dữ liệu để thực hiện các nhiệm vụ như phân loại hoặc dự đoán. Trong một hệ thống thông minh, mô hình học máy chỉ là một thành phần của toàn bộ quy trình. Dữ liệu cần được tìm hiểu, làm sạch, chuyển đổi và biểu diễn phù hợp trước khi được sử dụng để huấn luyện mô hình.

Assignment 02 tập trung vào việc xây dựng một hệ thống thông minh hoàn chỉnh theo quy trình:

Dữ liệu thô → Tìm hiểu → Làm sạch → Biểu diễn → Học → Đánh giá → Lưu trữ → Triển khai.

Trong đó, biểu diễn dữ liệu là bước kết nối giữa dữ liệu thực tế và mô hình tính toán. Assignment yêu cầu phải xác định rõ biểu diễn dữ liệu trước khi tiến hành huấn luyện mô hình.

2.2. Bài toán phân loại và hồi quy

2.2.1. Bài toán phân loại

Phân loại (Classification) là bài toán trong đó mô hình nhận dữ liệu đầu vào và dự đoán một lớp hoặc nhãn tương ứng.

Trong Assignment 02, ứng dụng dự đoán bệnh tiểu đường được xác định là bài toán phân loại. Các đặc trưng của bệnh nhân được sử dụng làm dữ liệu đầu vào và biến mục tiêu biểu diễn lớp liên quan đến bệnh tiểu đường.

Ứng dụng E-commerce cũng có thể được xây dựng dưới dạng bài toán phân loại nếu biến mục tiêu được xác định là một hành vi hoặc sở thích của khách hàng, chẳng hạn như có mua hàng hay không, phân khúc khách hàng hoặc nhóm sản phẩm mà khách hàng quan tâm.

2.2.2. Bài toán hồi quy

Hồi quy (Regression) là bài toán dự đoán một giá trị số từ các đặc trưng đầu vào.

Trong Assignment 02, ứng dụng dự đoán giá nhà là bài toán hồi quy. Các đặc điểm của căn nhà được sử dụng làm đầu vào và giá nhà là biến mục tiêu cần dự đoán.

2.3. Khái niệm biểu diễn dữ liệu

Dữ liệu thực tế có thể tồn tại dưới nhiều dạng khác nhau như dữ liệu dạng bảng, văn bản, hình ảnh, âm thanh hoặc các dạng dữ liệu khác. Tuy nhiên, mô hình học máy cần dữ liệu được chuyển thành một biểu diễn số phù hợp để có thể xử lý.

Lecture 02 giới thiệu nguyên tắc:

Dữ liệu thực tế → Biểu diễn số → Mô hình tính toán.

Do đó, biểu diễn dữ liệu không chỉ đơn giản là cách dữ liệu được lưu trong một tệp. Ví dụ, dữ liệu được lưu dưới dạng CSV có thể được chuyển thành vector hoặc ma trận trước khi đưa vào mô hình. Đối với dữ liệu có nhiều chiều, biểu diễn có thể sử dụng tensor.

Khi xây dựng biểu diễn dữ liệu, cần xác định các yếu tố chính gồm:

- Kích thước dữ liệu (Shape);
- Kiểu dữ liệu (Dtype);
- Miền giá trị (Range);
- Ý nghĩa của từng chiều;
- Cách mã hóa dữ liệu;
- Cách xử lý giá trị thiếu;
- Các bước tiền xử lý;
- Dữ liệu chính xác được đưa vào mô hình.

2.4. Vector đặc trưng

Một mẫu dữ liệu có thể được biểu diễn bằng một vector đặc trưng gồm nhiều giá trị số.

Dạng tổng quát: $\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d]^T \in \mathbb{R}^d$

Trong đó:

- $\mathbf{x}$ là vector đặc trưng của một mẫu;
- $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d$ là các đặc trưng của mẫu;
- d là số lượng đặc trưng;
- T biểu thị phép chuyển vị.

Ví dụ, một mẫu dữ liệu bệnh nhân có thể được biểu diễn bởi:

$\mathbf{x} = [\text{Age}, \text{BMI}, \text{Glucose}, \text{BloodPressure}, \dots]^T$

Tương tự, một căn nhà có thể được biểu diễn bởi:

$\mathbf{x} = [\text{Area}, \text{Bedroom}, \text{Bathroom}, \text{Location}, \dots]^T$

2.5. Ma trận đặc trưng

Khi có nhiều mẫu dữ liệu, các vector đặc trưng được kết hợp thành một ma trận đặc trưng.

Dạng tổng quát: $\mathbf{X} \in \mathbb{R}^{(N \times d)}$

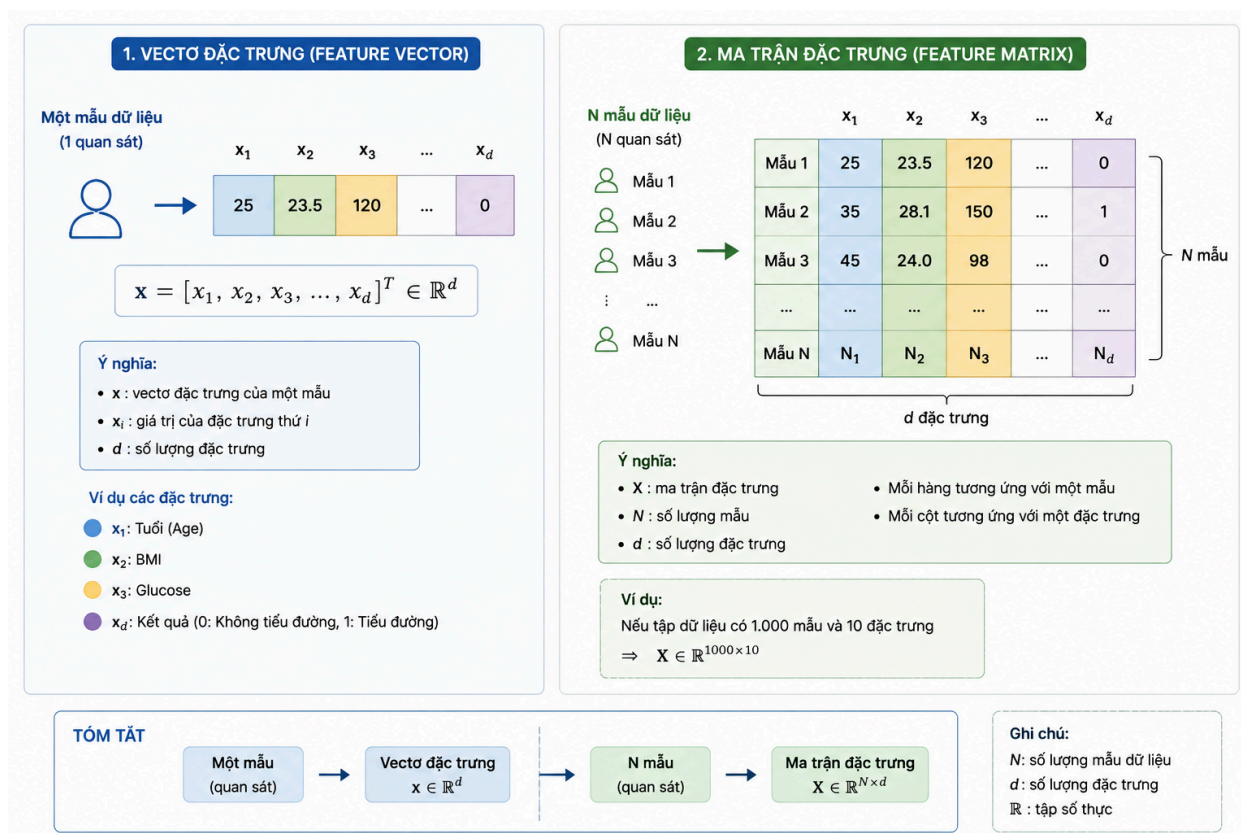
Trong đó:

- N là số lượng mẫu;
- d là số lượng đặc trưng;
- Mỗi hàng của $\mathbf{X}$ tương ứng với một mẫu;
- Mỗi cột của $\mathbf{X}$ tương ứng với một đặc trưng.

Ví dụ, nếu tập dữ liệu có 1.000 mẫu và 10 đặc trưng thì kích thước ma trận đặc trưng là:

$$\mathbf{X} \in \mathbb{R}^{(1000 \times 10)}$$

Điều này có nghĩa là tập dữ liệu có 1.000 mẫu, trong đó mỗi mẫu được biểu diễn bởi 10 đặc trưng.



Hình 2.1. Minh họa biểu diễn dữ liệu dưới dạng vector và ma trận.

2.6. Tensor và biểu diễn dữ liệu nhiều chiều

Tensor là dạng biểu diễn dữ liệu có thể chứa nhiều hơn hai chiều. Tensor được sử dụng khi cấu trúc dữ liệu cần nhiều chiều để thể hiện đầy đủ thông tin.

Ví dụ, với dữ liệu văn bản, nếu có **B** văn bản trong một batch, mỗi văn bản có **T** token và mỗi token được biểu diễn bằng một vector embedding có **d** chiều thì dữ liệu embedding có dạng: $\mathbf{E} \in \mathbb{R}^{(\mathbf{B} \times \mathbf{T} \times \mathbf{d})}$

Trong đó:

- **B**: số lượng văn bản trong một batch;
- **T**: số lượng token;
- **d**: số chiều của vector embedding.

2.7. Biểu diễn dữ liệu văn bản

Trong ứng dụng E-commerce, nếu bộ dữ liệu có chứa bình luận hoặc đánh giá của khách hàng, dữ liệu văn bản có thể được sử dụng để bổ sung thông tin cho các đặc trưng hành vi.

Quá trình chuyển đổi văn bản thành dữ liệu số có thể được mô tả:

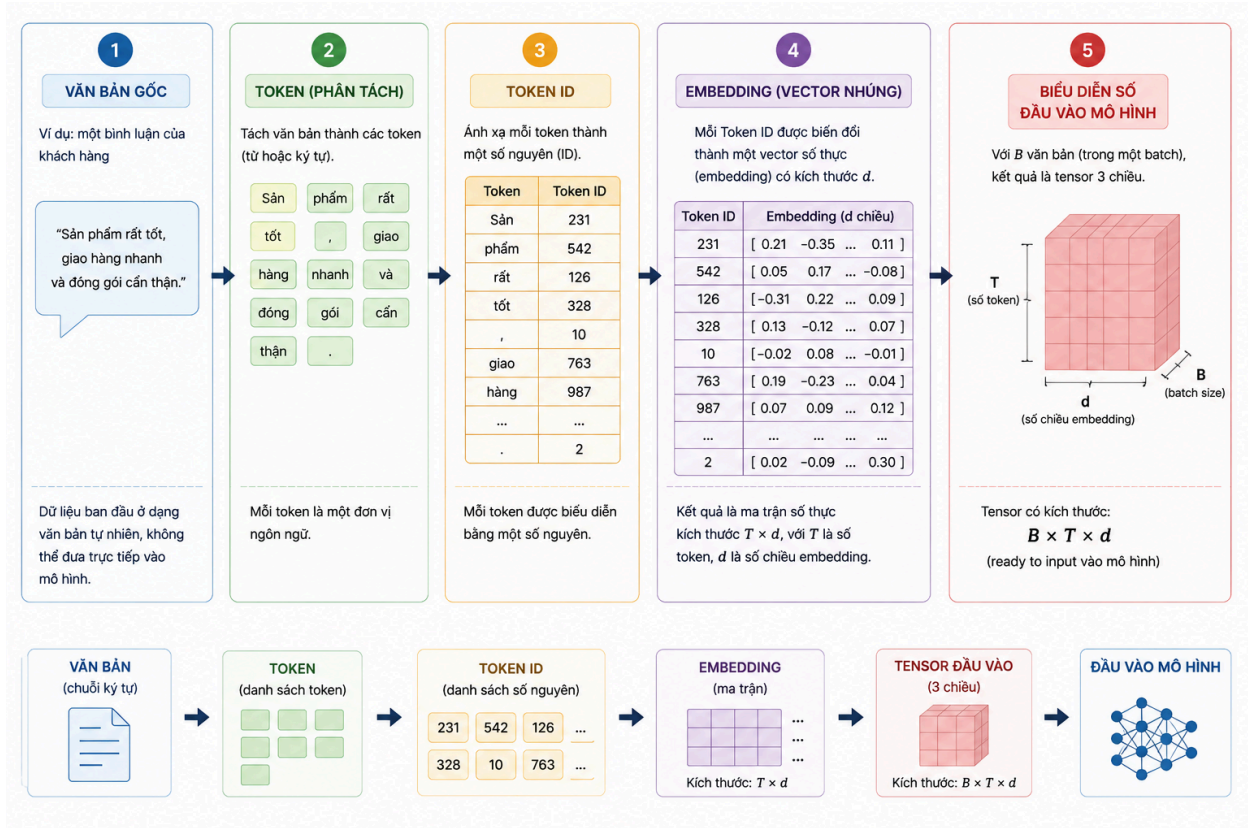
Văn bản $\rightarrow$ Token $\rightarrow$ Token ID $\rightarrow$ Embedding

Token: Là các đơn vị được tạo ra từ văn bản thông qua quá trình Tokenization. Tùy phương pháp được sử dụng, token có thể tương ứng với từ, một phần của từ hoặc các ký hiệu.

Token ID: Mỗi token được ánh xạ tới một số nguyên gọi là Token ID. Việc chuyển token thành ID giúp biểu diễn văn bản dưới dạng số để máy tính có thể xử lý.

Embedding: Embedding là một vector số có nhiều chiều được sử dụng để biểu diễn token trong không gian số.

Nếu sử dụng embedding cho nhiều văn bản, biểu diễn có thể có dạng: $\mathbf{E} \in \mathbb{R}^{(\mathbf{B} \times \mathbf{T} \times \mathbf{d})}$



Hình 2.2. Quy trình chuyển đổi dữ liệu văn bản thành biểu diễn số.

2.8. Mã hóa dữ liệu phân loại

Các đặc trưng dạng phân loại (Categorical Features) không thể được sử dụng trực tiếp trong nhiều mô hình học máy. Vì vậy, các giá trị phân loại cần được chuyển đổi thành dạng số.

Một số phương pháp phổ biến gồm:

- **Label Encoding:** ánh xạ các giá trị phân loại thành các số nguyên;
- **One-Hot Encoding:** tạo các biến nhị phân tương ứng với từng nhóm giá trị.

Sau khi mã hóa, số lượng đặc trưng có thể thay đổi. Vì vậy, cần xác định lại kích thước của ma trận đặc trưng sau quá trình mã hóa.

2.9. Kiểu dữ liệu và miền giá trị

Biểu diễn dữ liệu không chỉ phụ thuộc vào kích thước mà còn phụ thuộc vào kiểu dữ liệu và miền giá trị.

Các yếu tố cần kiểm tra gồm:

- Shape: kích thước dữ liệu;
- Dtype: kiểu dữ liệu;
- Range: miền giá trị;
- Encoding: phương pháp mã hóa;
- Ordering: thứ tự của các đặc trưng;
- Preprocessing: các bước tiền xử lý.

Ví dụ, hai ma trận có cùng kích thước nhưng nếu thứ tự các cột khác nhau thì ý nghĩa của dữ liệu đầu vào cũng khác nhau. Do đó, việc xác định chính xác biểu diễn dữ liệu là cần thiết để bảo đảm mô hình nhận đúng dữ liệu trong cả quá trình huấn luyện và dự đoán.

2.10. Tính nhất quán của biểu diễn dữ liệu

Biểu diễn dữ liệu phải được duy trì nhất quán từ quá trình huấn luyện đến quá trình triển khai.

Các bước tiền xử lý như mã hóa, chuẩn hóa hoặc xử lý giá trị thiếu được sử dụng trong quá trình huấn luyện phải được áp dụng giống nhau khi mô hình nhận dữ liệu mới.

Nếu quá trình triển khai sử dụng một cách tiền xử lý khác với quá trình huấn luyện, dữ liệu đầu vào có thể không còn cùng ý nghĩa với dữ liệu mà mô hình đã học.

2.11. Rò rỉ dữ liệu

Rò rỉ dữ liệu (Data Leakage) xảy ra khi thông tin không được phép từ tập kiểm tra hoặc dữ liệu tương lai được sử dụng trong quá trình huấn luyện hoặc tiền xử lý.

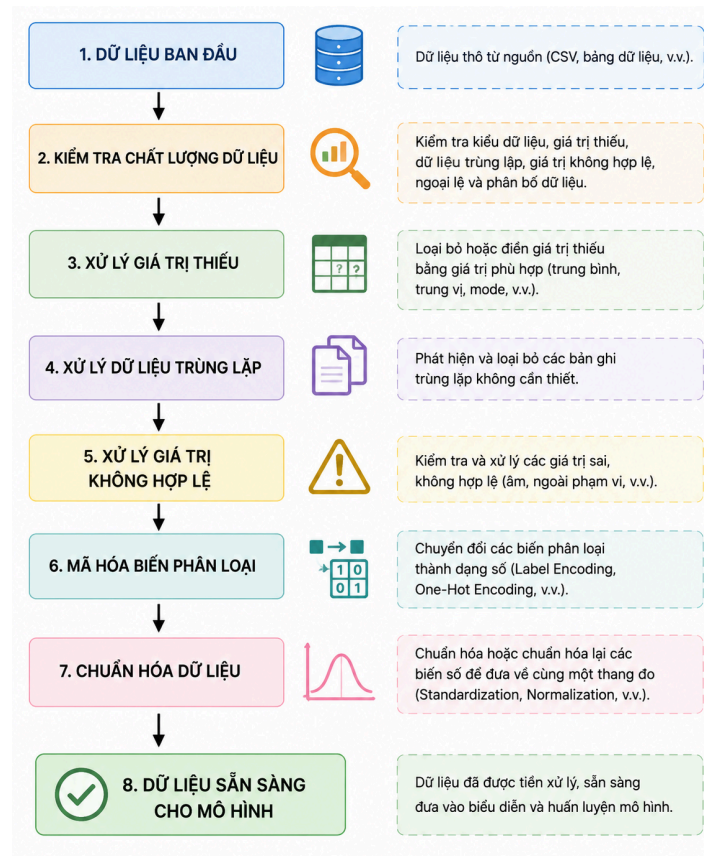
Ví dụ, nếu sử dụng toàn bộ dữ liệu để tính các tham số của một bộ chuẩn hóa trước khi chia thành tập huấn luyện và tập kiểm tra, thông tin từ tập kiểm tra đã ảnh hưởng đến quá trình huấn luyện.

Quy trình phù hợp là:

Chia dữ liệu → Huấn luyện preprocessing trên tập huấn luyện → Áp dụng preprocessing cho validation/test → Huấn luyện mô hình → Đánh giá.

Trong giai đoạn triển khai, hệ thống phải sử dụng preprocessing pipeline đã được học từ tập huấn luyện thay vì tạo một pipeline mới từ dữ liệu người dùng.

Chương III. Phương pháp tiền xử lý dữ liệu



Hình 3.1. Quy trình tiền xử lý dữ liệu

3.1. Tìm hiểu dữ liệu

Trước khi xây dựng mô hình, dữ liệu cần được kiểm tra để xác định cấu trúc và chất lượng của tập dữ liệu. Các nội dung cần kiểm tra bao gồm:

- Kích thước tập dữ liệu.
- Kiểu dữ liệu của từng cột.
- Giá trị thiếu.
- Dữ liệu trùng lặp.
- Giá trị không hợp lệ.
- Phân bố của biến mục tiêu.

Việc tìm hiểu dữ liệu giúp xác định các vấn đề cần xử lý trước khi đưa dữ liệu vào bước biểu diễn và huấn luyện mô hình.

Đối với mỗi ứng dụng trong Assignment 02, các thông tin trên sẽ được kiểm tra trực tiếp trên bộ dữ liệu được lựa chọn và kết quả sẽ được trình bày trong các chương ứng dụng

tương ứng.

3.2. Làm sạch dữ liệu

Làm sạch dữ liệu nhằm loại bỏ hoặc xử lý những vấn đề có thể ảnh hưởng đến quá trình huấn luyện mô hình.

3.2.1. Xử lý giá trị thiếu

Giá trị thiếu (Missing Value) xảy ra khi một trường dữ liệu không có giá trị. Nếu không được xử lý, giá trị thiếu có thể gây lỗi trong quá trình huấn luyện hoặc làm giảm chất lượng của mô hình.

Một số phương pháp xử lý gồm:

- Loại bỏ mẫu nếu số lượng giá trị thiếu không đáng kể.
- Thay thế bằng giá trị trung bình đối với một số biến số.
- Thay thế bằng trung vị khi dữ liệu có thể chứa ngoại lệ.
- Thay thế bằng giá trị phù hợp đối với biến phân loại.

Phương pháp cụ thể sẽ được lựa chọn dựa trên đặc điểm của từng bộ dữ liệu.

3.2.2. Xử lý dữ liệu trùng lặp

Dữ liệu trùng lặp (Duplicate Data) là các bản ghi xuất hiện nhiều lần nhưng không đại diện cho những quan sát khác nhau.

Các bản ghi trùng lặp cần được kiểm tra và xử lý vì chúng có thể làm thay đổi phân bố dữ liệu và ảnh hưởng đến quá trình huấn luyện mô hình.

Tuy nhiên, đối với dữ liệu giao dịch E-commerce, cần phân biệt giữa bản ghi trùng lặp thực sự và những giao dịch khác nhau của cùng một khách hàng. Không phải mọi bản ghi có cùng khách hàng đều được xem là dữ liệu trùng lặp.

3.2.3. Xử lý giá trị không hợp lệ

Giá trị không hợp lệ (Invalid Value) là những giá trị không phù hợp với ý nghĩa của đặc trưng.

Ví dụ:

- Giá trị tuổi âm.

- Diện tích nhà nhỏ hơn hoặc bằng 0.
- Số lượng sản phẩm âm.
- Giá sản phẩm không hợp lệ.
- Các giá trị phân loại không thuộc nhóm được định nghĩa.

Những giá trị này cần được kiểm tra trước khi huấn luyện để tránh đưa thông tin sai vào mô hình.

3.2.4. Phân tích ngoại lệ

Ngoại lệ (Outlier) là những giá trị khác biệt đáng kể so với phần lớn dữ liệu.

Ngoại lệ cần được phân tích thay vì tự động loại bỏ. Một giá trị lớn hoặc nhỏ bất thường có thể là lỗi nhập liệu, nhưng cũng có thể là một trường hợp thực tế.

Do đó, việc xử lý ngoại lệ cần dựa trên ý nghĩa của dữ liệu và mục tiêu của ứng dụng.

3.3. Mã hóa biến phân loại

Dữ liệu phân loại (Categorical Data) không thể được sử dụng trực tiếp bởi nhiều mô hình học máy nên cần được chuyển thành dạng số.

Hai phương pháp phổ biến là **Label Encoding** và **One-Hot Encoding**.

3.3.1. Label Encoding

Label Encoding ánh xạ mỗi giá trị phân loại thành một số nguyên.

Ví dụ:

- Nam $\rightarrow 0$
- Nữ $\rightarrow 1$

Phương pháp này phù hợp với một số trường hợp trong đó các giá trị có thể được biểu diễn bằng các nhãn số phù hợp.

3.3.2. One-Hot Encoding

One-Hot Encoding tạo một cột nhị phân cho mỗi nhóm giá trị.

Ví dụ, với đặc trưng Location có ba giá trị:

- Hà Nội.

- Đà Nẵng.
- Thành phố Hồ Chí Minh.

Sau khi mã hóa, có thể tạo thành ba đặc trưng:

- Location_HaNoi.
- Location_DaNang.
- Location_HoChiMinh.

Mỗi mẫu sẽ có giá trị 0 hoặc 1 tương ứng với nhóm của mẫu đó.

Sau quá trình encoding, số lượng đặc trưng có thể tăng lên. Vì vậy, cần xác định lại kích thước của ma trận đặc trưng.

3.4. Chuẩn hóa dữ liệu số

Các đặc trưng số có thể có những miền giá trị rất khác nhau. Ví dụ, tuổi có thể nằm trong khoảng vài chục trong khi giá nhà có thể có giá trị rất lớn.

Nếu một số mô hình nhạy với độ lớn của đặc trưng, sự khác biệt về miền giá trị có thể ảnh hưởng đến quá trình học. Vì vậy, các đặc trưng số có thể cần được chuẩn hóa hoặc đưa về cùng một miền giá trị.

Một phương pháp phổ biến là Standardization, trong đó dữ liệu được biến đổi dựa trên giá trị trung bình và độ lệch chuẩn của tập huấn luyện.

Việc chuẩn hóa phải được thực hiện bằng các tham số được học từ tập huấn luyện. Không được sử dụng tập kiểm tra để tính các tham số chuẩn hóa vì điều này có thể gây ra rò rỉ dữ liệu.

3.5. Chia tập dữ liệu

Sau khi xác định dữ liệu đầu vào và biến mục tiêu, dữ liệu được chia thành các tập phục vụ cho những mục đích khác nhau.

Có thể biểu diễn: $D = D_{\text{train}} \cup D_{\text{validation}} \cup D_{\text{test}}$

Trong đó:

- **D_train:** tập huấn luyện, được sử dụng để huấn luyện mô hình.
- **D_validation:** tập kiểm định, được sử dụng để lựa chọn hoặc điều chỉnh mô hình.
- **D_test:** tập kiểm tra, được sử dụng để đánh giá cuối cùng.

3.6. Ngăn ngừa rò rỉ dữ liệu

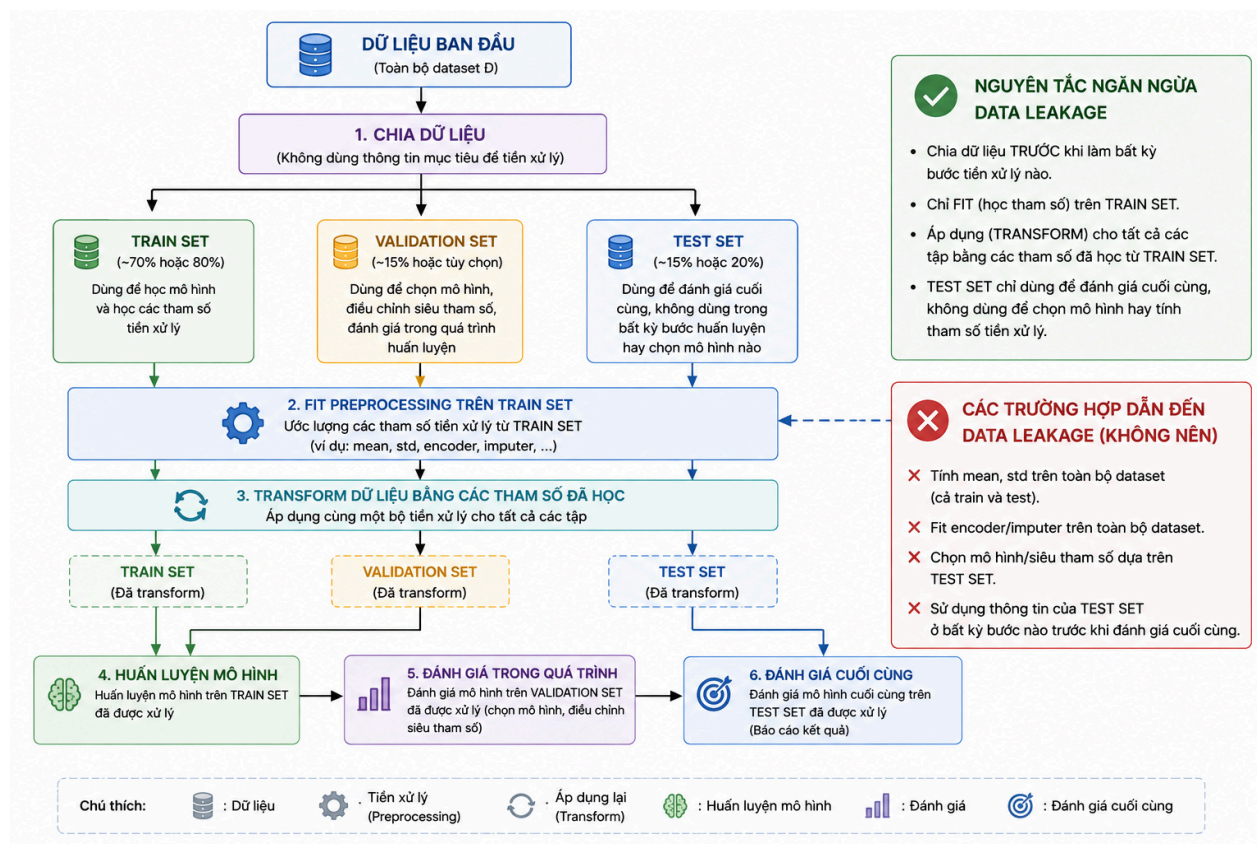
Rò rỉ dữ liệu (Data Leakage) xảy ra khi thông tin từ tập kiểm tra hoặc dữ liệu không được phép được sử dụng trong quá trình huấn luyện hoặc tiền xử lý.

Một số trường hợp có thể gây rò rỉ dữ liệu gồm:

- Chuẩn hóa dữ liệu bằng toàn bộ dataset trước khi chia train/test.
- Tính các tham số của bộ mã hóa từ cả tập train và test.
- Sử dụng thông tin của biến mục tiêu để tạo đặc trưng đầu vào.
- Sử dụng dữ liệu kiểm tra để lựa chọn mô hình.
- Tạo lại preprocessing dựa trên dữ liệu người dùng trong quá trình triển khai.

Quy trình đúng là: **Chia dữ liệu** → **Fit preprocessing trên tập train** → **Transform train/validation/test** → **Huấn luyện mô hình** → **Đánh giá trên test**.

Trong đó, các thành phần tiền xử lý như scaler, encoder hoặc imputer chỉ được học từ dữ liệu huấn luyện.



Hình 3.2. Quy trình chia dữ liệu và ngăn ngừa Data Leakage

3.7. Pipeline tiền xử lý

Để bảo đảm tính nhất quán, các bước tiền xử lý có thể được kết hợp thành một pipeline.

Một pipeline tổng quát: **Dữ liệu đầu vào** → **Xử lý giá trị thiếu** → **Mã hóa** → **Chuẩn hóa** → **Biểu diễn đặc trưng** → **Mô hình học máy**

Pipeline được sử dụng trong quá trình huấn luyện phải được lưu cùng với mô hình hoặc được lưu dưới dạng một thành phần riêng để có thể sử dụng lại trong quá trình dự đoán.

Điều này giúp bảo đảm rằng dữ liệu mới được xử lý giống với dữ liệu đã được sử dụng để huấn luyện mô hình.

3.8. Nguyên tắc tái lập quá trình tiền xử lý

Một hệ thống học máy có khả năng tái lập cần bảo đảm rằng cùng một quy trình tiền xử lý được sử dụng trong cả giai đoạn huấn luyện và triển khai.

Do đó, quá trình tiền xử lý cần được ghi nhận rõ ràng và có thể thực hiện lại khi cần thiết.

Chương IV. Các độ đo đánh giá mô hình

4.1. Đối với bài toán Hồi quy

Hệ thống sử dụng 5 độ đo tiêu chuẩn để lượng hóa mức độ sai lệch giữa giá nhà dự đoán và giá nhà thực tế:

- **R^2 Score (Hệ số xác định)**

Đo lường tỷ lệ phương sai của biến mục tiêu được giải thích bởi mô hình. Giá trị lý tưởng là 1. Mô hình càng xấp xỉ 1 thì dự đoán càng khớp với dữ liệu thực tế.

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

- **MAE (Mean Absolute Error - Trung bình sai số tuyệt đối)**

Tính trung bình trị tuyệt đối của các sai số. Độ đo này giúp đánh giá độ lệch trung bình mà không phạt quá nặng các giá trị ngoại lai. Mô hình có MAE càng nhỏ càng tốt.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **MSE (Mean Squared Error - Trung bình bình phương sai số)**

Tính trung bình bình phương các sai số. Do việc bình phương, độ đo này trừng phạt rất nặng các dự đoán sai lệch lớn. Giá trị càng nhỏ càng tốt.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **RMSE (Root Mean Squared Error - Căn bậc hai của MSE)**

Có cùng đơn vị đo lường với biến mục tiêu, giúp diễn giải sai số một cách trực quan hơn so với MSE.

$$RMSE = \sqrt{MSE}$$

4.2. Đối với bài toán Phân lớp

Vì dữ liệu chẩn đoán y tế có hiện tượng mất cân bằng lớp (bệnh nhân âm tính nhiều hơn dương tính), hệ thống không chỉ dùng độ chính xác tổng thể mà còn đánh giá qua Ma trận nhầm lẫn (Confusion Matrix) bao gồm: TP (Dương tính thật), TN (Âm tính thật), FB (Dương tính giả), và FN (Âm tính giả). Các độ đo phái sinh bao gồm:

- **Accuracy (Độ chính xác tổng thể):** Tỷ lệ tổng số dự đoán đúng trên tổng số ca bệnh.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision (Độ chuẩn xác):** Trong số các ca được mô hình chẩn đoán là mắc tiểu đường, có bao nhiêu ca thực sự mắc bệnh.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall/ Sensitivity (Độ phủ/ Độ nhạy):** Trong số tất cả các bệnh nhân thực sự mắc tiểu đường, mô hình phát hiện được bao nhiêu phần trăm. Trong y tế, độ đo này đặc biệt quan trọng vì việc bỏ sót bệnh nhân mắc bệnh (FN) gây nguy hiểm hơn nhiều so với việc chẩn đoán nhầm người khỏe (FP).

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score:** Là trung bình điều hòa của Precision và Recall. Độ đo này giúp đánh giá sự cân bằng của mô hình khi xử lý tập dữ liệu mất cân bằng.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- **ROC-AUC:** ROC-AUC có thể được sử dụng để đánh giá khả năng phân biệt giữa các lớp của mô hình khi phù hợp với bài toán. ROC-AUC càng cao cho thấy mô hình càng có khả năng phân biệt hai lớp tốt.
- **Confusion Matrix:** Confusion Matrix thể hiện số lượng dự đoán đúng và sai của mô hình theo từng lớp.

Chương V. Phương pháp xây dựng mô hình

5.1. Tổng quan về xây dựng mô hình

Sau khi dữ liệu đã được làm sạch, biểu diễn và chia thành các tập dữ liệu phù hợp, bước tiếp theo là xây dựng các mô hình học máy để thực hiện nhiệm vụ dự đoán.

Quá trình xây dựng mô hình gồm các bước chính:

- Xác định loại bài toán và biến mục tiêu.
- Xây dựng mô hình baseline.
- Huấn luyện nhiều mô hình học máy phù hợp.
- Đánh giá các mô hình trên cùng một tập dữ liệu.
- So sánh kết quả giữa các mô hình.
- Lựa chọn mô hình phù hợp nhất để triển khai.

Việc sử dụng nhiều mô hình giúp đánh giá liệu một phương pháp có thực sự phù hợp với dữ liệu hay không, thay vì chỉ sử dụng một mô hình duy nhất.

5.2. Mô hình Baseline

Baseline là mô hình cơ sở được sử dụng làm mốc để so sánh với các mô hình học máy phức tạp hơn.

Đối với bài toán phân loại, một baseline đơn giản có thể dự đoán lớp xuất hiện nhiều nhất trong dữ liệu. Đối với bài toán hồi quy, baseline có thể sử dụng một giá trị thống kê đơn giản để làm giá trị dự đoán.

Nếu các mô hình học máy không cải thiện đáng kể so với baseline, cần xem xét lại đặc trưng, phương pháp tiền xử lý hoặc lựa chọn mô hình.

5.3. Các mô hình cho bài toán hồi quy

- Mô hình Linear Regression:

- Dữ liệu biểu diễn tiếp nhận: Nhận ma trận đặc trưng đầu vào $X \in R^{N \times d}$ đã qua xử lý giá trị khuyết thiếu và chuẩn hóa thang đo bằng StandardScaler.
- Mối quan hệ cần học: Học mối quan hệ tuyến tính kết hợp tuyến tính giữa các đặc trưng đầu vào và biến mục tiêu liên tục ($y = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$).
- Tham số/ cấu trúc học được: Học vector trọng số (weights/ coefficients) w và hệ số tự do (bias/intercept) b .

- Tiêu chuẩn hướng dẫn học (Criterion): Phương pháp bình phương tối thiểu (Ordinary Least Squares - OLS) nhằm tối thiểu hóa tổng bình phương phần dư (Mean Squared Error).
- Giả định của mô hình: Giả định rằng mối quan hệ thực tế giữa các biến độc lập và biến mục tiêu mang tính tuyến tính, các đặc trưng độc lập với nhau và sai số có phân phối chuẩn.
- Ưu điểm: Tốc độ huấn luyện cực nhanh, dễ dàng diễn giải trọng số của từng đặc trưng.
- Nhược điểm: Dễ bị nhiễu và sụt giảm độ chính xác nghiêm trọng nếu dữ liệu thực tế tồn tại phi tuyến tính hoặc hiện tượng đa cộng tuyến cao.
- Mô hình Ridge Regression:

Ridge là biến thể của Linear Regression có bổ sung cơ chế regularization nhằm hạn chế việc mô hình quá phức tạp. Phương pháp này giúp kiểm soát ảnh hưởng của các đặc trưng và có thể cải thiện khả năng tổng quát hóa của mô hình.
- Mô hình Decision Tree Regressor:
 - Dữ liệu biểu diễn tiếp nhận: Ma trận đặc trưng thô hoặc đã chuẩn hóa (Cây quyết định không phụ thuộc vào scale của dữ liệu).
 - Mối quan hệ cần học: Học các quy tắc phân chia không gian đặc trưng thành các vùng chữ nhật vuông góc thông qua các câu hỏi điều kiện dạng phân tầng.
 - Tham số/ cấu trúc học được: Cấu trúc cây gồm các nút điều kiện (split nodes), ngưỡng chia đặc trưng và các giá trị lá (leaf values). Trong code giới hạn độ sâu `max_depth = 8`.
 - Tiêu chuẩn hướng dẫn học (Criterion): Tiêu chuẩn giảm thiểu phương sai (Variance Reduction / MSE) tại mỗi nút chia để tối ưu tính thuần nhất của dữ liệu con.
 - Giả định của mô hình: Không đặt ra các giả định khắt khe về phân phối tuyến tính của dữ liệu đầu vào.
 - Ưu điểm: Dễ dàng trực quan hóa, mô hình hóa được các mối quan hệ phi tuyến tính phức tạp và tự động chọn lọc đặc trưng.
 - Nhược điểm: Rất dễ bị hiện tượng quá khớp (overfitting) nếu không giới hạn độ sâu của cây.
- Mô hình Random Forest Regressor:
 - Dữ liệu biểu diễn tiếp nhận: Tập hợp ma trận đặc trưng X được lấy mẫu ngẫu nhiên theo cơ chế Bootstrap.
 - Mối quan hệ cần học: Học tổ hợp các mối quan hệ phi tuyến tính từ một quần thể gồm nhiều cây quyết định độc lập.

- Tham số / Cấu trúc học được: Một tập hợp gồm $n_{\text{estimators}} = 100$ cây quyết định kết hợp.
- Tiêu chuẩn hướng dẫn học (Criterion): Tiêu chuẩn giảm phương sai trên từng cây con và lấy trung bình cộng kết quả dự đoán tổng hợp (Ensemble Bagging).
- Giả định của mô hình: Giả định rằng việc kết hợp nhiều mô hình yếu (weak learners) độc lập sẽ triệt tiêu sai số và tạo ra một mô hình mạnh (strong learner).
- Ưu điểm: Khắc phục triệt để hiện tượng overfitting của cây đơn lẻ, mang lại độ chính xác tổng quát hóa cao và ổn định nhất trước các biến nhiễu.
- Nhược điểm: Tốn kém tài nguyên tính toán, thời gian huấn luyện lâu hơn và khó diễn giải trực quan chi tiết như một cây đơn lẻ.
- Gradient Boosting Regressor
 - Gradient Boosting Regressor xây dựng các mô hình cây tuần tự, trong đó mỗi mô hình mới tập trung cải thiện những sai số của các mô hình trước đó.
 - Phương pháp này có khả năng mô hình hóa các mối quan hệ phi tuyến phức tạp và thường cho hiệu năng tốt trên dữ liệu dạng bảng.

5.4. Các mô hình cho bài toán phân loại

- Mô hình Decision Tree:
 - Dữ liệu biểu diễn tiếp nhận: Ma trận đặc trưng thô hoặc đã chuẩn hóa (Cây quyết định không phụ thuộc vào scale của dữ liệu).
 - Mối quan hệ cần học: Học các quy tắc phân chia không gian đặc trưng thành các vùng chữ nhật vuông góc thông qua các câu hỏi điều kiện dạng phân tầng.
 - Tham số/ cấu trúc học được: Cấu trúc cây gồm các nút điều kiện (split nodes), ngưỡng chia đặc trưng và các giá trị lá (leaf values). Trong code giới hạn độ sâu $\text{max_depth} = 8$.
 - Tiêu chuẩn hướng dẫn học (Criterion): Tiêu chuẩn giảm thiểu phương sai (Variance Reduction / MSE) tại mỗi nút chia để tối ưu tính thuần nhất của dữ liệu con.
 - Giả định của mô hình: Không đặt ra các giả định khắt khe về phân phối tuyến tính của dữ liệu đầu vào.
 - Ưu điểm: Dễ dàng trực quan hóa, mô hình hóa được các mối quan hệ phi tuyến tính phức tạp và tự động chọn lọc đặc trưng.
 - Nhược điểm: Rất dễ bị hiện tượng quá khớp (overfitting) nếu không giới hạn độ sâu của cây.
- Mô hình Random Forest:

- Dữ liệu biểu diễn tiếp nhận: Tập hợp ma trận đặc trưng X được lấy mẫu ngẫu nhiên theo cơ chế Bootstrap.
- Mỗi quan hệ cần học: Học tổ hợp các mối quan hệ phi tuyến tính từ một quần thể gồm nhiều cây quyết định độc lập.
- Tham số / Cấu trúc học được: Một tập hợp gồm $n_estimators = 100$ cây quyết định kết hợp.
- Tiêu chuẩn hướng dẫn học (Criterion): Tiêu chuẩn giảm phương sai trên từng cây con và lấy trung bình cộng kết quả dự đoán tổng hợp (Ensemble Bagging).
- Giả định của mô hình: Giả định rằng việc kết hợp nhiều mô hình yếu (weak learners) độc lập sẽ triệt tiêu sai số và tạo ra một mô hình mạnh (strong learner).
- Ưu điểm: Khắc phục triệt để hiện tượng overfitting của cây đơn lẻ, mang lại độ chính xác tổng quát hóa cao và ổn định nhất trước các biến nhiễu.
- Nhược điểm: Tốn kém tài nguyên tính toán, thời gian huấn luyện lâu hơn và khó diễn giải trực quan chi tiết như một cây đơn lẻ.
- Mô hình K-Nearest Neighbors (KNN)
 - Dữ liệu biểu diễn tiếp nhận: Không gian vector đặc trưng X đã được chuẩn hóa (Bắt buộc vì KNN tính khoảng cách dựa trên trị số tuyệt đối).
 - Mỗi quan hệ cần học: Học cấu trúc lân cận cục bộ; giả định rằng các quan sát có đặc trưng hình học gần nhau sẽ có giá trị mục tiêu (giá nhà) tương tự nhau.
 - Tham số/ cấu trúc học được: Bản chất KNN là mô hình phi tham số (non-parametric), không học trọng số cố định mà lưu trữ toàn bộ ma trận dữ liệu huấn luyện và sử dụng tham số siêu định lượng k .
 - Tiêu chuẩn hướng dẫn học (Criterion): Khoảng cách Euclidean (hoặc Manhattan) để tìm ra k điểm láng giềng gần nhất.
 - Giả định của mô hình: Giả định không gian bài toán liên tục cục bộ, các đặc trưng đều có mức độ quan trọng ngang nhau đối với khoảng cách không gian.
 - Ưu điểm: Không cần thời gian huấn luyện (lazy learning), dễ thích ứng với dữ liệu phân phối phức tạp.
 - Nhược điểm: Tốc độ dự đoán chậm khi tập dữ liệu lớn và cực kỳ nhạy cảm với hiện tượng số chiều lớn (curse of dimensionality).
- Mô hình Logistic Regression
 - Dữ liệu biểu diễn tiếp nhận: Không gian vector đặc trưng X đã qua chuẩn hóa thang đo.
 - Mỗi quan hệ cần học: Học xác suất để một bệnh nhân thuộc về lớp dương

tính (mắc bệnh tiểu đường) dựa trên hàm Sigmoid ($\sigma(z) = \frac{1}{1+e^{-z}}$).

- Tham số/ cấu trúc học được: Bản chất là mô hình tuyến tính, học vector trọng số w và hệ số tự do b .
 - Tiêu chuẩn hướng dẫn học (Criterion): Tối thiểu hóa hàm mất mát Log-Loss (Binary Cross-Entropy) thông qua thuật toán Gradient Descent.
 - Giả định của mô hình: Giả định các quan sát độc lập với nhau và tồn tại mối quan hệ tuyến tính giữa log-odds (logarit của tỷ lệ xác suất) và các biến đầu vào.
 - Ưu điểm: Tính toán cực kỳ nhanh, kết quả dự đoán trả về dưới dạng xác suất giúp dễ dàng diễn giải ý nghĩa y khoa.
 - Nhược điểm: Rất khó phân chia các tập dữ liệu có ranh giới phi tuyến tính phức tạp nếu không có khâu tạo thêm đặc trưng đa thức (polynomial features).
- Mô hình Support Vector Machine
 - Dữ liệu biểu diễn tiếp nhận: Vector đặc trưng X bắt buộc phải được chuẩn hóa (do SVM tính toán khoảng cách vector).
 - Mối quan hệ cần học: Tìm kiếm một siêu phẳng (hyperplane) trong không gian d chiều giúp phân tách rạch ròi hai lớp Khỏe mạnh và Mắc bệnh.
 - Tham số/ cấu trúc học được: Vector pháp tuyến w , hệ số b và các điểm dữ liệu đóng vai trò là "Support Vectors".
 - Tiêu chuẩn hướng dẫn học (Criterion): Tối đa hóa lề (Margin Maximization) kết hợp với hàm phạt Hinge Loss để xử lý các điểm phân loại sai.
 - Giả định của mô hình: Giả định rằng dữ liệu có thể phân tách tuyến tính (hoặc xấp xỉ tuyến tính với dải dung sai mềm).
 - Ưu điểm: Hiệu quả trong không gian nhiều chiều, bộ nhớ tối ưu do chỉ lưu các điểm Support Vectors.
 - Nhược điểm: Không tính toán trực tiếp xác suất phân lớp và hiệu năng giảm mạnh khi tập dữ liệu có quá nhiều nhiễu chéo.

Chương VI. Phương pháp lưu trữ và triển khai mô hình

6.1. Tổng quan

Sau khi lựa chọn được mô hình phù hợp, mô hình cần được lưu trữ để có thể sử dụng lại mà không phải huấn luyện lại từ đầu. Tiếp theo, mô hình được tích hợp vào một dịch vụ dự đoán để người dùng có thể gửi dữ liệu đầu vào và nhận kết quả.

Quá trình triển khai cần đảm bảo rằng dữ liệu đầu vào khi dự đoán được xử lý giống với dữ liệu đã sử dụng trong quá trình huấn luyện.

Quy trình tổng quát được sử dụng trong Assignment 02 là:

User Input → Validation → Preprocessing → Saved Model → Prediction → Result.

6.2. Lưu trữ mô hình

Mô hình sau khi được huấn luyện cần được lưu thành file để có thể sử dụng trong giai đoạn triển khai.

Một hệ thống hoàn chỉnh cần lưu trữ các thành phần cần thiết của quá trình học máy, bao gồm:

- Pipeline tiền xử lý dữ liệu.
- Các phép biến đổi đặc trưng.
- Mô hình đã được huấn luyện.
- Cấu hình của mô hình khi cần thiết.

Assignment yêu cầu mô hình cuối cùng phải có khả năng được sử dụng lại mà không cần huấn luyện lại.

Trong Python, thư viện như joblib có thể được sử dụng để lưu và tải pipeline hoặc mô hình.

Việc lưu cả preprocessing pipeline cùng với mô hình giúp đảm bảo quá trình xử lý dữ liệu khi triển khai giống với quá trình đã sử dụng khi huấn luyện.

6.3. Tính nhất quán giữa huấn luyện và dự đoán

Một yêu cầu quan trọng của hệ thống triển khai là preprocessing được sử dụng trong quá trình dự đoán phải giống với preprocessing được sử dụng trong quá trình huấn luyện.

Ví dụ, nếu dữ liệu huấn luyện được xử lý bằng một scaler hoặc encoder cụ thể, hệ thống

triển khai phải sử dụng chính thành phần đã được học từ dữ liệu huấn luyện đó.

Không được tạo một scaler, encoder hoặc imputer mới dựa trên dữ liệu test hoặc dữ liệu người dùng trong quá trình triển khai. Đây là một trong những nguyên tắc quan trọng để tránh Data Leakage và đảm bảo tính nhất quán của hệ thống.

6.4. Triển khai mô hình dưới dạng Web Service

Mô hình có thể được triển khai dưới dạng một web service để các ứng dụng khác gửi yêu cầu dự đoán.

Có thể sử dụng Flask, FastAPI hoặc một REST framework phù hợp khác để xây dựng dịch vụ.

Một API dự đoán thường thực hiện các bước:

- Nhận dữ liệu đầu vào từ client.
- Kiểm tra tính hợp lệ của dữ liệu.
- Áp dụng preprocessing pipeline đã lưu.
- Đưa dữ liệu sau xử lý vào mô hình đã lưu.
- Thực hiện dự đoán.
- Trả kết quả về cho client.

Ví dụ, đối với bài toán phân loại, API có thể trả về lớp dự đoán và xác suất hoặc độ tin cậy khi phù hợp.

Đối với bài toán hồi quy, API có thể trả về giá trị dự đoán như giá nhà.

6.5. Web Application

Web application đóng vai trò là giao diện để người dùng tương tác với mô hình.

Ứng dụng web cần cung cấp các thành phần cơ bản:

- Form nhập dữ liệu.
- Các control phù hợp với kiểu dữ liệu.
- Kiểm tra dữ liệu đầu vào.
- Nút thực hiện dự đoán.
- Kết quả dự đoán.
- Xác suất hoặc độ tin cậy khi phù hợp.
- Giải thích ngắn gọn về kết quả.

Web application không cần huấn luyện lại mô hình. Thay vào đó, ứng dụng phải tải mô hình và preprocessing pipeline đã được lưu để thực hiện inference trên dữ liệu mới.

6.6. Triển khai trên ứng dụng mobile

Ứng dụng mobile cung cấp giao diện thuận tiện để người dùng nhập thông tin và nhận kết quả dự đoán.

Mobile application không nhất thiết phải chạy trực tiếp mô hình học máy trên thiết bị. Một kiến trúc phù hợp là để mobile gửi yêu cầu đến REST API, sau đó API thực hiện preprocessing và prediction.

Quy trình có thể được mô tả như sau:

Mobile UI → REST API → Preprocessing → ML Model → Prediction → Mobile UI.

Ứng dụng mobile cần cho phép người dùng:

- Nhập hoặc lựa chọn các thông tin đầu vào.
- Gửi yêu cầu dự đoán.
- Nhận kết quả dự đoán.
- Hiển thị giải thích hoặc giá trị xác suất khi phù hợp.

6.7. Kiến trúc hoàn chỉnh của hệ thống

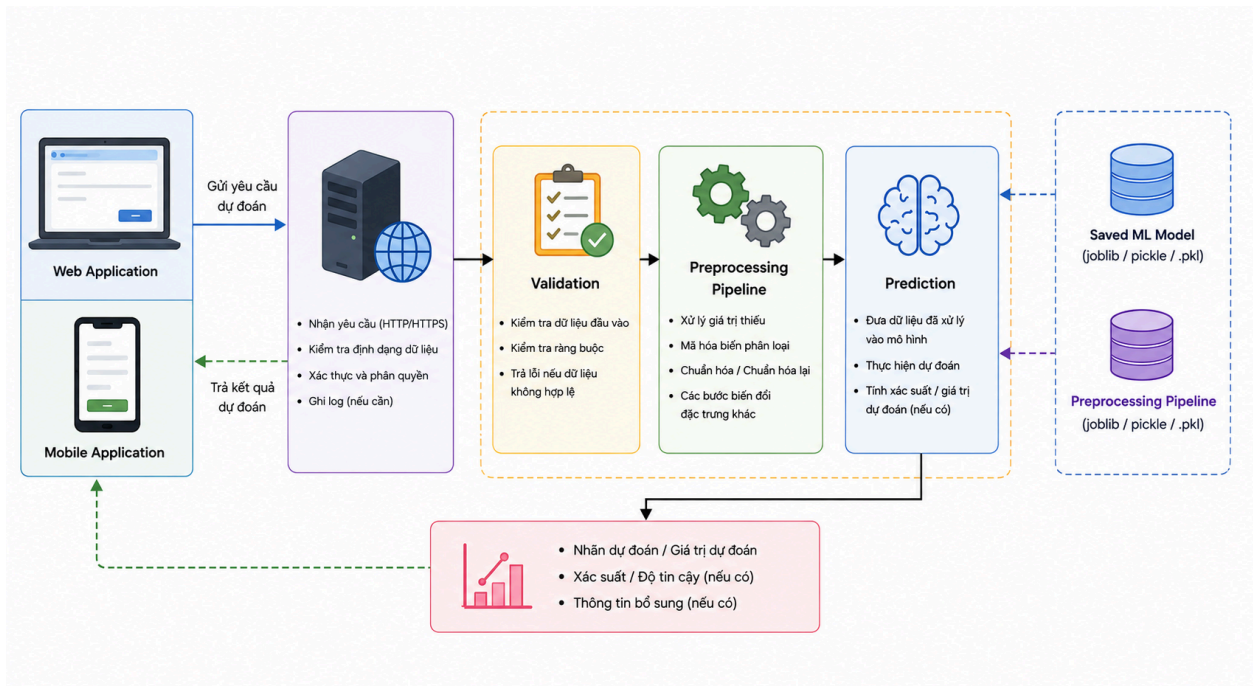
Một hệ thống thông minh hoàn chỉnh kết hợp giữa mô hình học máy và các thành phần phần mềm để cung cấp dịch vụ dự đoán.

Kiến trúc tổng quát có thể được mô tả:

User → Web/Mobile UI → API → Input Validation → Preprocessing → Saved ML Model → Prediction → Web/Mobile UI.

Trong kiến trúc này:

- Web hoặc mobile application tiếp nhận thông tin từ người dùng.
- API tiếp nhận và kiểm tra dữ liệu.
- Preprocessing pipeline chuyển đổi dữ liệu về đúng dạng mà mô hình yêu cầu.
- Mô hình đã lưu thực hiện dự đoán.
- API trả kết quả về cho ứng dụng.
- Giao diện hiển thị kết quả cho người dùng.



Hình 6.1. Kiến trúc triển khai hệ thống thông minh.

Chương VII. Ứng dụng 1 - Dự đoán bệnh tiểu đường

Mã nguồn toàn bộ dự án: <https://github.com/dthyfu/Assignment02.git>

7.1. Mô tả bài toán

Bài toán Diabetes Prediction nhằm dự đoán một bệnh nhân có thuộc nhóm mắc bệnh tiểu đường hay không dựa trên các đặc trưng về nhân khẩu học và sức khỏe.

Đây là bài toán **phân loại nhị phân**, trong đó diabetes = 0 biểu thị không mắc bệnh tiểu đường và diabetes = 1 biểu thị thuộc nhóm mắc bệnh tiểu đường.

Mục tiêu là xây dựng một quy trình hoàn chỉnh từ dữ liệu thô đến mô hình có khả năng thực hiện dự đoán trên dữ liệu mới.

7.2. Giới thiệu tập dữ liệu

Tập dữ liệu được sử dụng cho ứng dụng Diabetes Prediction là **Diabetes Prediction Dataset**. Dữ liệu được lưu dưới dạng file CSV và gồm **100.000 quan sát** với **8 đặc trưng đầu vào** và **1 biến mục tiêu**.

Link dataset: <https://www.kaggle.com/datasets/ghnshymsaini/diabetes-prediction-dataset>

```
print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
display(df.head())
```

Dataset shape: (100000, 9)

First 5 rows:

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	Female	80.0	0	1	never	25.19	6.6	140	0
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
2	Male	28.0	0	0	never	27.32	5.7	158	0
3	Female	36.0	0	0	current	23.45	5.0	155	0
4	Male	76.0	1	1	current	20.14	4.8	155	0

Hình 7.1. Kích thước và một số bản ghi đầu tiên của tập dữ liệu Diabetes Prediction.

Mỗi dòng dữ liệu đại diện cho **một bệnh nhân**, trong đó mỗi cột biểu diễn một đặc trưng hoặc nhân của bệnh nhân.

Đặc trưng	Loại	Ý nghĩa
-----------	------	---------

gender	Categorical	Giới tính của bệnh nhân.
age	Numerical	Tuổi của bệnh nhân.
hypertension	Numerical	Tình trạng tăng huyết áp, gồm 0 hoặc 1.
heart_disease	Numerical	Tình trạng bệnh tim, gồm 0 hoặc 1.
smoking_history	Categorical	Tiền sử hút thuốc.
bmi	Numerical	Chỉ số khối cơ thể.
HbA1c_level	Numerical	Mức HbA1c.
blood_glucose_level	Numerical	Mức đường huyết.
diabetes	Target	Nhãn tiểu đường, gồm 0 và 1.

Trong đó:

X = tập các đặc trưng đầu vào của bệnh nhân.

y = lớp tiểu đường.

Các đặc trưng số gồm age, hypertension, heart_disease, bmi, HbA1c_level và blood_glucose_level. Hai đặc trưng phân loại là gender và smoking_history.

7.3. Khảo sát và tìm hiểu dữ liệu

```
print("Dataset information:")
df.info()

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 100000 non-null  str
1   age                   100000 non-null  float64
2   hypertension           100000 non-null  int64
3   heart_disease          100000 non-null  int64
4   smoking_history        100000 non-null  str
5   bmi                   100000 non-null  float64
6   HbA1c_level            100000 non-null  float64
7   blood_glucose_level    100000 non-null  int64
8   diabetes               100000 non-null  int64
dtypes: float64(3), int64(4), str(2)
memory usage: 8.0 MB
```

Hình 7.2. Thông tin cấu trúc và kiểu dữ liệu của tập dữ liệu.

Tập dữ liệu ban đầu gồm 100.000 dòng và 9 cột. Trong đó có 6 thuộc tính số, 2 thuộc tính phân loại và 1 biến mục tiêu diabetes.

```
missing_values = df.isna().sum()

print("Missing values per column:")
display(missing_values)

Missing values per column:
gender                0
age                  0
hypertension          0
heart_disease         0
smoking_history        0
bmi                   0
HbA1c_level           0
blood_glucose_level    0
diabetes              0
dtype: int64
```

Hình 7.3. Kết quả kiểm tra giá trị thiếu

```
duplicate_count = df.duplicated().sum()

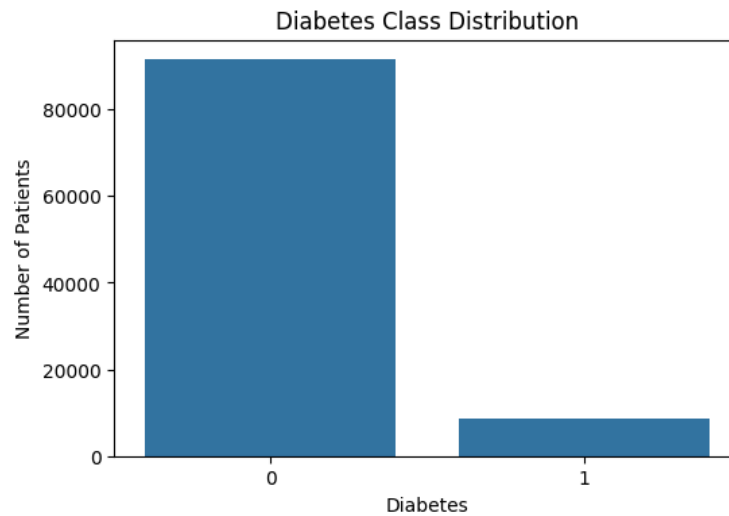
print("Number of duplicated records:", duplicate_count)
print("Duplicate percentage:", duplicate_count / len(df) * 100, "%")

Number of duplicated records: 3854
Duplicate percentage: 3.8539999999999996 %
```

Hình 7.4. Kết quả kiểm tra bản ghi trùng lặp

Kết quả kiểm tra cho thấy tập dữ liệu không có giá trị thiếu. Tuy nhiên, có 3.854 bản ghi

trùng lặp, chiếm khoảng 3,85% tổng số bản ghi.



Hình 7.5. Phân bố biến mục tiêu trước khi làm sạch dữ liệu.

Về phân bố lớp, trước khi làm sạch dữ liệu, lớp diabetes = 0 có 91.500 mẫu (91,5%), trong khi lớp diabetes = 1 có 8.500 mẫu (8,5%). Điều này cho thấy dữ liệu có sự mất cân bằng lớp, do số lượng bệnh nhân thuộc lớp 0 lớn hơn đáng kể so với lớp 1.

7.4. Làm sạch dữ liệu

Quá trình làm sạch dữ liệu được thực hiện nhằm loại bỏ các vấn đề có thể ảnh hưởng đến chất lượng dữ liệu và kết quả huấn luyện mô hình.

Xử lý giá trị thiếu:

```
missing_values = df.isna().sum()

print("Missing values per column:")
display(missing_values)
```

Missing values per column:

gender	0
age	0
hypertension	0
heart_disease	0
smoking_history	0
bmi	0
HbA1c_level	0
blood_glucose_level	0
diabetes	0

dtype: int64

Hình 7.6. Kết quả kiểm tra giá trị thiếu trong tập dữ liệu

Kết quả kiểm tra cho thấy dữ liệu không có giá trị thiếu nên không cần thực hiện thay thế

hoặc loại bỏ dữ liệu.

Xử lý bản ghi trùng lặp

```
df_clean = df.drop_duplicates().copy()
print("Cleaned dataset shape:", df_clean.shape)

Cleaned dataset shape: (96146, 9)
```

Hình 7.7. Kết quả loại bỏ bản ghi trùng lặp

Từ kết quả kiểm tra, tập dữ liệu có 3.854 bản ghi trùng lặp, chiếm 3,85%. Các bản ghi trùng lặp được loại bỏ bằng phương thức `drop_duplicates()` nhằm tránh việc một mẫu dữ liệu xuất hiện nhiều lần trong quá trình huấn luyện mô hình. Sau khi xử lý, dữ liệu còn 96.146 bản ghi.

Xử lý giá trị không hợp lệ

```
print("Invalid value analysis")
print("-" * 50)

invalid_checks = {
    "age <= 0": (X["age"] <= 0).sum(),
    "age > 100": (X["age"] > 100).sum(),
    "bmi <= 0": (X["bmi"] <= 0).sum(),
    "HbA1c_level <= 0": (X["HbA1c_level"] <= 0).sum(),
    "blood_glucose_level <= 0": (X["blood_glucose_level"] <= 0).sum(),
}

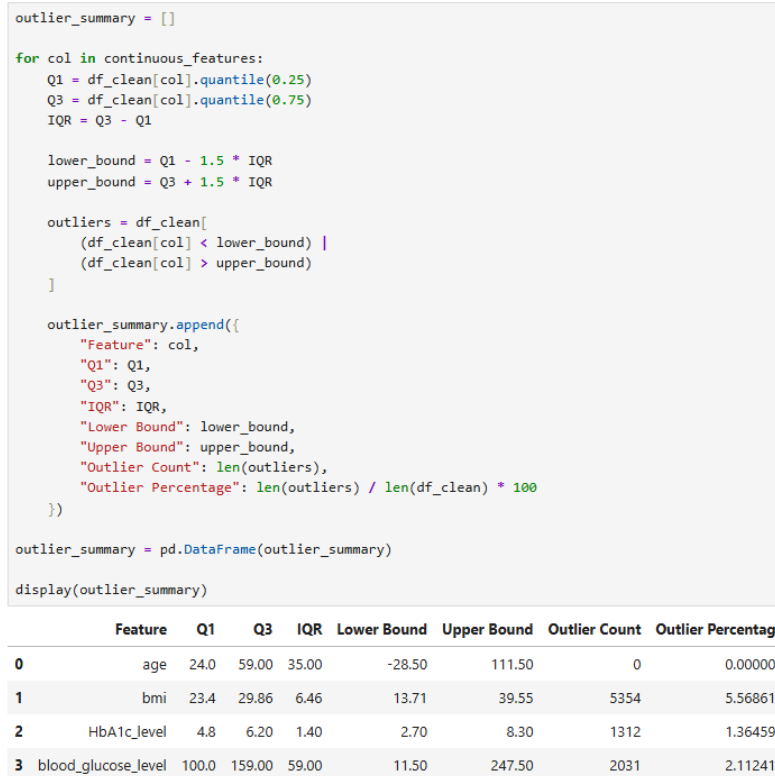
for condition, count in invalid_checks.items():
    print(f"{condition}: {count}")

Invalid value analysis
-----
age <= 0: 0
age > 100: 0
bmi <= 0: 0
HbA1c_level <= 0: 0
blood_glucose_level <= 0: 0
```

Hình 7.8. Kết quả kiểm tra các giá trị không hợp lệ

Các thuộc tính nhị phân hypertension, heart_disease và diabetes được kiểm tra để đảm bảo chỉ chứa giá trị 0 hoặc 1. Các thuộc tính age, bmi, HbA1c_level và blood_glucose_level cũng được kiểm tra các giá trị không hợp lệ. Kết quả không phát hiện giá trị cần loại bỏ.

Phân tích giá trị ngoại lệ



Hình 7.9. Thống kê giá trị ngoại lệ theo phương pháp IQR

Phương pháp IQR được sử dụng để phát hiện ngoại lệ trong các thuộc tính liên tục. Một số giá trị ngoại lệ được phát hiện ở bmi, HbA1c_level và blood_glucose_level. Tuy nhiên, các giá trị này không bị loại bỏ tự động vì chúng có thể phản ánh những trường hợp thực tế trong dữ liệu. Do đó, các giá trị ngoại lệ được giữ lại để tránh làm mất thông tin.

Xử lý thuộc tính phân loại

Hai thuộc tính phân loại gender và smoking_history được giữ lại để đưa vào bước mã hóa. Việc mã hóa được thực hiện ở bước biểu diễn dữ liệu bằng phương pháp One-Hot Encoding.

7.5. Biểu diễn dữ liệu

Ma trận đặc trưng ban đầu

```
print("Feature matrix shape:", X.shape)

N, d = X.shape

print(f"N (number of observations): {N:,}")
print(f"d (number of input features): {d}")
```

Feature matrix shape: (96146, 8)
 N (number of observations): 96,146
 d (number of input features): 8

Hình 7.10. Kích thước ma trận đặc trưng đầu vào của tập dữ liệu

Ma trận đặc trưng có kích thước 96.146×8 , trong đó $N = 96.146$ là số lượng quan sát và $d = 8$ là số lượng thuộc tính đầu vào của mỗi quan sát.

Kiểu dữ liệu của các đặc trưng

```
print("Data types of input features:")
display(X.dtypes)
```

Data types of input features:

gender	str
age	float64
hypertension	int64
heart_disease	int64
smoking_history	str
bmi	float64
HbA1c_level	float64
blood_glucose_level	int64
dtype:	object

Hình 7.11. Kiểu dữ liệu của các thuộc tính đầu vào

Ma trận đặc trưng ban đầu X gồm 8 thuộc tính đầu vào với hai dạng dữ liệu chính. Các thuộc tính age, hypertension, heart_disease, bmi, HbA1c_level và blood_glucose_level là các thuộc tính dạng số. Các thuộc tính gender và smoking_history là các thuộc tính dạng phân loại.

Phương pháp biến đổi dữ liệu

```

preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numerical_features
        ),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

```

```

X_train_processed = preprocessor.fit_transform(X_train)

X_val_processed = preprocessor.transform(X_val)

X_test_processed = preprocessor.transform(X_test)

```

Hình 7.12. Cấu hình tiền xử lý dữ liệu bằng StandardScaler và One-Hot Encoding.

Kích thước dữ liệu sau tiền xử lý

```

print("Processed feature matrix shapes:")
print(f"X_train_processed: {X_train_processed.shape}")
print(f"X_val_processed: {X_val_processed.shape}")
print(f"X_test_processed: {X_test_processed.shape}")

```

```

Processed feature matrix shapes:
X_train_processed: (67302, 15)
X_val_processed: (14422, 15)
X_test_processed: (14422, 15)

```

Hình 7.13. Kích thước dữ liệu sau quá trình chuẩn hóa và mã hóa

Tập huấn luyện có kích thước 67.302×15 , tập validation có kích thước 14.422×15 và tập test có kích thước 14.422×15 . Số lượng quan sát giữa các tập được giữ nguyên so với quá trình chia dữ liệu trước đó, trong khi số lượng đặc trưng tăng từ 8 lên 15 do quá trình One-Hot Encoding các thuộc tính phân loại.

Cụ thể, 6 thuộc tính số sau khi chuẩn hóa vẫn tương ứng với 6 đặc trưng. Hai thuộc tính phân loại gender và smoking_history được chuyển thành tổng cộng 9 đặc trưng nhị phân. Do đó, dữ liệu cuối cùng có 15 đặc trưng đầu vào cho mỗi quan sát.

Vector đầu vào cuối cùng của mô hình

```

processed_sample = X_train_processed[0]

print("One processed patient feature vector:")
print(processed_sample)

print("\nVector dimension:")
print(processed_sample.shape)

```

One processed patient feature vector:

```

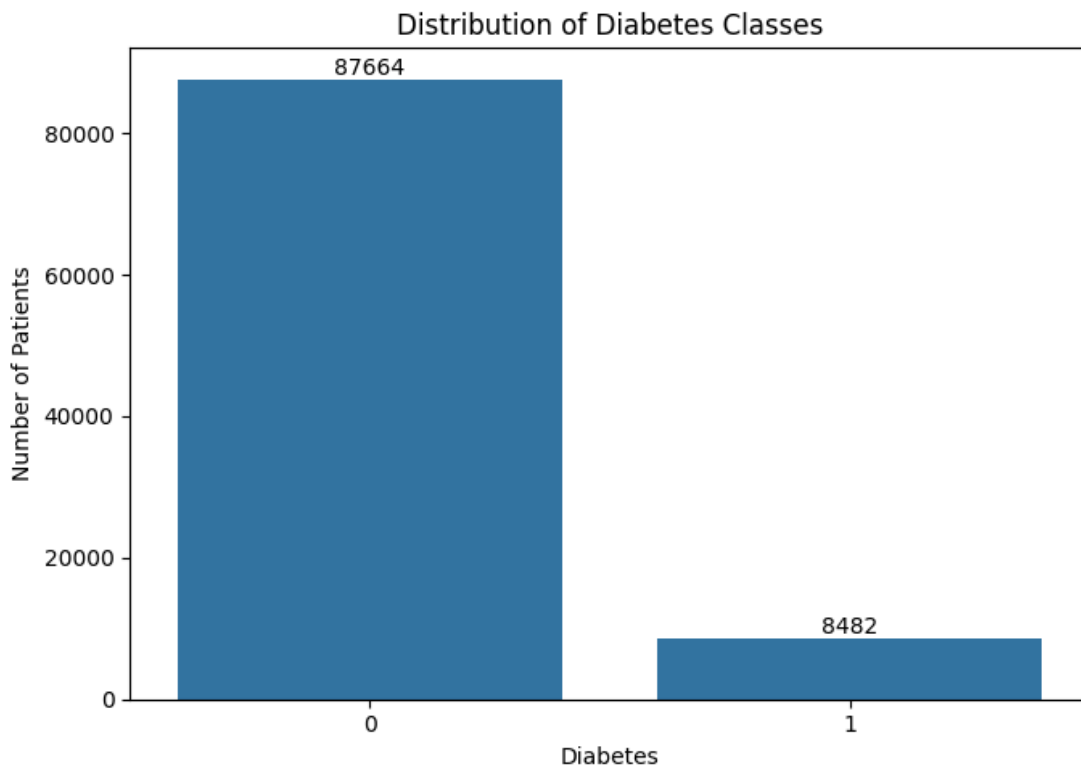
[ 0.81280525 -0.29075169 -0.20636173 -0.24238738  0.43742462 -1.17574744
  0.          1.          0.          0.          0.          0.
  0.          1.          0.          ]

```

Hình 7.14. Vector đặc trưng 15 chiều của một quan sát sau tiền xử lý.

Kết quả kiểm tra một quan sát trong tập huấn luyện cho thấy vector đầu vào có dạng một mảng số với kích thước (15,). Điều này cho thấy mỗi bệnh nhân được biểu diễn thống nhất bằng 15 đặc trưng số sau bước tiền xử lý.

7.6. Phân tích khám phá dữ liệu (EDA)



Hình 7.15. Phân bố biến mục tiêu sau khi làm sạch dữ liệu

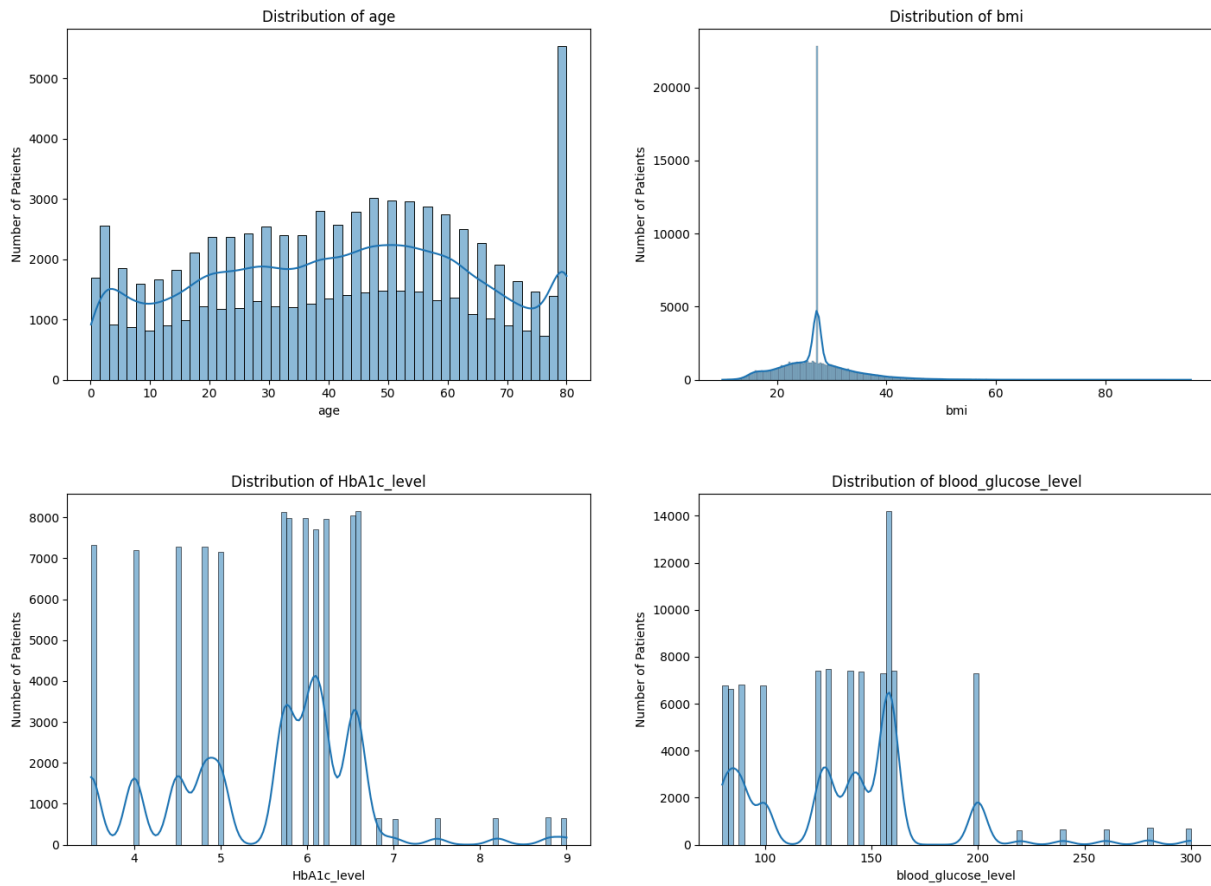
Biểu đồ cho thấy tập dữ liệu có hai lớp diabetes:

- Lớp 0: 87.664 bệnh nhân, chiếm khoảng 91,18%.
- Lớp 1: 8.482 bệnh nhân, chiếm khoảng 8,82%.

Observation: Lớp 0 chiếm tỷ lệ rất lớn, trong khi lớp 1 chỉ chiếm khoảng 8,82%, cho thấy dữ liệu bị mất cân bằng.

Interpretation: Số lượng mẫu giữa hai lớp chênh lệch đáng kể, đặc biệt lớp 1 là lớp thiểu số.

ML implication: Không nên chỉ sử dụng Accuracy để đánh giá mô hình. Cần quan tâm thêm Precision, Recall, F1-score và ROC-AUC để đánh giá khả năng nhận diện lớp 1.

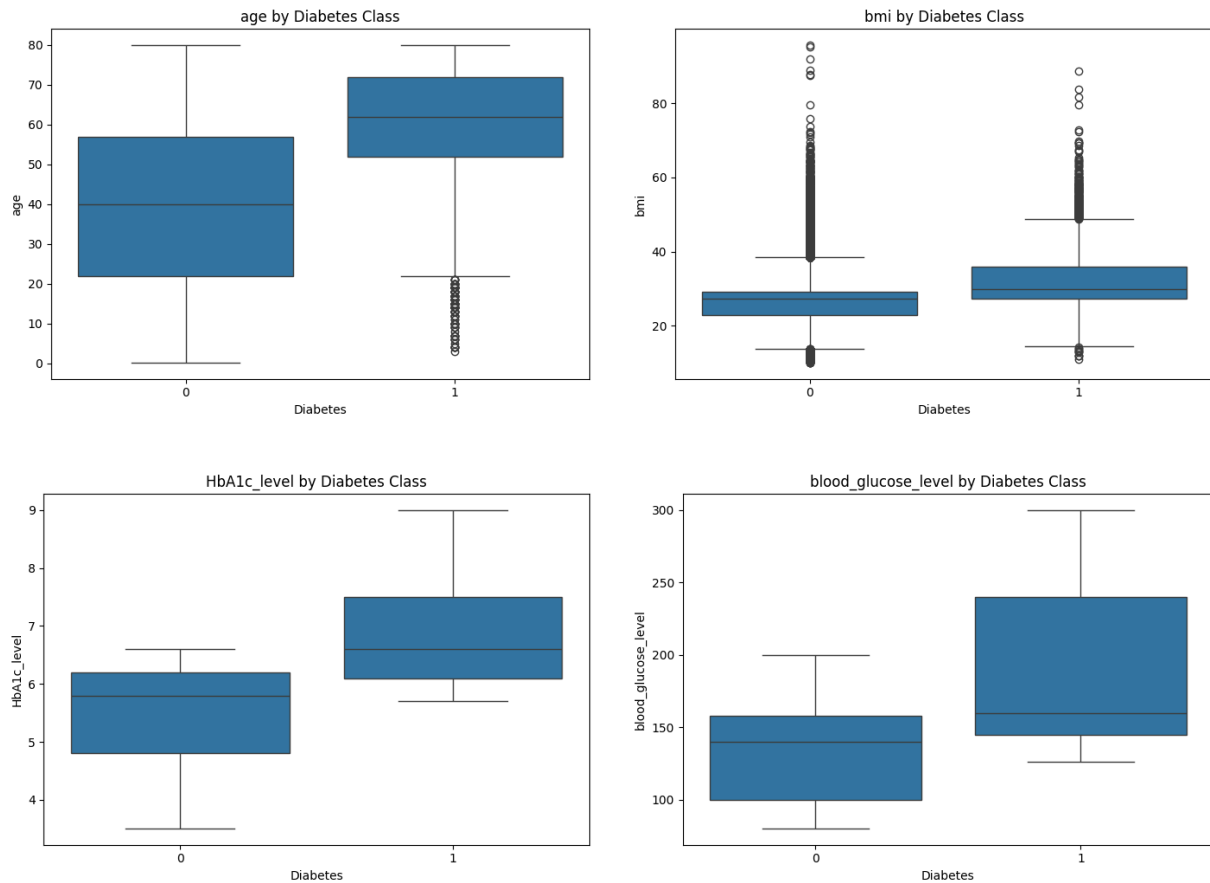


Hình 7.16. Phân bố của các thuộc tính số trong tập dữ liệu

Observation: age phân bố trên khoảng rộng, trong khi HbA1c_level và blood_glucose_level tập trung trong những khoảng giá trị nhất định. bmi có độ phân tán tương đối lớn.

Interpretation: Các thuộc tính số có đặc điểm phân bố khác nhau và có sự khác biệt về thang đo.

ML implication: Việc chuẩn hóa các thuộc tính số bằng StandardScaler là cần thiết, đặc biệt đối với các mô hình nhạy với thang đo như Logistic Regression, SVM và KNN.

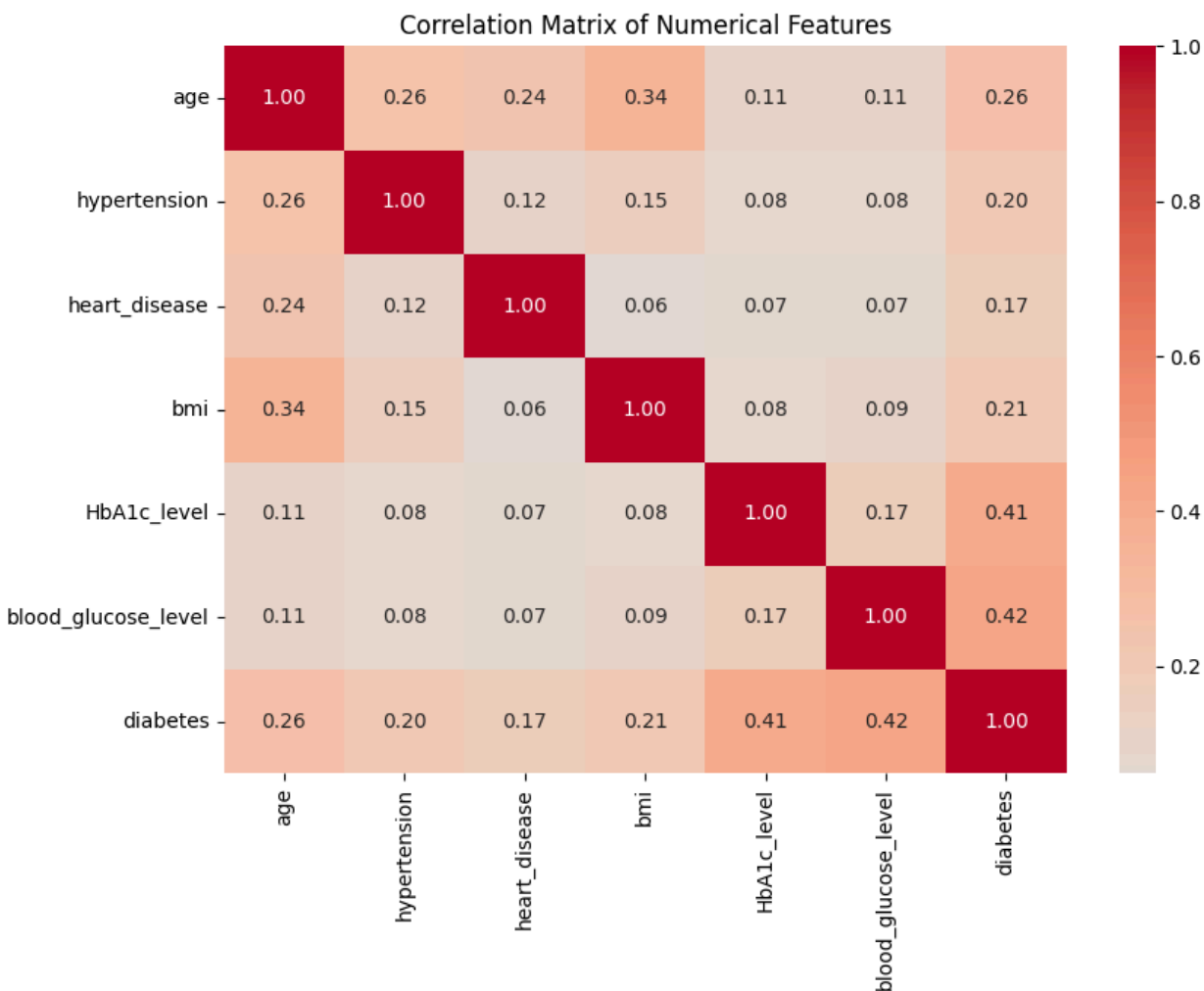


Hình 7.17. Phân bố các thuộc tính số theo biến diabetes

Observation: Nhóm diabetes = 1 có xu hướng có giá trị HbA1c_level và blood_glucose_level cao hơn nhóm diabetes = 0. Các thuộc tính age và bmi cũng có sự khác biệt nhất định giữa hai nhóm.

Interpretation: HbA1c_level và blood_glucose_level có sự phân tách tương đối rõ giữa hai lớp, trong khi các thuộc tính còn lại có mức độ chồng lấn lớn hơn.

ML implication: Các thuộc tính có khả năng phân biệt hai lớp, đặc biệt HbA1c_level và blood_glucose_level, có thể đóng vai trò quan trọng trong quá trình huấn luyện mô hình.

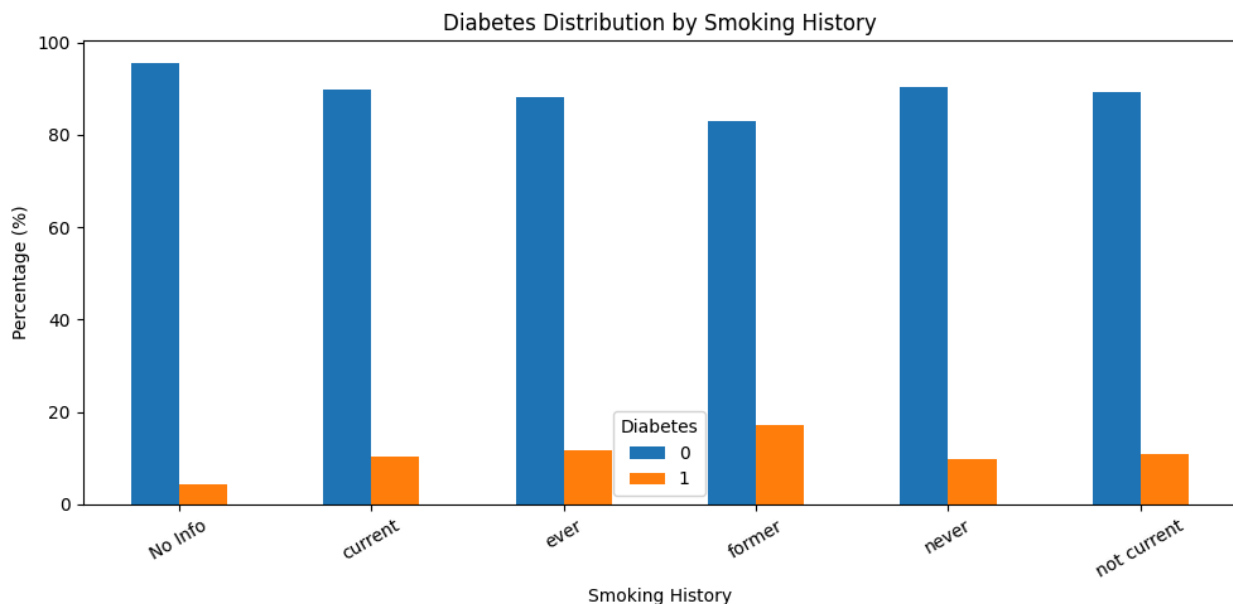


Hình 7.18. Ma trận tương quan giữa các thuộc tính số và biến mục tiêu

Observation: blood_glucose_level và HbA1c_level có hệ số tương quan với diabetes cao hơn các thuộc tính còn lại.

Interpretation: Hai thuộc tính này có mối liên hệ đáng chú ý với biến mục tiêu trong tập dữ liệu.

ML implication: Các thuộc tính có tương quan cao với diabetes có thể cung cấp thông tin hữu ích cho mô hình trong quá trình phân loại.



Hình 7.19. Phân bố diabetes theo lịch sử hút thuốc

Biểu đồ thể hiện tỷ lệ phân bố diabetes theo từng nhóm trong thuộc tính `smoking_history`. Tỷ lệ giữa các nhóm lịch sử hút thuốc có sự khác nhau, cho thấy thuộc tính này có thể cung cấp thông tin hữu ích cho việc phân loại.

Observation: Các nhóm `smoking_history` có tỷ lệ diabetes khác nhau.

Interpretation: Lịch sử hút thuốc có sự khác biệt nhất định giữa các nhóm có và không có diabetes.

ML implication: `smoking_history` được giữ lại làm đặc trưng đầu vào và được mã hóa bằng One-Hot Encoding trước khi huấn luyện mô hình.

7.7. Xây dựng mô hình

7.7.1. Chia tập Train, Validation và Test

Tập dữ liệu sau tiền xử lý được chia thành ba tập gồm Train, Validation và Test theo tỷ lệ 70% – 15% – 15%. Tập Train được sử dụng để huấn luyện mô hình, tập Validation được sử dụng để so sánh và lựa chọn mô hình, trong khi tập Test được giữ riêng để đánh giá cuối cùng. Quá trình chia dữ liệu sử dụng phương pháp stratification nhằm duy trì tỷ lệ giữa hai lớp diabetes trong các tập dữ liệu.

```

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=RANDOM_SEED,
    stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=RANDOM_SEED,
    stratify=y_temp
)

print("Dataset split:")
print("-" * 40)

print(f"Training set: {X_train.shape}")
print(f"Validation set: {X_val.shape}")
print(f"Test set: {X_test.shape}")

print("\nTarget shapes:")
print(f"y_train: {y_train.shape}")
print(f"y_val: {y_val.shape}")
print(f"y_test: {y_test.shape}")

Dataset split:
-----
Training set: (67302, 8)
Validation set: (14422, 8)
Test set: (14422, 8)

Target shapes:
y_train: (67302,)
y_val: (14422,)
y_test: (14422,)

```

Hình 7.20. Kết quả chia tập dữ liệu Train, Validation và Test

Kết quả:

- Tập Train: 67.302 mẫu.
- Tập Validation: 14.422 mẫu.
- Tập Test: 14.422 mẫu.

7.7.2. Xây dựng các mô hình phân loại

```
models = {  
    "Logistic Regression": LogisticRegression(  
        max_iter=1000,  
        random_state=RANDOM_SEED  
    ),  
  
    "KNN": KNeighborsClassifier(  
        n_neighbors=5  
    ),  
  
    "Decision Tree": DecisionTreeClassifier(  
        random_state=RANDOM_SEED  
    ),  
  
    "Random Forest": RandomForestClassifier(  
        n_estimators=200,  
        random_state=RANDOM_SEED,  
        n_jobs=-1  
    ),  
  
    "SVM": SVC(  
        probability=True,  
        random_state=RANDOM_SEED  
    )  
}
```

Hình 7.21. Các mô hình phân loại được sử dụng trong bài toán dự đoán diabetes

7.7.3. Baseline và huấn luyện mô hình

```
baseline = DummyClassifier(  
    strategy="most_frequent"  
)  
  
start_time = time.time()  
  
baseline.fit(X_train_processed, y_train)  
  
baseline_training_time = time.time() - start_time  
  
baseline_pred = baseline.predict(X_val_processed)  
baseline_prob = baseline.predict_proba(X_val_processed)[: , 1]
```

Hình 7.22. Xây dựng mô hình Baseline

```

trained_models = {}
validation_results = []

for model_name, model in models.items():

    print(f"Training {model_name}...")

    start_time = time.time()

    model.fit(
        X_train_processed,
        y_train
    )

    training_time = time.time() - start_time

    y_val_pred = model.predict(X_val_processed)

    y_val_prob = model.predict_proba(
        X_val_processed
    )[:, 1]

    metrics = calculate_classification_metrics(
        y_val,
        y_val_pred,
        y_val_prob
    )

    metrics["Model"] = model_name
    metrics["Training Time (s)"] = training_time

    validation_results.append(metrics)

    trained_models[model_name] = model

    print(
        f"{model_name} completed "
        f"in {training_time:.2f} seconds."
    )

```

Hình 7.23. Huấn luyện 5 mô hình

7.8. Đánh giá mô hình

	Model	Accuracy	Precision	Recall	F1	ROC-AUC	Training Time (s)
0	Dummy Classifier	0.9117	0.0000	0.0000	0.0000	0.5000	0.0028
1	Random Forest	0.9672	0.9435	0.6685	0.7825	0.9557	2.6269
2	SVM	0.9596	0.9688	0.5601	0.7098	0.9239	261.6430
3	Logistic Regression	0.9562	0.8589	0.6025	0.7082	0.9557	0.0958
4	KNN	0.9569	0.8817	0.5915	0.7080	0.8905	0.3621
5	Decision Tree	0.9472	0.6925	0.7235	0.7076	0.8468	0.3113

Hình 7.24. So sánh hiệu quả các mô hình trên tập Validation

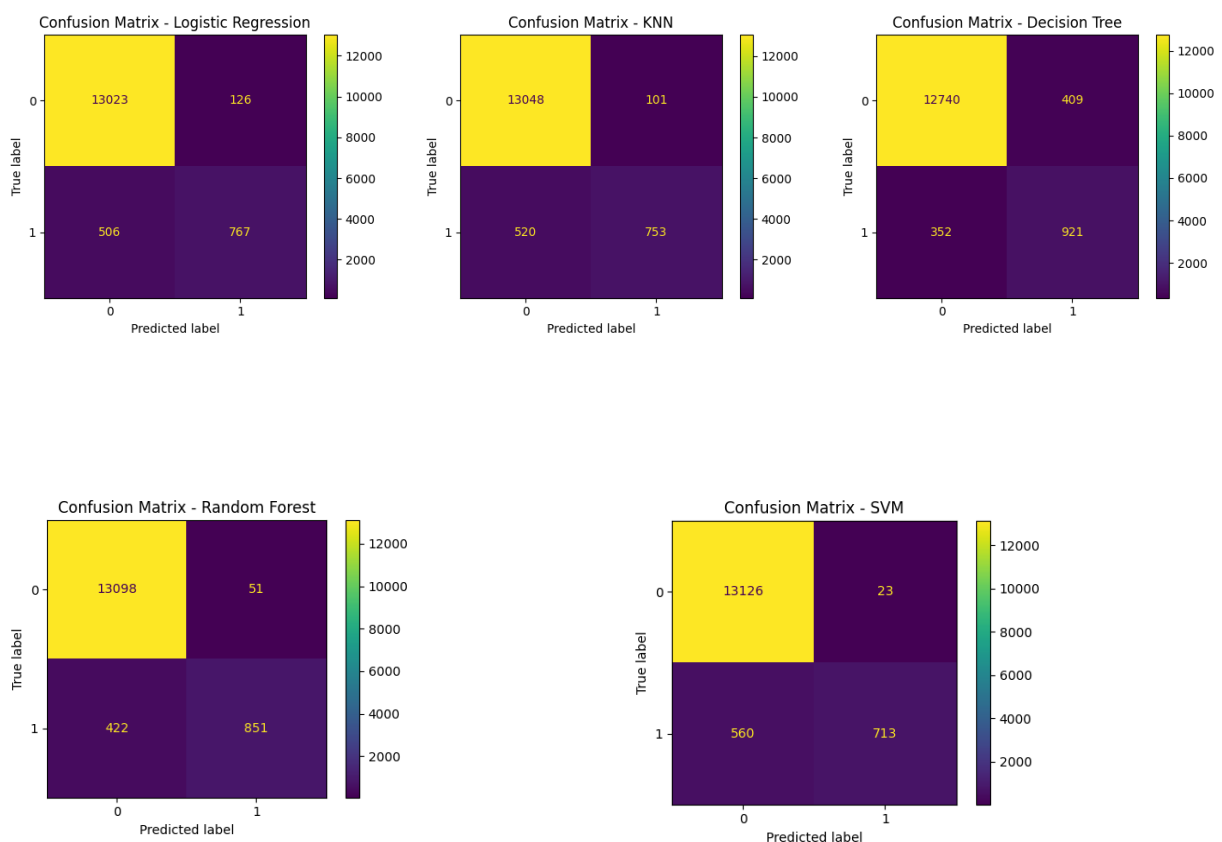
Kết quả trên tập Validation cho thấy cả năm mô hình học máy đều cải thiện đáng kể so với Baseline. Random Forest đạt Accuracy 0,9672 và F1-score 0,7825, cao nhất

trong các mô hình được so sánh. Do dữ liệu mất cân bằng, F1-score được ưu tiên trong quá trình lựa chọn mô hình thay vì chỉ dựa vào Accuracy.

Observation: Random Forest đạt F1-score cao nhất, trong khi SVM đạt Precision cao nhất nhưng Recall thấp hơn.

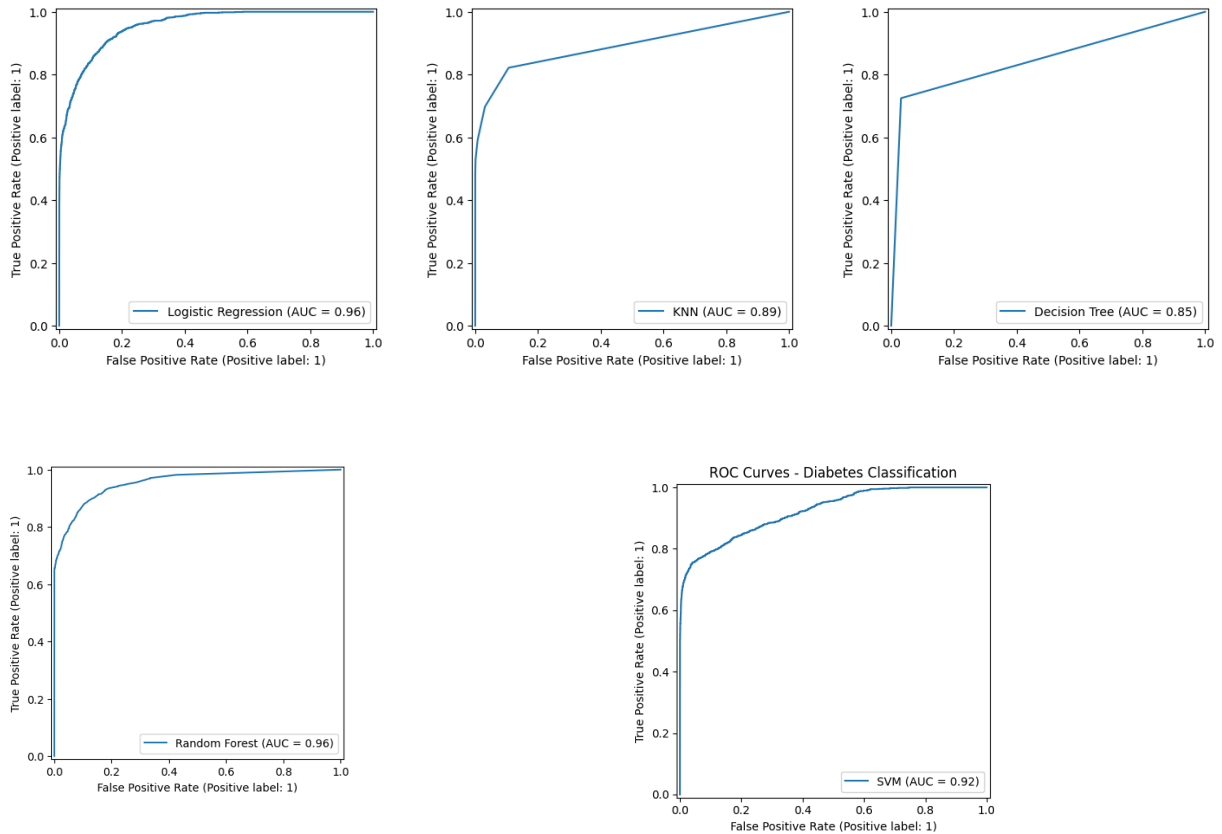
Interpretation: Random Forest tạo được sự cân bằng tốt hơn giữa Precision và Recall so với các mô hình còn lại.

ML implication: Random Forest được lựa chọn làm mô hình tốt nhất trên tập Validation.



Hình 7.25. Confusion Matrix của các mô hình trên tập Validation

Confusion Matrix được sử dụng để phân tích chi tiết kết quả phân loại của từng mô hình trên tập Validation. Kết quả cho thấy các mô hình có khả năng nhận diện cả hai lớp, tuy nhiên số lượng dự đoán sai giữa lớp 0 và lớp 1 có sự khác biệt.



Hình 7.26. ROC Curve của các mô hình trên tập Validation

ROC Curve được sử dụng để đánh giá khả năng phân biệt hai lớp của các mô hình ở nhiều ngưỡng dự đoán khác nhau. Random Forest và Logistic Regression có ROC-AUC cao nhất trong nhóm mô hình được thử nghiệm, đạt khoảng 0,956 trên tập Validation.

7.9. Lựa chọn mô hình

	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	0.971	0.9495	0.7091	0.8119	0.9617

Hình 7.27. Kết quả đánh giá mô hình Random Forest trên tập Test

Sau khi lựa chọn Random Forest trên tập Validation, mô hình được đánh giá cuối cùng trên tập Test. Kết quả cho thấy mô hình đạt Accuracy 0,9710, Precision 0,9495, Recall 0,7091, F1-score 0,8119 và ROC-AUC 0,9617. Kết quả cho thấy mô hình có khả năng phân loại tốt trên tập dữ liệu chưa được sử dụng trong quá trình lựa chọn mô hình.

Observation: Random Forest đạt Accuracy và Precision cao, đồng thời F1-score đạt 0,8119 và ROC-AUC đạt 0,9617.

Interpretation: Mô hình duy trì hiệu quả tốt khi đánh giá trên tập Test, cho thấy kết quả trên Validation không bị suy giảm đáng kể khi áp dụng trên dữ liệu chưa được sử dụng để

lựa chọn mô hình.

ML implication: Random Forest phù hợp để sử dụng làm mô hình cuối cùng cho hệ thống dự đoán diabetes.

Error Analysis

```
-----
Total samples:      14,422
Correct predictions: 14,004
Incorrect predictions: 418
False positives:    48
False negatives:    370
```

Hình 7.28. Kết quả phân tích lỗi trên tập Test

Phân tích lỗi trên tập Test cho thấy mô hình Random Forest dự đoán đúng 14.004 trên tổng số 14.422 mẫu và có 418 dự đoán sai. Trong đó, có 48 trường hợp False Positive và 370 trường hợp False Negative. Số lượng False Negative cao hơn False Positive, cho thấy mô hình vẫn còn một số trường hợp thuộc lớp 1 nhưng dự đoán thành lớp 0.

7.10. Triển khai hệ thống

Diabetes Prediction
Enter patient information to predict diabetes status.

Gender
Female

Age
29

Hypertension
No

Heart Disease
No

Smoking History
Never

BMI
120

HbA1c Level
6.6

Blood Glucose Level
85

Predict Diabetes Reset

Prediction Result
Prediction: No Diabetes
Probability of diabetes: 4.50%
The model predicts that the patient does not belong to the diabetes class.

Hình 7.29. Deploy dạng web

00:37

Diabetes Prediction

Enter patient information to predict diabetes status.

Gender

Female

Age

29

Hypertension

No

Heart Disease

No

Smoking History

never

BMI

150

HbA1c Level

6

Blood Glucose Level

85

Predict Diabetes

Reset

Prediction Result

Prediction: No Diabetes

Probability of diabetes: 0.04%

The model predicts that the patient does not belong to the diabetes class.

00:37

Hypertension

No

Heart Disease

No

Smoking History

never

BMI

150

HbA1c Level

6

Blood Glucose Level

85

Predict Diabetes

Reset

Prediction Result

Prediction: No Diabetes

Probability of diabetes: 0.04%

The model predicts that the patient does not belong to the diabetes class.

Hình 7.30. Deploy dạng mobile bằng expo go

Chương VIII. Ứng dụng 2 - Dự đoán giá nhà

8.1. Mô tả bài toán

Mục tiêu của ứng dụng là dự đoán giá của một căn nhà dựa trên các đặc điểm của bất động sản. Biến mục tiêu là Price, đại diện cho giá nhà. Các thuộc tính còn lại được sử dụng làm đặc trưng đầu vào cho mô hình. Đây là bài toán hồi quy vì giá nhà là một biến có giá trị liên tục.

8.2. Giới thiệu tập dữ liệu

Tập dữ liệu được sử dụng cho ứng dụng dự đoán giá nhà được lưu dưới dạng CSV và được đọc vào DataFrame bằng thư viện Pandas.

Link dataset:

<https://www.kaggle.com/datasets/chershi/house-price-prediction-dataset-2000-rows>

```
import pandas as pd
import numpy as np

DATA_PATH = "enhanced_house_price_dataset.csv"

df = pd.read_csv(DATA_PATH)

print("Dataset loaded successfully.")
print(f"Shape: {df.shape}")
```

Dataset loaded successfully.
Shape: (2000, 16)

Hình 8.1. Kích thước của tập dữ liệu House Price.

Đặc trưng	Loại	Ý nghĩa
Area	Numerical	Diện tích của căn nhà.
Bedrooms	Numerical	Số lượng phòng ngủ của căn nhà.
Bathrooms	Numerical	Số lượng phòng tắm của căn nhà.

Stories	Numerical	Số tầng của căn nhà.
Parking	Numerical	Số lượng vị trí đỗ xe.
Location	Categorical	Khu vực hoặc địa điểm của căn nhà.
Condition	Categorical	Tình trạng của căn nhà.
Age	Numerical	Tuổi của căn nhà.
Locality Rating	Numerical	Mức đánh giá của khu vực xung quanh căn nhà.
Price	Numerical – Target	Giá của căn nhà, là biến mục tiêu cần dự đoán.

X = tập các đặc trưng của căn nhà.

y = giá nhà.

8.3. Khảo sát và tìm hiểu dữ liệu

```
print("=== SHAPE ===")
print(df.shape)

print("\n=== HEAD ===")
display(df.head())

print("\n=== INFO ===")
df.info()

print("\n=== DESCRIPTIVE STATISTICS ===")
display(df.describe())

print("\n=== MISSING VALUES ===")
display(df.isna().sum())

print("\n=== DUPLICATES ===")
print(df.duplicated().sum())
```

Hình 8.2. Lấy thông tin của tập dữ liệu

```
=== SHAPE ===
(2000, 16)
```

```
=== HEAD ===
```

	Area	Bedrooms	Bathrooms	Stories	Parking	Age	City	Furnishing	Main Road	Guest Room	Basement	Water Supply	Air Conditioning	Preferred Tenant	Locality Rating	Price
0	1260	4	3	2	1	24	Pune	Semi-Furnished	Yes	Yes	Yes	Both	No	Company	4	1274350
1	5790	2	1	1	1	7	Kolkata	Unfurnished	Yes	Yes	Yes	Both	Yes	Bachelor	5	1094846
2	5626	5	2	3	0	15	Chennai	Semi-Furnished	No	Yes	Yes	Corporation	Yes	Company	6	1495933
3	5591	1	1	1	1	47	Delhi	Furnished	Yes	No	Yes	Borewell	No	Company	3	1003442
4	4172	5	1	1	1	44	Delhi	Unfurnished	No	No	No	Borewell	Yes	Bachelor	1	1306676

Hình 8.3. Kích thước và 5 dòng dữ liệu đầu tiên của tập dữ liệu

```
=== INFO ===
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Area                  2000 non-null  int64
1   Bedrooms              2000 non-null  int64
2   Bathrooms            2000 non-null  int64
3   Stories               2000 non-null  int64
4   Parking               2000 non-null  int64
5   Age                   2000 non-null  int64
6   City                  2000 non-null  str
7   Furnishing            2000 non-null  str
8   Main Road             2000 non-null  str
9   Guest Room            2000 non-null  str
10  Basement              2000 non-null  str
11  Water Supply          2000 non-null  str
12  Air Conditioning      2000 non-null  str
13  Preferred Tenant      2000 non-null  str
14  Locality Rating       2000 non-null  int64
15  Price                 2000 non-null  int64
dtypes: int64(8), str(8)
memory usage: 333.6 KB
```

Hình 8.4. Kết quả kiểm tra kiểu dữ liệu

Kết quả kiểm tra kiểu dữ liệu cho thấy tập dữ liệu gồm 8 thuộc tính dạng số và 8 thuộc tính dạng chuỗi. Các thuộc tính dạng số bao gồm Area, Bedrooms, Bathrooms, Stories, Parking, Age, Locality Rating và Price. Các thuộc tính dạng phân loại gồm City, Furnishing, Main Road, Guest Room, Basement, Water Supply, Air Conditioning và Preferred Tenant.

```

=== MISSING VALUES ===
Area          0
Bedrooms      0
Bathrooms     0
Stories       0
Parking       0
Age           0
City          0
Furnishing    0
Main Road     0
Guest Room    0
Basement      0
Water Supply  0
Air Conditioning 0
Preferred Tenant 0
Locality Rating 0
Price         0
dtype: int64

=== DUPLICATES ===
0

```

Hình 8.5. Kết quả kiểm tra dữ liệu thiếu và bản ghi trùng lặp

Kết quả kiểm tra dữ liệu thiếu cho thấy tất cả các thuộc tính đều có 2.000 giá trị hợp lệ, do đó không xuất hiện giá trị thiếu. Đồng thời, số lượng bản ghi trùng lặp là 0. Vì vậy, dữ liệu không cần thực hiện xử lý missing value hoặc loại bỏ bản ghi trùng lặp ở bước này.

```

numerical_features = X.select_dtypes(include=np.number).columns.tolist()
categorical_features = X.select_dtypes(exclude=np.number).columns.tolist()

print("Numerical features:")
print(numerical_features)

print("\nCategorical features:")
print(categorical_features)

Numerical features:
['Area', 'Bedrooms', 'Bathrooms', 'Stories', 'Parking', 'Age', 'Locality Rating']

Categorical features:
['City', 'Furnishing', 'Main Road', 'Guest Room', 'Basement', 'Water Supply', 'Air Conditioning', 'Preferred Tenant']

```

Hình 8.6. Phân loại các biến số và biến phân loại

```
invalid_checks = {
    "Area <= 0": (df["Area"] <= 0).sum(),
    "Bedrooms <= 0": (df["Bedrooms"] <= 0).sum(),
    "Bathrooms <= 0": (df["Bathrooms"] <= 0).sum(),
    "Stories <= 0": (df["Stories"] <= 0).sum(),
    "Parking < 0": (df["Parking"] < 0).sum(),
    "Age < 0": (df["Age"] < 0).sum(),
    "Locality Rating < 0": (df["Locality Rating"] < 0).sum(),
    "Locality Rating > 10": (df["Locality Rating"] > 10).sum(),
    "Price <= 0": (df["Price"] <= 0).sum()
}

invalid_df = pd.Series(invalid_checks, name="Invalid count")

display(invalid_df.to_frame())
```

Invalid count	
Area <= 0	0
Bedrooms <= 0	0
Bathrooms <= 0	0
Stories <= 0	0
Parking < 0	0
Age < 0	0
Locality Rating < 0	0
Locality Rating > 10	0
Price <= 0	0

Hình 8.7. Kết quả kiểm tra các giá trị không hợp lệ

Do không phát hiện giá trị không hợp lệ, dữ liệu không cần thực hiện bước loại bỏ hoặc hiệu chỉnh giá trị ở nội dung này. Điều này giúp giữ nguyên số lượng quan sát và tránh làm mất thông tin có giá trị trong tập dữ liệu.

```
outlier_counts = {}

for col in numerical_features + ["Price"]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outlier_counts[col] = ((df[col] < lower) | (df[col] > upper)).sum()

outlier_df = pd.DataFrame.from_dict(
    outlier_counts,
    orient="index",
    columns=["Outlier count"]
)

display(outlier_df)
```

Outlier count	
Area	0
Bedrooms	0
Bathrooms	0
Stories	0
Parking	0
Age	0
Locality Rating	0
Price	7

Hình 8.8. Kết quả phát hiện outlier bằng phương pháp IQR

Outlier được kiểm tra bằng phương pháp IQR, trong đó các giá trị nằm ngoài khoảng $Q1 - 1,5 \times IQR$ và $Q3 + 1,5 \times IQR$ được xác định là giá trị ngoại lệ. Kết quả cho thấy các biến đầu vào gồm Area, Bedrooms, Bathrooms, Stories, Parking, Age và Locality Rating không xuất hiện outlier theo tiêu chí này. Trong khi đó, biến mục tiêu Price phát hiện 7 giá trị ngoại lệ.

```
skewness = df[numerical_features + ["Price"]].skew().sort_values(ascending=False)
display(skewness.to_frame("Skewness"))
```

	Skewness
Locality Rating	0.055934
Age	0.048395
Parking	0.017242
Stories	0.007311
Price	-0.004797
Area	-0.015810
Bathrooms	-0.039345
Bedrooms	-0.050423

Hình 8.9. Kết quả kiểm tra độ lệch của các biến số

Kết quả kiểm tra độ lệch cho thấy các biến số có giá trị skewness rất gần 0. Cụ thể, Locality Rating có giá trị 0.055934, Age có giá trị 0.048395, trong khi Bedrooms có giá trị -0.050423. Các biến còn lại cũng có độ lệch nhỏ và nằm gần 0.

Đặc biệt, biến mục tiêu Price có skewness bằng -0.004797, cho thấy phân phối giá nhà gần như đối xứng và không bị lệch đáng kể. Vì vậy, dựa trên kết quả kiểm tra skewness, dữ liệu không có dấu hiệu cần thực hiện biến đổi log đối với các biến số trước khi xây dựng mô hình.

8.4. Làm sạch dữ liệu

Quá trình làm sạch dữ liệu được thực hiện nhằm loại bỏ các vấn đề có thể ảnh hưởng đến chất lượng dữ liệu và kết quả huấn luyện mô hình.

Xử lý giá trị thiếu

Kết quả kiểm tra cho thấy toàn bộ 16 thuộc tính đều có đầy đủ 2.000 giá trị và không xuất hiện giá trị thiếu. Do đó, dữ liệu không cần thực hiện các phương pháp điền giá trị thiếu như thay thế bằng giá trị trung bình, trung vị hoặc giá trị phổ biến. Việc giữ nguyên dữ liệu giúp tránh đưa thêm giá trị nhân tạo vào tập dữ liệu.

Xử lý bản ghi trùng lặp

Kết quả kiểm tra cho thấy tập dữ liệu không chứa bản ghi trùng lặp, với tổng số bản ghi

trùng lặp bằng 0. Do đó, không cần thực hiện thao tác loại bỏ các dòng dữ liệu trùng nhau. Việc giữ nguyên 2.000 quan sát đảm bảo không làm mất thông tin hợp lệ trong tập dữ liệu.

Xử lý giá trị không hợp lệ

Kết quả kiểm tra cho thấy không có giá trị không hợp lệ trong các thuộc tính của tập dữ liệu. Diện tích, số phòng ngủ, số phòng tắm và số tầng đều có giá trị dương. Các thuộc tính Parking và Age không xuất hiện giá trị âm. Locality Rating nằm trong khoảng hợp lệ từ 0 đến 10 và Price không có giá trị nhỏ hơn hoặc bằng 0. Vì vậy, dữ liệu không cần thực hiện bước loại bỏ hoặc thay thế các giá trị không hợp lệ.

Xử lý outlier

Outlier count	
Area	0
Bedrooms	0
Bathrooms	0
Stories	0
Parking	0
Age	0
Locality Rating	0
Price	7

Hình 8.10. Thống kê giá trị ngoại lệ theo phương pháp IQR

Phương pháp IQR được sử dụng để phát hiện các giá trị ngoại lệ trên các biến số. Kết quả cho thấy các biến đầu vào gồm Area, Bedrooms, Bathrooms, Stories, Parking, Age và Locality Rating không có giá trị ngoại lệ. Đối với biến mục tiêu Price, có 7 quan sát được xác định là outlier.

Các giá trị này được giữ lại trong tập dữ liệu vì đây là biến mục tiêu của bài toán dự đoán giá nhà và các giá trị giá cao hoặc thấp bất thường vẫn có thể đại diện cho những căn nhà thực tế. Việc loại bỏ trực tiếp các quan sát này có thể làm mất thông tin và làm giảm tính đa dạng của dữ liệu. Do đó, dữ liệu được giữ nguyên để tiếp tục thực hiện các bước phân tích và xây dựng mô hình.

8.5. Biểu diễn dữ liệu

Tách đặc trưng và biến mục tiêu


```
target = "Price"

X = df.drop(columns=target)
y = df[target]

print("One observation:")
display(df.iloc[[0]])

print("\nFeature columns:")
print(X.columns.tolist())

print("\nTarget:", target)
print("X shape:", X.shape)
print("y shape:", y.shape)
```

One observation:

	Area	Bedrooms	Bathrooms	Stories	Parking	Age	City	Furnishing	Main Road	Guest Room	Basement	Water Supply	Air Conditioning	Preferred Tenant	Locality Rating	Price
0	1260	4	3	2	1	24	Pune	Semi-Furnished	Yes	Yes	Yes	Both	No	Company	4	1274350

Feature columns:
['Area', 'Bedrooms', 'Bathrooms', 'Stories', 'Parking', 'Age', 'City', 'Furnishing', 'Main Road', 'Guest Room', 'Basement', 'Water Supply', 'Air Conditioning', 'Preferred Tenant', 'Locality Rating']

Target: Price
X shape: (2000, 15)
y shape: (2000,)

Hình 8.11. Tách đặc trưng và biến mục tiêu

Sau bước làm sạch, biến mục tiêu Price được tách khỏi các biến đầu vào. Tập đặc trưng X gồm 15 thuộc tính mô tả đặc điểm của căn nhà, trong khi y là biến mục tiêu biểu diễn giá nhà. Tập dữ liệu ban đầu có 2.000 quan sát nên ma trận đặc trưng có kích thước 2.000×15 và vector mục tiêu có 2.000 giá trị.

Phân loại đặc trưng trước khi mã hóa

```
numerical_features = X.select_dtypes(include=np.number).columns.tolist()
categorical_features = X.select_dtypes(exclude=np.number).columns.tolist()

print("Numerical features:")
print(numerical_features)

print("\nCategorical features:")
print(categorical_features)
```

Numerical features:
['Area', 'Bedrooms', 'Bathrooms', 'Stories', 'Parking', 'Age', 'Locality Rating']

Categorical features:
['City', 'Furnishing', 'Main Road', 'Guest Room', 'Basement', 'Water Supply', 'Air Conditioning', 'Preferred Tenant']

Hình 8.12. Phân loại đặc trưng

Các đặc trưng đầu vào được chia thành hai nhóm. Nhóm biến số gồm 7 thuộc tính có kiểu dữ liệu số và có thể sử dụng trực tiếp cho các phép tính trong mô hình. Nhóm biến phân loại gồm 8 thuộc tính dạng chuỗi, cần được mã hóa thành dạng số trước khi đưa vào mô hình. Việc phân loại này là cơ sở để xây dựng quy trình tiền xử lý phù hợp cho từng nhóm biến.

Mã hóa biến phân loại và chuẩn hóa dữ liệu

```

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

print("Preprocessing pipeline created.")

```

Preprocessing pipeline created.

Hình 8.13. Mã hóa biến phân loại và chuẩn hóa dữ liệu

Fit và transform dữ liệu

```

X_train_processed = preprocessor.fit_transform(X_train)
X_val_processed = preprocessor.transform(X_val)
X_test_processed = preprocessor.transform(X_test)

print("Processed train shape:", X_train_processed.shape)
print("Processed validation shape:", X_val_processed.shape)
print("Processed test shape:", X_test_processed.shape)

```

Processed train shape: (1400, 31)
 Processed validation shape: (300, 31)
 Processed test shape: (300, 31)

Hình 8.14. Fit và transform dữ liệu

Sau khi thực hiện biến đổi, số lượng đặc trưng tăng từ 15 lên 31 do các biến phân loại được chuyển sang dạng One-Hot Encoding. 3 tập dữ liệu đều có cùng 31 đặc trưng và dữ liệu đầu vào cho mô hình có kiểu float64.

```

print("=== MODEL INPUT ===")
print("X_train_processed shape:", X_train_processed.shape)
print("X_val_processed shape:", X_val_processed.shape)
print("X_test_processed shape:", X_test_processed.shape)

print("\nData type:", X_train_processed.dtype)

```

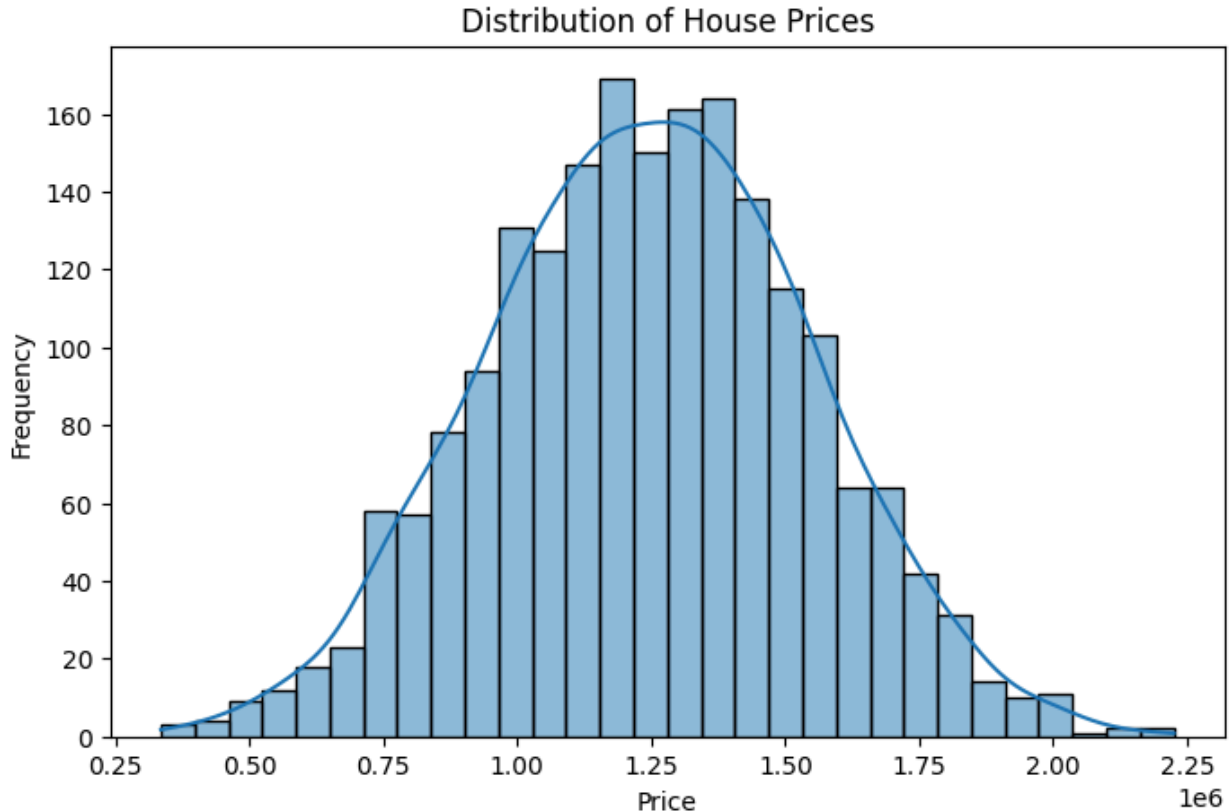
=== MODEL INPUT ===
 X_train_processed shape: (1400, 31)
 X_val_processed shape: (300, 31)
 X_test_processed shape: (300, 31)

Data type: float64

Hình 8.15. Kiểm tra Model Input

Sau quá trình mã hóa và chuẩn hóa, dữ liệu đầu vào cuối cùng của mô hình có 31 đặc trưng với kiểu dữ liệu float64. Tập huấn luyện gồm 1.400 quan sát, tập validation gồm 300 quan sát và tập kiểm tra gồm 300 quan sát. Dữ liệu sau biến đổi đã ở dạng số và sẵn sàng được sử dụng để huấn luyện các mô hình hồi quy.

8.6. Phân tích khám phá dữ liệu (EDA)

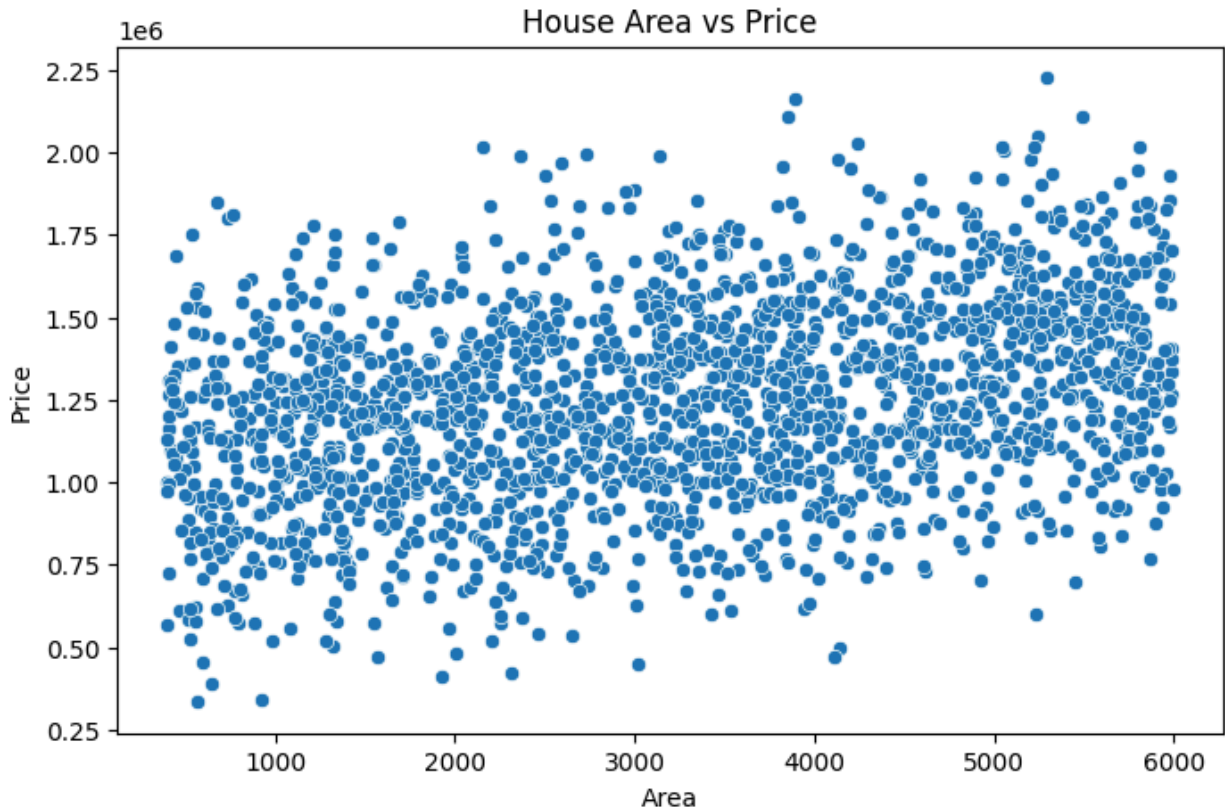


Hình 8.16. Phân bố biến mục tiêu Price

Nhận xét: Phân phối của Price tập trung chủ yếu quanh vùng giá trung tâm của tập dữ liệu. Phân phối tương đối cân đối và không xuất hiện hiện tượng lệch mạnh. Kết quả này phù hợp với giá trị skewness của Price bằng -0.004797 đã được xác định ở bước khảo sát dữ liệu.

Diễn giải: Giá nhà trong tập dữ liệu có xu hướng phân bố tương đối đồng đều quanh giá trị trung tâm, không bị tập trung quá mạnh về một phía.

Ý nghĩa đối với mô hình ML: Phân phối giá nhà tương đối cân đối giúp mô hình hồi quy học mối quan hệ giữa các đặc trưng và giá mục tiêu thuận lợi hơn. Không cần thực hiện biến đổi log đối với Price chỉ dựa trên đặc điểm phân phối này.

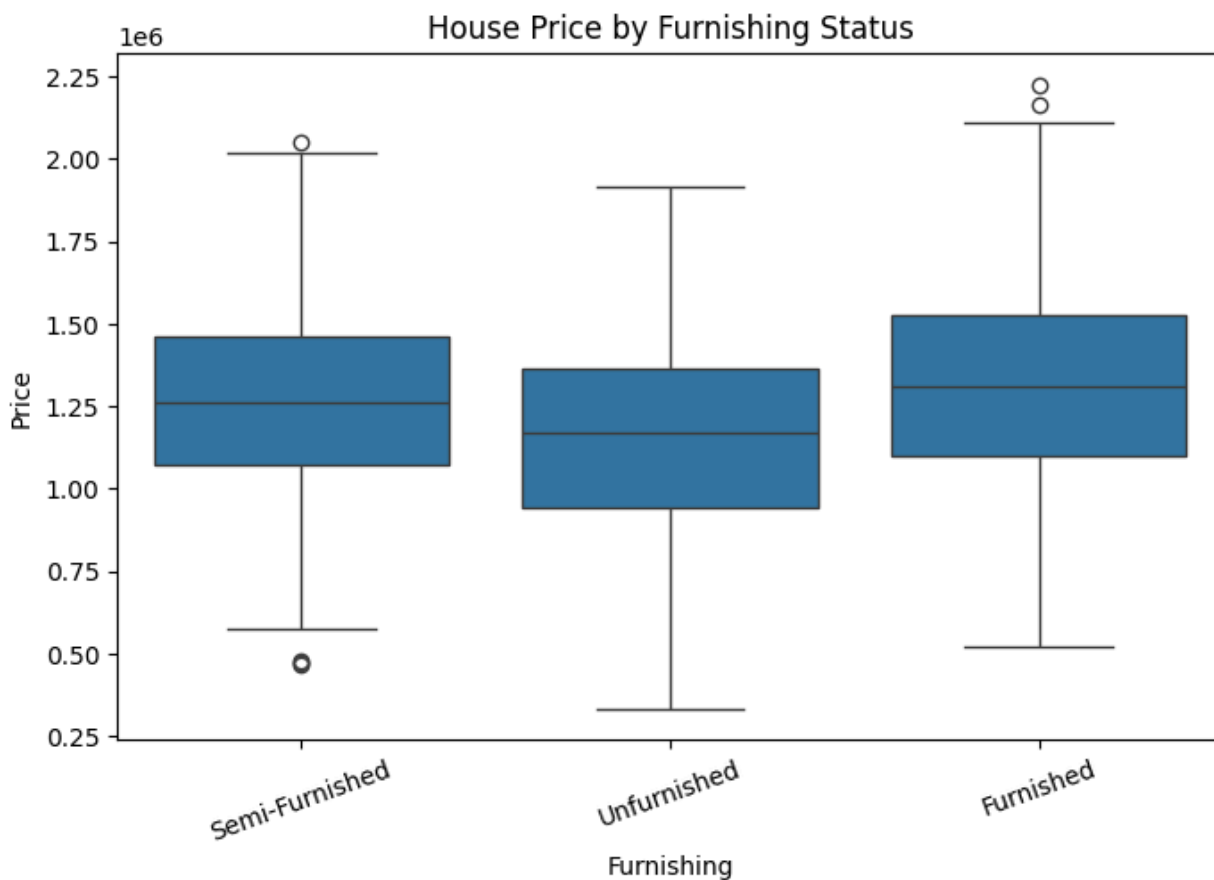


Hình 8.17. Mối quan hệ giữa diện tích và giá nhà

Nhận xét: Các điểm dữ liệu cho thấy giá nhà có xu hướng thay đổi theo diện tích. Nhìn chung, những căn nhà có diện tích lớn có xu hướng có mức giá cao hơn. Tuy nhiên, các điểm dữ liệu vẫn có sự phân tán, cho thấy giá nhà không chỉ phụ thuộc vào diện tích mà còn chịu ảnh hưởng bởi các đặc trưng khác.

Diễn giải: Area có khả năng là một trong những đặc trưng quan trọng đối với việc dự đoán Price. Sự phân tán của dữ liệu cho thấy cùng một diện tích vẫn có thể có các mức giá khác nhau do sự khác biệt về các đặc điểm của căn nhà.

Ý nghĩa đối với mô hình ML: Area nên được giữ lại làm đặc trưng đầu vào cho mô hình hồi quy. Đồng thời, cần kết hợp với các thuộc tính khác để mô hình có thể biểu diễn đầy đủ hơn mối quan hệ với giá nhà.

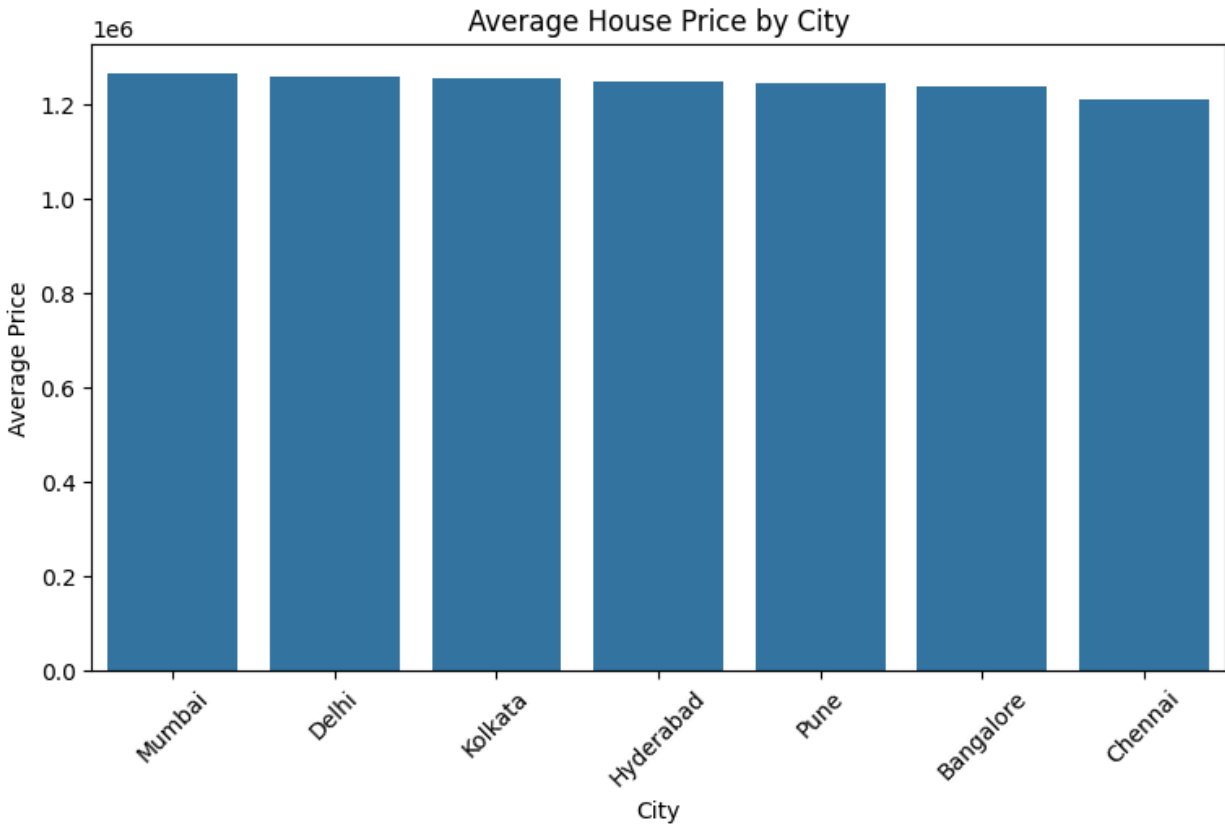


Hình 8.18. Giá nhà theo tình trạng nội thất

Nhận xét: Các nhóm tình trạng nội thất có sự khác biệt về phân phối giá nhà. Mỗi nhóm có khoảng giá và mức giá trung tâm khác nhau, đồng thời một số nhóm có mức độ phân tán giá lớn hơn các nhóm còn lại.

Diễn giải: Tình trạng nội thất có thể có mối liên hệ với giá nhà. Tuy nhiên, sự khác biệt về giá giữa các nhóm không thể hiện rằng Furnishing là nguyên nhân trực tiếp quyết định giá, vì giá nhà còn phụ thuộc vào nhiều đặc trưng khác.

Ý nghĩa đối với mô hình ML: Furnishing nên được giữ lại làm một đặc trưng phân loại và được mã hóa bằng One-Hot Encoding trước khi đưa vào mô hình hồi quy.

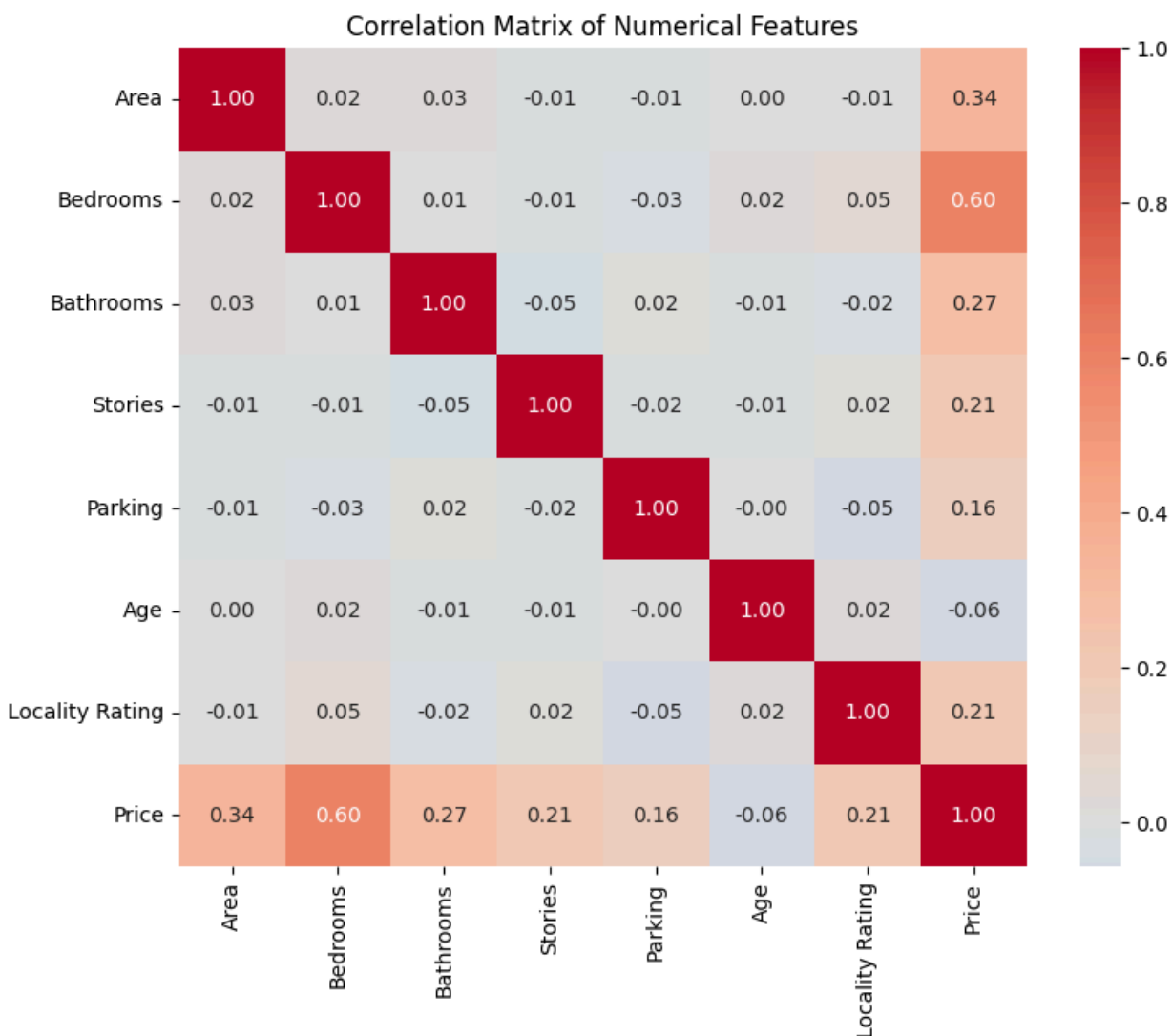


Hình 8.19. Giá nhà trung bình theo thành phố

Nhận xét: Giá nhà trung bình có sự khác biệt giữa các thành phố. Một số thành phố có mức giá trung bình cao hơn trong khi các thành phố khác có mức giá thấp hơn.

Diễn giải: Đặc điểm khu vực, được biểu diễn thông qua biến City, có thể liên quan đến sự khác biệt về giá nhà.

Ý nghĩa đối với mô hình ML: City là một đặc trưng phân loại có khả năng cung cấp thông tin cho bài toán dự đoán giá nhà. Do đó, biến này được giữ lại và mã hóa bằng One-Hot Encoding trong bước biểu diễn dữ liệu.

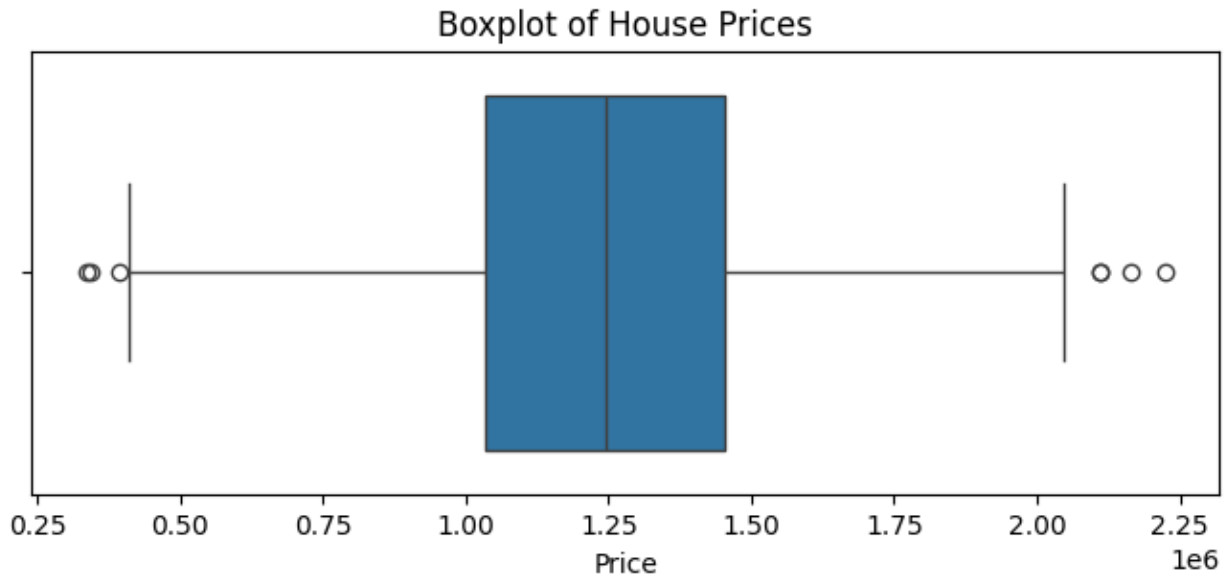


Hình 8.20. Ma trận tương quan giữa các biến số

Nhận xét: Ma trận tương quan cho thấy mức độ liên hệ giữa Price và các đặc trưng số không giống nhau. Một số đặc trưng có tương quan với giá nhà rõ hơn các đặc trưng còn lại.

Diễn giải: Các đặc trưng có tương quan với Price có thể cung cấp thông tin hữu ích cho bài toán dự đoán giá nhà. Tuy nhiên, tương quan không đồng nghĩa với quan hệ nhân quả.

Ý nghĩa đối với mô hình ML: Kết quả tương quan được sử dụng như một cơ sở hỗ trợ phân tích và lựa chọn đặc trưng. Các đặc trưng số vẫn được giữ lại trong quá trình xây dựng mô hình để mô hình có thể khai thác đồng thời nhiều nguồn thông tin.



Hình 8.21. Boxplot của giá nhà

Nhận xét: Biểu đồ cho thấy phần lớn giá nhà tập trung trong một khoảng nhất định, đồng thời xuất hiện một số điểm nằm ngoài phạm vi chính của phân phối. Kết quả này phù hợp với kiểm tra IQR trước đó, trong đó biến Price có 7 giá trị được xác định là outlier.

Diễn giải: Các giá trị này có thể đại diện cho những căn nhà có mức giá khác biệt đáng kể so với phần lớn dữ liệu. Do chưa có cơ sở để khẳng định đây là dữ liệu sai, các giá trị được giữ lại trong tập dữ liệu.

Ý nghĩa đối với mô hình ML: Việc giữ lại các outlier giúp mô hình vẫn có khả năng học từ những trường hợp giá nhà đặc biệt. Tuy nhiên, các giá trị này cần được lưu ý khi đánh giá sai số dự đoán của mô hình.

8.7. Xây dựng mô hình

8.7.1. Chia tập Train, Validation và Test

Tập dữ liệu sau tiền xử lý được chia thành ba tập gồm Train, Validation và Test theo tỷ lệ 70% – 15% – 15%. Tập Train được sử dụng để huấn luyện mô hình, tập Validation được sử dụng để so sánh và lựa chọn mô hình, trong khi tập Test được giữ riêng để đánh giá cuối cùng.


```

from sklearn.model_selection import train_test_split

# 70% train, 30% temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)

# Split temporary set into 15% validation and 15% test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42
)

print("Train:", X_train.shape, y_train.shape)
print("Validation:", X_val.shape, y_val.shape)
print("Test:", X_test.shape, y_test.shape)

Train: (1400, 15) (1400,)
Validation: (300, 15) (300,)
Test: (300, 15) (300,)

```

Hình 8.22. Kết quả chia tập dữ liệu Train, Validation và Test

Kết quả chia dữ liệu gồm 1.400 quan sát cho tập Train, 300 quan sát cho tập Validation và 300 quan sát cho tập Test. Mỗi tập vẫn chứa 15 đặc trưng đầu vào trước khi thực hiện bước biểu diễn dữ liệu.

8.7.2. Xây dựng các mô hình hồi quy

```

models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),
    "Gradient Boosting": GradientBoostingRegressor(
        n_estimators=200,
        random_state=42
    )
}

```

Hình 8.23. Các mô hình hồi quy được sử dụng trong bài toán dự đoán giá nhà

8.7.3. Huấn luyện mô hình

```
trained_models = {}

for name, model in models.items():
    model.fit(X_train_processed, y_train)
    trained_models[name] = model

print("Training completed.")

for name, model in trained_models.items():
    print(f"{name}: trained")

Training completed.
Linear Regression: trained
Ridge: trained
Decision Tree: trained
Random Forest: trained
Gradient Boosting: trained
```

Hình 8.24. Kết quả huấn luyện các mô hình hồi quy

8.8. Đánh giá mô hình

	Model	MAE	MSE	RMSE	R ²
1	Ridge	115895.284453	2.164648e+10	147127.418627	0.756293
0	Linear Regression	115921.994226	2.165715e+10	147163.691625	0.756173
4	Gradient Boosting	126717.840789	2.462584e+10	156926.236201	0.722750
3	Random Forest	134924.783333	2.926706e+10	171076.193288	0.670497
2	Decision Tree	217223.736667	7.146630e+10	267331.819714	0.195396

Hình 8.25. So sánh hiệu quả các mô hình trên tập Validation

Kết quả trên tập Validation cho thấy Ridge Regression đạt RMSE thấp nhất với giá trị 147127.42, đồng thời có R² đạt 0.7563. Linear Regression cho kết quả rất gần với Ridge Regression, với RMSE 147163.69 và R² 0.7562. Gradient Boosting, Random Forest và Decision Tree có kết quả kém hơn hai mô hình tuyến tính trên tập Validation.

Dựa trên tiêu chí RMSE, Ridge Regression được lựa chọn là mô hình tốt nhất trên tập Validation và được sử dụng để đánh giá cuối cùng trên tập Test.

8.9. Lựa chọn mô hình

	Model	MAE	MSE	RMSE	R ²
0	Ridge	117323.294731	2.126660e+10	145830.730431	0.746486

Hình 8.26. Kết quả đánh giá mô hình Ridge Regression trên tập Test

Các căn nhà phù hợp trong Dataset

Thành phố: Toàn quốc

Tìm thấy 8 căn

Thành phố	Diện tích	Phòng ngủ	Phòng tắm	Số tầng	Chỗ đậu xe	Tuổi nhà	Nội thất	Đường chính	Phòng khách	Tầng hầm	Nguồn nước	Điều hòa	Đồ
Delhi	527 m ²	3	3	2	1	18	Đầy đủ nội thất	Không	Không	Có	Giếng khoan	Không	Cế
Pune	1260 m ²	4	3	2	1	24	Nội thất cơ bản	Có	Có	Có	Cả hai	Không	Cế
Hyderabad	1214 m ²	4	3	2	0	25	Không nội thất	Không	Không	Không	Nước máy	Có	Gi
Chennai	1789 m ²	4	3	2	1	40	Đầy đủ nội thất	Không	Không	Không	Cả hai	Không	Gi
Mumbai	904 m ²	4	2	2	1	32	Đầy đủ nội thất	Không	Có	Có	Giếng khoan	Không	Cế
Pune	1226 m ²	5	3	2	0	23	Đầy đủ nội thất	Có	Có	Không	Nước máy	Có	Nc
Hyderabad	899 m ²	5	3	2	1	15	Đầy đủ nội thất	Có	Có	Không	Cả hai	Không	Nc
Mumbai	1320 m ²	5	3	2	0	20	Đầy đủ nội thất	Có	Không	Không	Cả hai	Có	Cế

Hình 8.27. Deploy dạng web

11:37

Dự đoán giá nhà

Nhập thông tin căn nhà để dự đoán giá

Thông tin căn nhà

Diện tích

Số phòng ngủ

1260

3

Số phòng tắm

Số tầng

2

2

Chỗ đậu xe

Tuổi căn nhà

1

1

Thành phố

Delhi

Nội thất

Nội thất cơ bản

Đường chính

Có

Phòng khách

Có

Tầng hầm

Không

Nguồn nước

Không

11:37

Nguồn nước

Nước máy

Điều hòa

Có

Đổi tượng thuê

Gia đình

Đánh giá khu vực

7

Dự đoán giá

Giá nhà dự đoán

₹1,264,108

Giá được dự đoán bởi mô hình Machine Learning

Nhà phù hợp trong Dataset

20

Các căn cùng thành phố và có giá thực tế gần với giá dự đoán

Delhi

₹1,264,650

3,511 m²

1 phòng ngủ

1 phòng tắm

3 tầng

Semi-Furnished

9 năm

Delhi

₹1,262,995

1,338 m²

4 phòng ngủ

1 phòng tắm

3 tầng

Semi-Furnished

35 năm

Delhi

₹1,265,473

5,492 m²

4 phòng ngủ

1 phòng tắm

1 tầng

Furnished

35 năm

Delhi

₹1,261,512

525 m²

5 phòng ngủ

1 phòng tắm

1 tầng

Semi-Furnished

9 năm

Delhi

₹1,266,996

3,319 m²

5 phòng ngủ

3 phòng tắm

1 tầng

Unfurnished

42 năm

Delhi

₹1,267,606

Hình 8.28. Deploy dạng mobile bằng expo go

67

Chương IX. Ứng dụng 3 - Phân tích hành vi khách hàng và dự đoán khả năng đề xuất sản phẩm

9.1. Mô tả bài toán

Ứng dụng 3 tập trung vào việc phân tích hành vi khách hàng trong lĩnh vực thương mại điện tử thông qua dữ liệu đánh giá sản phẩm. Mục tiêu của ứng dụng là xây dựng mô hình học máy có khả năng dự đoán liệu khách hàng có đề xuất sản phẩm sau khi sử dụng và đánh giá sản phẩm hay không.

9.2. Giới thiệu tập dữ liệu

Tập dữ liệu được sử dụng trong ứng dụng là **Women's Clothing E-Commerce Reviews**, được cung cấp trên Kaggle. Tập dữ liệu chứa các đánh giá của khách hàng đối với các sản phẩm thời trang nữ trong môi trường thương mại điện tử. Dữ liệu bao gồm thông tin về khách hàng, sản phẩm, nội dung đánh giá, mức đánh giá và hành vi đề xuất sản phẩm.

Link dataset:

<https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>

```
print("Dataset shape:", df.shape)

print("\nFirst 5 rows:")
display(df.head())

print("\nDataset information:")
df.info()

print("\nDescriptive statistics:")
display(df.describe(include="all").T)

Dataset shape: (23486, 11)
```

Hình 9.1. Kích thước của tập dữ liệu Ecommerce.

Tập dữ liệu có 23.486 quan sát và 11 thuộc tính.

	Unnamed: 0	Clothing ID	Age	Title	Review Text	Rating	Recommended IND	Positive Feedback Count	Division Name	Department Name	Class Name
0	0	767	33	NaN	Absolutely wonderful - silky and sexy and comf...	4	1	0	Initmates	Intimate	Intimates
1	1	1080	34	NaN	Love this dress! it's sooo pretty. i happene...	5	1	4	General	Dresses	Dresses
2	2	1077	60	Some major design flaws	I had such high hopes for this dress and reall...	3	0	0	General	Dresses	Dresses
3	3	1049	50	My favorite buy!	I love, love, love this jumpsuit. it's fun, fl...	5	1	0	General Petite	Bottoms	Pants
4	4	847	47	Flattering shirt	This shirt is very flattering to all due to th...	5	1	6	General	Tops	Blouses

Hình 9.2. Một số bản ghi đầu tiên của tập dữ liệu

Unnamed: 0 là cột chỉ mục được lưu kèm theo dữ liệu và sẽ được loại bỏ trong quá trình tiền xử lý. Từ các bản ghi đầu tiên có thể thấy mỗi quan sát tương ứng với một đánh giá của khách hàng về một sản phẩm. Dữ liệu bao gồm các thông tin như mã sản phẩm (Clothing ID), độ tuổi khách hàng (Age), tiêu đề đánh giá (Title), nội dung đánh giá (Review Text), điểm đánh giá (Rating), hành vi đề xuất (Recommended IND) và số lượng phản hồi tích cực (Positive Feedback Count).

```
Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 23486 entries, 0 to 23485
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            23486 non-null  int64
1   Clothing ID                            23486 non-null  int64
2   Age                                    23486 non-null  int64
3   Title                                  19676 non-null  str
4   Review Text                            22641 non-null  str
5   Rating                                23486 non-null  int64
6   Recommended IND                        23486 non-null  int64
7   Positive Feedback Count                23486 non-null  int64
8   Division Name                          23472 non-null  str
9   Department Name                        23472 non-null  str
10  Class Name                             23472 non-null  str
dtypes: int64(6), str(5)
memory usage: 9.5 MB
```

Hình 9.3. Thông tin cấu trúc và kiểu dữ liệu của tập dữ liệu

Tập dữ liệu gồm cả dữ liệu số, dữ liệu phân loại và dữ liệu văn bản. Các thuộc tính Age, Rating và Positive Feedback Count thuộc nhóm dữ liệu số. Các thuộc tính Division Name, Department Name và Class Name là các thuộc tính phân loại. Title và Review Text là các thuộc tính dạng văn bản, cung cấp thông tin trực tiếp từ phản hồi của khách hàng.

Biến mục tiêu của bài toán là Recommended IND. Đây là biến phân loại nhị phân, trong đó giá trị 0 biểu thị khách hàng không đề xuất sản phẩm và giá trị 1 biểu thị khách hàng đề xuất sản phẩm. Các thuộc tính còn lại được sử dụng làm thông tin đầu vào để xây

dựng mô hình dự đoán.

Thuộc tính	Kiểu dữ liệu	Ý nghĩa
Clothing ID	Số nguyên	Mã định danh của sản phẩm.
Age	Số nguyên	Độ tuổi của khách hàng.
Title	Văn bản	Tiêu đề của đánh giá.
Review Text	Văn bản	Nội dung đánh giá của khách hàng.
Rating	Số nguyên	Điểm đánh giá sản phẩm từ 1 đến 5.
Recommended IND	Nhị phân	Biến mục tiêu, biểu thị khách hàng có đề xuất sản phẩm hay không.
Positive Feedback Count	Số nguyên	Số lượng phản hồi tích cực đối với đánh giá.
Division Name	Phân loại	Nhóm sản phẩm cấp cao.
Department Name	Phân loại	Nhóm ngành hàng của sản phẩm.
Class Name	Phân loại	Phân loại chi tiết của sản phẩm.

9.3. Biểu diễn khách hàng

Trong tập dữ liệu được sử dụng, mỗi quan sát tương ứng với một đánh giá của khách hàng đối với một sản phẩm. Do dữ liệu không cung cấp mã định danh khách hàng hoặc thông tin về nhiều giao dịch của cùng một khách hàng, ứng dụng không thực hiện bước tổng hợp từ nhiều giao dịch thành một hồ sơ khách hàng. Thay vào đó, mỗi đánh giá được xem là một hồ sơ thể hiện hành vi của khách hàng trong quá trình tương tác và đánh giá sản phẩm.

Thông tin ban đầu của một quan sát gồm các thuộc tính về khách hàng, đánh giá và sản phẩm. Để đưa dữ liệu vào mô hình học máy, các thông tin này được chuyển thành các đặc trưng số và đặc trưng phân loại. Cụ thể, độ tuổi của khách hàng (Age), điểm đánh giá (Rating) và số lượng phản hồi tích cực (Positive Feedback Count) được sử dụng làm các đặc trưng số.

Bên cạnh các đặc trưng ban đầu, hệ thống xây dựng thêm một số đặc trưng từ nội dung đánh giá. Review Length biểu diễn độ dài của nội dung đánh giá, Title Length biểu diễn độ dài tiêu đề đánh giá, Has Review cho biết đánh giá có chứa nội dung hay không và Has Title cho biết đánh giá có tiêu đề hay không. Các đặc trưng này giúp chuyển một phần thông tin về mức độ hoạt động và mức độ cung cấp phản hồi của khách hàng thành các giá trị số.

Đối với thông tin phân loại sản phẩm, ba thuộc tính Division Name, Department Name và Class Name được sử dụng. Các thuộc tính này được chuyển đổi sang dạng số bằng phương pháp One-Hot Encoding để có thể kết hợp với các đặc trưng số trong cùng một ma trận đầu vào.

Như vậy, một quan sát được biểu diễn bởi nhóm các đặc trưng gồm: Age, Rating, Positive Feedback Count, Review Length, Title Length, Has Review, Has Title, Division Name, Department Name và Class Name. Các đặc trưng số được xử lý bằng cách thay thế giá trị thiếu bằng trung vị và chuẩn hóa bằng StandardScaler. Các đặc trưng phân loại được thay thế giá trị thiếu bằng giá trị xuất hiện thường xuyên nhất và mã hóa bằng OneHotEncoder.

Quá trình biểu diễn có thể được mô tả theo các bước:

Thông tin khách hàng và đánh giá → Xây dựng đặc trưng → Xử lý đặc trưng số và phân loại → Feature Vector → Đầu vào mô hình.

Sau bước tiền xử lý, các đặc trưng được kết hợp thành ma trận đầu vào cho mô hình học máy. Trong notebook, ma trận sau preprocessing có kích thước **(16440, 36)** đối với tập huấn luyện. Trong đó, mỗi hàng tương ứng với một quan sát và mỗi cột tương ứng với một đặc trưng sau quá trình xử lý và mã hóa.

Cách biểu diễn này chuyển thông tin ban đầu của khách hàng và đánh giá sản phẩm thành dạng số phù hợp với các thuật toán học máy, đồng thời tạo ra một biểu diễn thống nhất để

mô hình có thể sử dụng trong quá trình huấn luyện và dự đoán.

```
X_train_processed = preprocessor.fit_transform(X_train_tabular)
X_val_processed = preprocessor.transform(X_val_tabular)
X_test_processed = preprocessor.transform(X_test_tabular)

print("Original training shape:", X_train_tabular.shape)
print("Processed training shape:", X_train_processed.shape)

print("Original validation shape:", X_val_tabular.shape)
print("Processed validation shape:", X_val_processed.shape)

print("Original test shape:", X_test_tabular.shape)
print("Processed test shape:", X_test_processed.shape)

print("\nProcessed dtype:", X_train_processed.dtype)

Original training shape: (16440, 10)
Processed training shape: (16440, 36)
Original validation shape: (3523, 10)
Processed validation shape: (3523, 36)
Original test shape: (3523, 10)
Processed test shape: (3523, 36)

Processed dtype: float64
```

Hình 9.4. Kích thước dữ liệu trước và sau quá trình biểu diễn đặc trưng

9.4. Làm sạch dữ liệu

Quá trình làm sạch dữ liệu được thực hiện nhằm loại bỏ các vấn đề có thể ảnh hưởng đến chất lượng dữ liệu và kết quả huấn luyện mô hình.

Xử lý giá trị thiếu:

```
missing = pd.DataFrame({
    "Missing Count": df.isna().sum(),
    "Missing Percentage": (df.isna().mean() * 100).round(2)
})

display(missing[missing["Missing Count"] > 0].sort_values(
    "Missing Count", ascending=False
))
```

	Missing Count	Missing Percentage
Title	3810	16.22
Review Text	845	3.60
Division Name	14	0.06
Department Name	14	0.06
Class Name	14	0.06

Hình 9.5. Kết quả kiểm tra giá trị thiếu trong tập dữ liệu

Kết quả kiểm tra cho thấy một số thuộc tính dạng văn bản và phân loại có giá trị thiếu. Cụ thể, Title có 3.810 giá trị thiếu, Review Text có 845 giá trị thiếu, trong khi Division Name, Department Name và Class Name mỗi thuộc tính có 14 giá trị thiếu.

Các giá trị thiếu không được loại bỏ trực tiếp khỏi toàn bộ dữ liệu vì có thể làm mất các quan sát hữu ích. Trong quá trình tiền xử lý, các đặc trưng số được thay thế giá trị thiếu bằng giá trị trung vị. Đối với các đặc trưng phân loại, giá trị thiếu được thay thế bằng giá trị xuất hiện thường xuyên nhất.

Xử lý bản ghi trùng lặp

```
duplicate_count = df.duplicated().sum()

print("Number of duplicated rows:", duplicate_count)
```

Number of duplicated rows: 0

Hình 9.6. Kết quả loại bỏ bản ghi trùng lặp

Dữ liệu được kiểm tra bằng số lượng bản ghi trùng lặp. Kết quả cho thấy tập dữ liệu không có bản ghi trùng lặp. Vì vậy, không cần thực hiện bước loại bỏ dữ liệu trùng lặp.

Xử lý giá trị không hợp lệ

```
invalid_checks = {
    "Age <= 0": (df["Age"] <= 0).sum(),
    "Rating outside 1-5": (~df["Rating"].between(1, 5)).sum(),
    "Recommended IND not 0/1": (~df["Recommended IND"].isin([0, 1])).sum(),
    "Positive Feedback Count < 0": (df["Positive Feedback Count"] < 0).sum()
}

invalid_df = pd.DataFrame.from_dict(
    invalid_checks,
    orient="index",
    columns=["Invalid Count"]
)

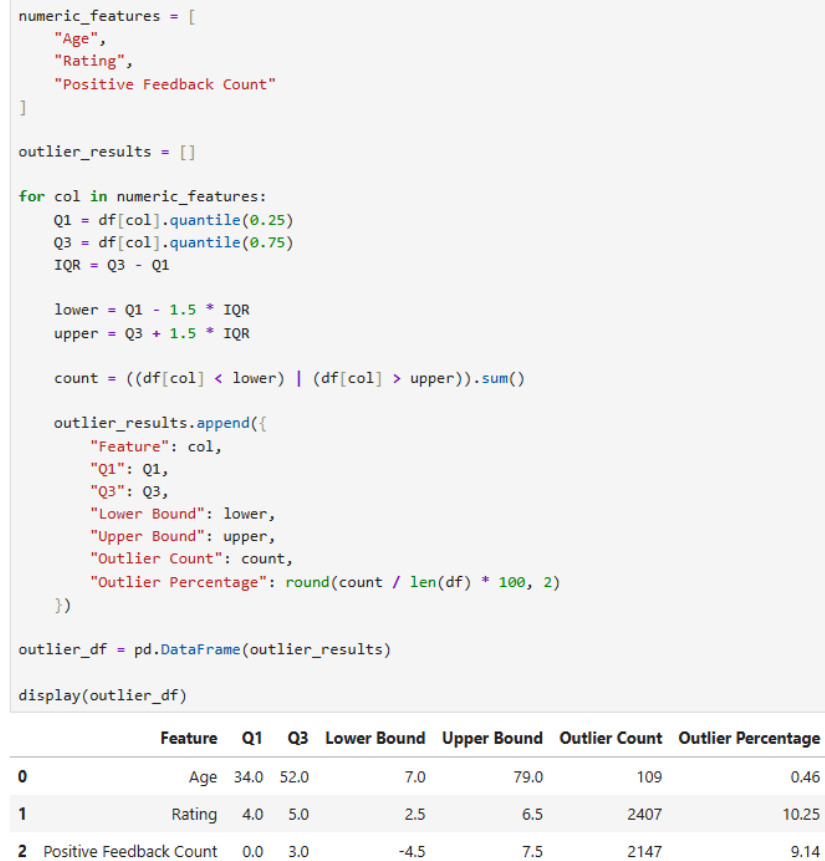
display(invalid_df)
```

Invalid Count	
Age <= 0	0
Rating outside 1-5	0
Recommended IND not 0/1	0
Positive Feedback Count < 0	0

Hình 9.7. Kết quả kiểm tra các giá trị không hợp lệ

Kết quả kiểm tra cho thấy các thuộc tính chính của dữ liệu đáp ứng các điều kiện hợp lệ. Do đó, không cần loại bỏ thêm các quan sát dựa trên các điều kiện này.

Kiểm tra giá trị ngoại lệ



Hình 9.8. Thống kê giá trị ngoại lệ theo phương pháp IQR

Các giá trị ngoại lệ được giữ lại trong dữ liệu thay vì loại bỏ tự động, bởi chúng có thể phản ánh hành vi thực tế của khách hàng, đặc biệt đối với Positive Feedback Count, nơi một số đánh giá có thể nhận được số lượng phản hồi cao hơn đáng kể so với phần lớn các đánh giá khác.

Loại bỏ thuộc tính không cần thiết và xây dựng đặc trưng

Cột Unnamed: 0 được loại bỏ vì chỉ đóng vai trò là chỉ mục được lưu kèm theo tập dữ liệu và không mang thông tin hữu ích cho việc dự đoán.

Ngoài các thuộc tính ban đầu, một số đặc trưng mới được xây dựng từ nội dung đánh giá. Review Length biểu diễn độ dài của nội dung đánh giá, Title Length biểu diễn độ dài tiêu đề, Has Review cho biết một quan sát có nội dung đánh giá hay không và Has Title cho biết một quan sát có tiêu đề hay không.

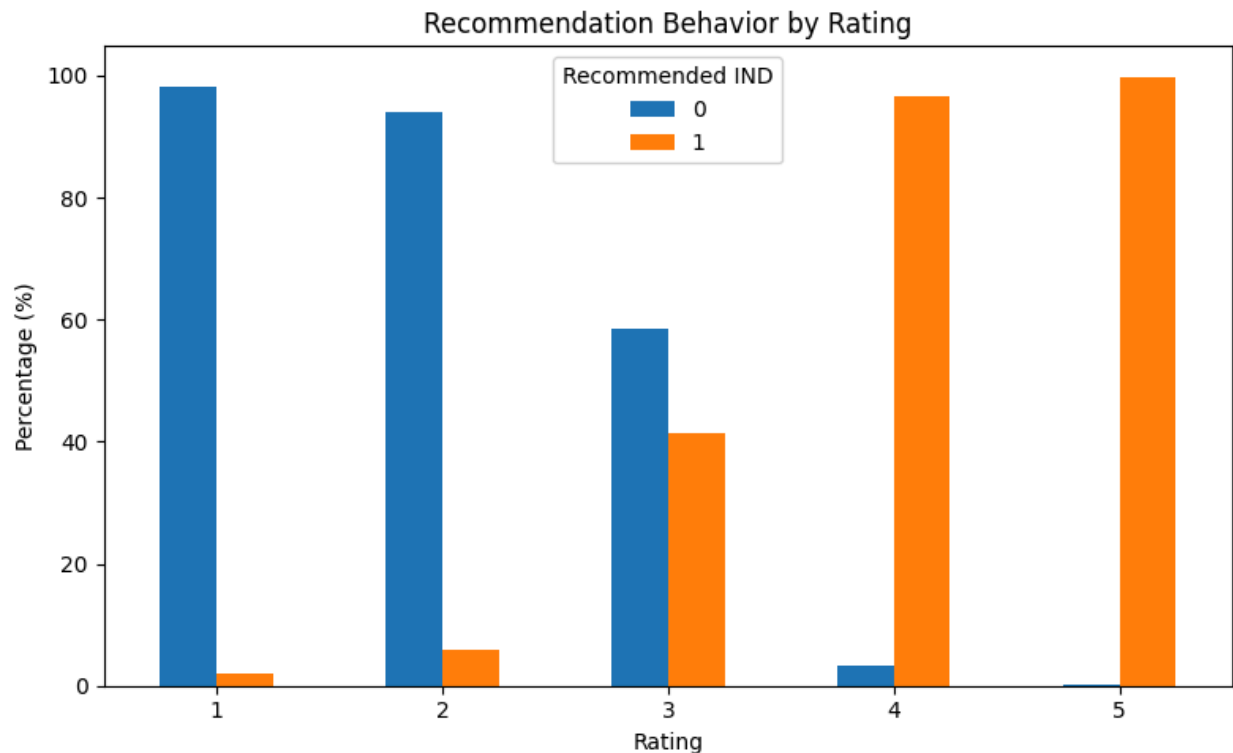
9.5. Khám phá sở thích và hành vi khách hàng

Sau khi chuyển đổi dữ liệu thành các đặc trưng có thể sử dụng cho mô hình, bước tiếp theo là xác định những mẫu hành vi và đặc điểm phản hồi quan trọng của khách hàng. Hành vi khách hàng trong ứng dụng được khám phá thông qua mức độ đánh giá, hành vi

đề xuất sản phẩm, mức độ tương tác với đánh giá, mức độ chi tiết của phản hồi và nhóm sản phẩm được khách hàng đánh giá.

- **Mức độ hài lòng và hành vi đề xuất**

Rating là đặc trưng thể hiện trực tiếp mức độ đánh giá của khách hàng đối với sản phẩm. Kết hợp Rating với Recommended IND cho phép xác định mối quan hệ giữa mức độ hài lòng và hành vi đề xuất sản phẩm.



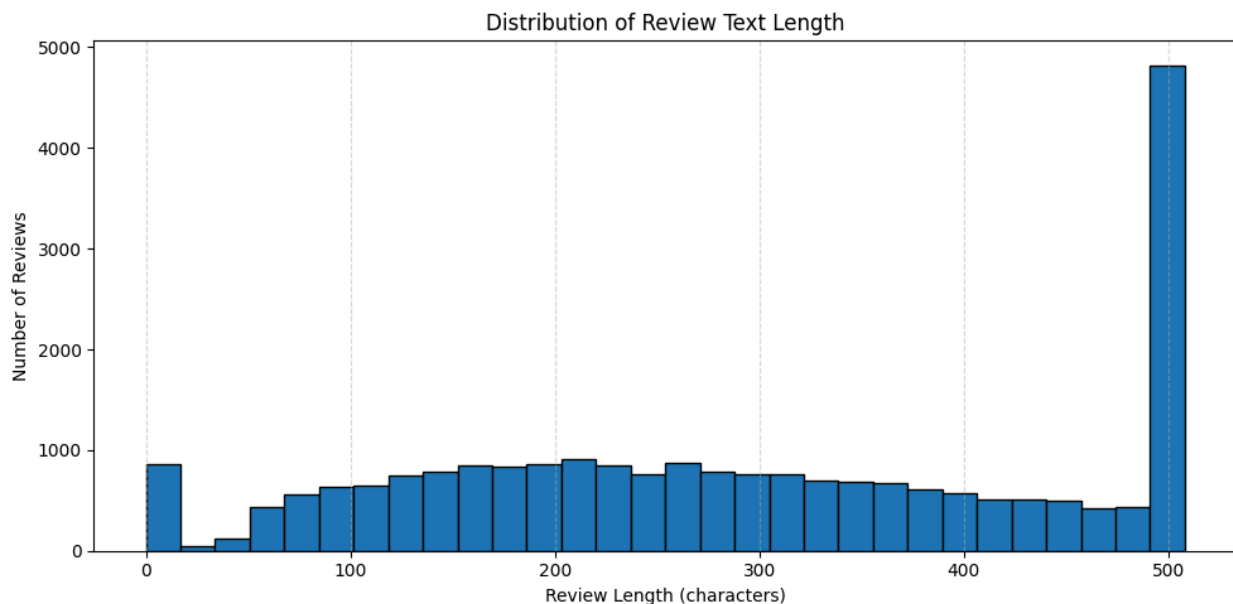
Hình 9.9. Tỷ lệ khách hàng đề xuất sản phẩm theo mức đánh giá

Nhận xét: Tỷ lệ đề xuất sản phẩm thay đổi theo từng mức Rating, trong đó các mức đánh giá cao có xu hướng đi kèm tỷ lệ đề xuất cao hơn.

Đánh giá: Rating là một đặc trưng có mối quan hệ rõ ràng với Recommended IND, phản ánh mức độ hài lòng của khách hàng đối với sản phẩm.

Ý nghĩa đối với mô hình: Rating có khả năng đóng góp đáng kể vào việc dự đoán hành vi đề xuất và được giữ lại làm đặc trưng đầu vào.

- **Hoạt động phản hồi của khách hàng**



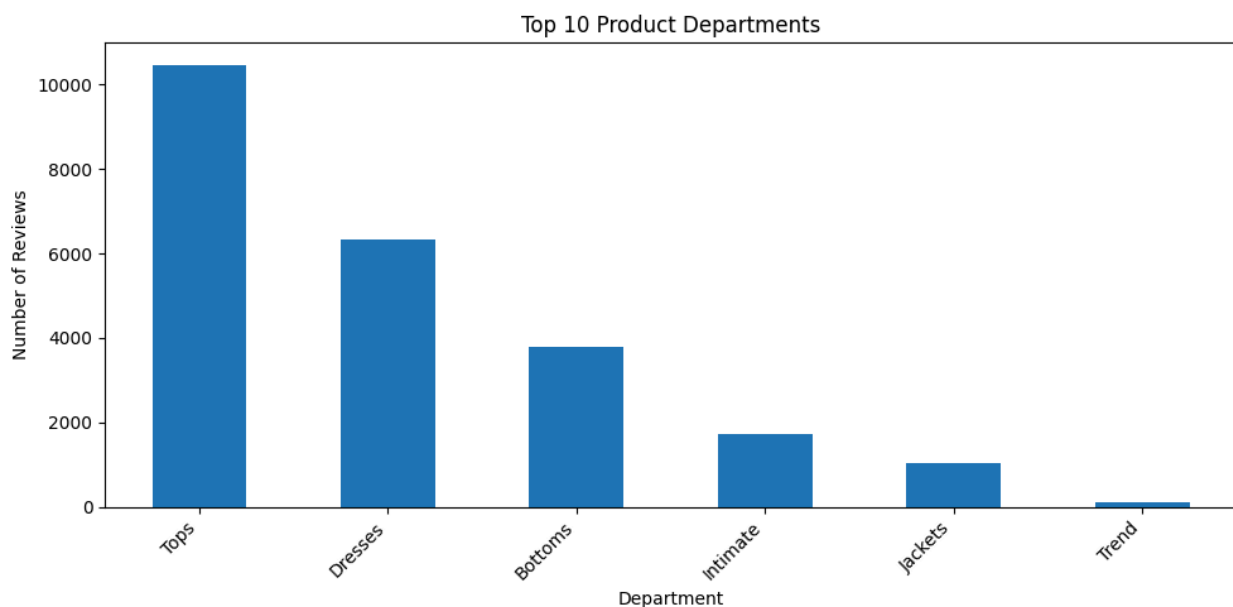
Hình 9.10. Phân bố độ dài nội dung đánh giá của khách hàng

Nhận xét: Độ dài nội dung đánh giá phân bố không đồng đều, phần lớn khách hàng viết các đánh giá tương đối ngắn nhưng vẫn xuất hiện một số đánh giá dài hơn đáng kể.

Đánh giá: Review Length phản ánh mức độ chi tiết trong phản hồi của khách hàng và có thể được sử dụng như một chỉ báo về mức độ hoạt động của khách hàng.

Ý nghĩa đối với mô hình: Đặc trưng Review Length giúp chuyển đặc điểm của văn bản thành giá trị số, từ đó có thể kết hợp với các đặc trưng hành vi khác.

- **Hoạt động theo nhóm sản phẩm**



Hình 9.11. Phân bố số lượng đánh giá theo nhóm sản phẩm

Nhận xét: Số lượng đánh giá phân bố khác nhau giữa các Department Name, trong đó một số nhóm sản phẩm có số lượng đánh giá cao hơn rõ rệt.

Đánh giá: Các nhóm sản phẩm xuất hiện với tần suất khác nhau cho thấy mức độ xuất hiện của khách hàng trong từng ngành hàng không đồng đều. Tuy nhiên, số lượng đánh giá không thể được xem trực tiếp là mức độ yêu thích.

Ý nghĩa đối với mô hình: Department Name cung cấp thông tin về bối cảnh sản phẩm mà khách hàng đang đánh giá và được mã hóa bằng One-Hot Encoding để đưa vào mô hình.

- **Nội dung phản hồi và đặc điểm khách hàng quan tâm**



Hình 9.12. Các từ xuất hiện thường xuyên trong nội dung đánh giá

Nhận xét: Một số từ khóa xuất hiện với tần suất cao trong nội dung đánh giá, cho thấy khách hàng thường đề cập đến một số đặc điểm hoặc trải nghiệm nhất định khi nhận xét sản phẩm.

Đánh giá: Phân tích từ khóa giúp nhận diện những chủ đề thường xuất hiện trong phản hồi, nhưng tần suất xuất hiện đơn thuần chưa thể xác định khách hàng có thái độ tích cực hay tiêu cực.

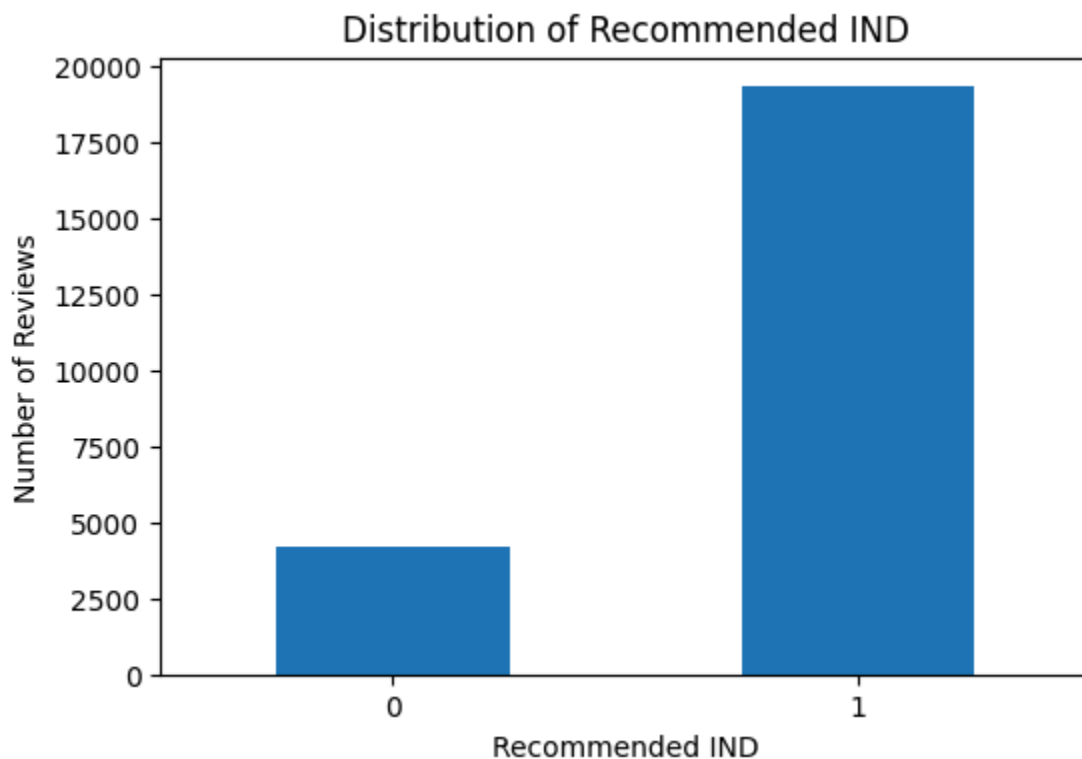
Ý nghĩa đối với mô hình: Nội dung đánh giá cung cấp thông tin bổ sung về phản hồi và sở thích của khách hàng. Phần văn bản được xử lý bằng TF-IDF và sử dụng trong mô hình phân loại văn bản riêng.

- **Tổng hợp các mẫu hành vi quan trọng**

Từ các phân tích trên, những mẫu hành vi quan trọng có thể được xác định gồm:

- Mức độ đánh giá sản phẩm có mối quan hệ rõ ràng với hành vi đề xuất sản phẩm.
- Mức độ hoạt động của khách hàng có thể được biểu diễn thông qua việc cung cấp tiêu đề, nội dung đánh giá và độ dài phản hồi.
- Số lượng phản hồi tích cực cung cấp thông tin bổ sung về mức độ tương tác đối với đánh giá của khách hàng.
- Các nhóm sản phẩm cho phép xác định bối cảnh sản phẩm mà khách hàng đang đánh giá.
- Nội dung Review Text và Title cung cấp thông tin về những đặc điểm thường được khách hàng đề cập.

9.6. Phân tích khám phá dữ liệu (EDA)



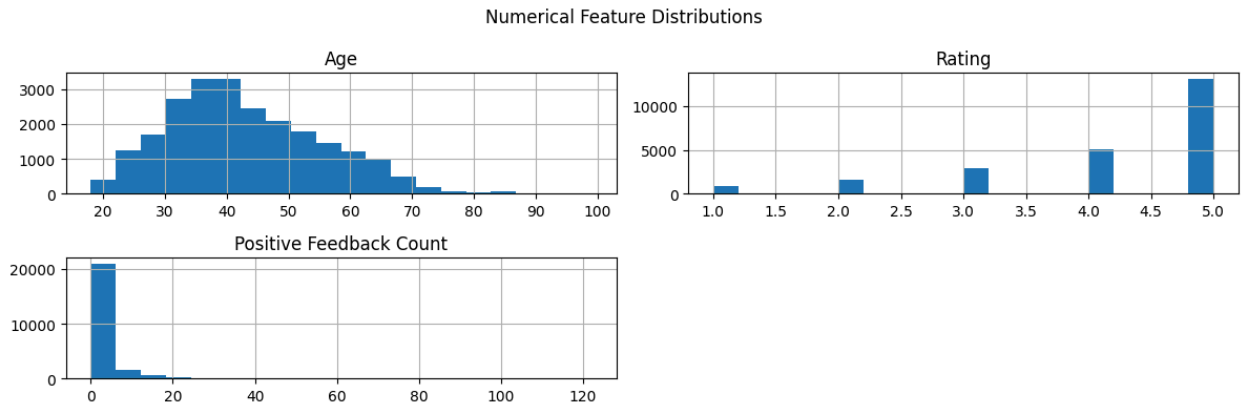
Hình 9.13. Phân bố biến mục tiêu

Nhận xét:

- Biến Recommended IND có hai giá trị 0 và 1, trong đó lớp 1 chiếm phần lớn dữ liệu với 19.314 đánh giá, tương đương khoảng 82,24%.
- Lớp 0 chỉ có 4.172 đánh giá, chiếm khoảng 17,76%.
- Dữ liệu có sự mất cân bằng giữa hai lớp.

Ý nghĩa đối với mô hình ML:

- Mô hình có thể dễ thiên về lớp Recommended IND = 1 do đây là lớp chiếm đa số.
- Vì vậy, không nên chỉ sử dụng Accuracy để đánh giá mô hình mà cần xem thêm Precision, Recall, F1-score và Confusion Matrix.



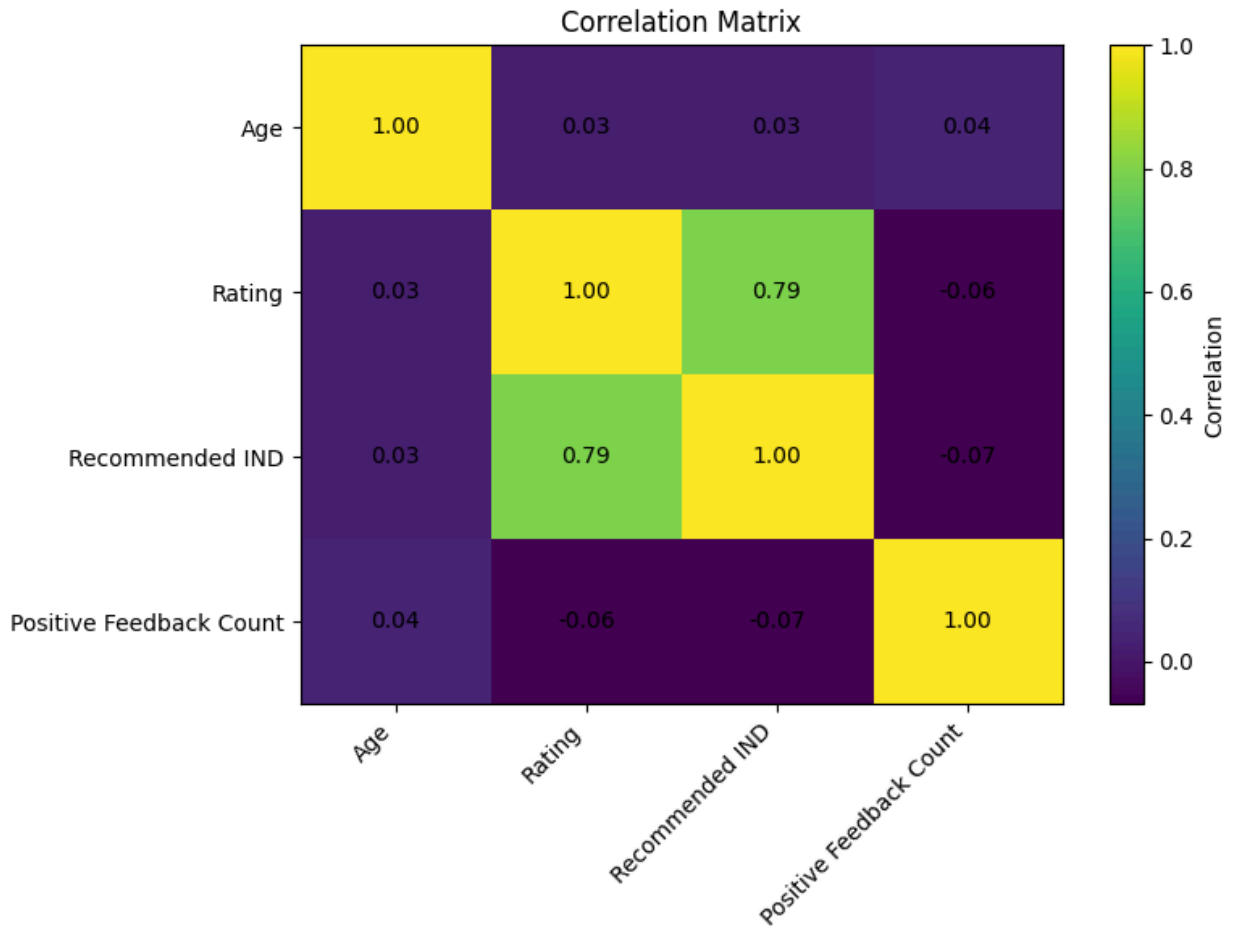
Hình 9.14. Phân bố các đặc trưng số

Nhận xét:

- Age có phạm vi từ 18 đến 99 tuổi, với dữ liệu tập trung chủ yếu ở nhóm tuổi trung niên.
- Rating nằm trong khoảng từ 1 đến 5 và có xu hướng tập trung ở các mức đánh giá cao.
- Positive Feedback Count có phân bố lệch phải, phần lớn đánh giá nhận số lượng phản hồi tích cực thấp, trong khi một số đánh giá có số lượng phản hồi cao hơn đáng kể.
- Các đặc trưng số có phân bố khác nhau và không hoàn toàn tuân theo phân phối chuẩn.

Ý nghĩa đối với mô hình ML:

- Sự khác biệt về thang đo giữa các biến số cần được xử lý trước khi đưa vào mô hình.
- Trong notebook, các biến số được điền giá trị thiếu bằng **median** và chuẩn hóa bằng **StandardScaler**.
- Phân bố lệch của Positive Feedback Count cũng cần được lưu ý khi đánh giá ảnh hưởng của đặc trưng này đến mô hình.



Hình 9.15. Ma trận tương quan

Nhận xét:

- Ma trận tương quan cho thấy mức độ liên hệ tuyến tính giữa Age, Rating, Recommended IND và Positive Feedback Count.
- Trong các biến được phân tích, Rating có mối liên hệ đáng chú ý nhất với Recommended IND.
- Age và Positive Feedback Count có mức tương quan thấp hơn với biến mục tiêu.

Ý nghĩa đối với mô hình ML:

- Rating là một đặc trưng có khả năng đóng góp đáng kể cho việc dự đoán hành vi đề xuất sản phẩm.
- Tuy nhiên, tương quan tuyến tính không phản ánh toàn bộ quan hệ giữa đặc trưng và target, do đó cần đánh giá thông qua các mô hình ML.
- Các đặc trưng có tương quan thấp vẫn được giữ lại vì chúng có thể cung cấp thông tin bổ sung khi kết hợp với các đặc trưng khác.

9.7. Xây dựng mô hình

9.7.1. Chia tập Train, Validation và Test

Tập dữ liệu sau tiền xử lý được chia thành ba tập gồm Train, Validation và Test theo tỷ lệ 70% – 15% – 15%. Tập Train được sử dụng để huấn luyện mô hình, tập Validation được sử dụng để so sánh và lựa chọn mô hình, trong khi tập Test được giữ riêng để đánh giá cuối cùng. Việc chia dữ liệu được thực hiện theo phương pháp **stratified split** dựa trên biến mục tiêu Recommended IND, nhằm duy trì tỷ lệ giữa hai lớp trong các tập dữ liệu.

```
target = "Recommended IND"

X = df_model.drop(columns=[target])
y = df_model[target]

# 70% Train, 30% Temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Split temporary set into 15% Validation and 15% Test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

print("Train:", X_train.shape, y_train.shape)
print("Validation:", X_val.shape, y_val.shape)
print("Test:", X_test.shape, y_test.shape)

Train: (16440, 13) (16440,)
Validation: (3523, 13) (3523,)
Test: (3523, 13) (3523,)
```

Hình 9.16. Kết quả chia tập dữ liệu Train, Validation và Test

Kết quả: Tập Train gồm 16.440 mẫu, tập Validation và Test mỗi tập gồm 3.523 mẫu.

9.7.2. Tiền xử lý dữ liệu

```
numeric_features = [
    "Age",
    "Rating",
    "Positive Feedback Count",
    "Review Length",
    "Title Length",
    "Has Review",
    "Has Title"
]

categorical_features = [
    "Division Name",
    "Department Name",
    "Class Name"
]
```

Hình 9.17. Xác định các nhóm đặc trưng

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

preprocessor = ColumnTransformer([
    ("numerical", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])
```

Hình 9.18. Xây dựng Pipeline tiền xử lý

```
X_train_processed = preprocessor.fit_transform(X_train_tabular)

X_val_processed = preprocessor.transform(X_val_tabular)

X_test_processed = preprocessor.transform(X_test_tabular)

print("Original training shape:", X_train_tabular.shape)
print("Processed training shape:", X_train_processed.shape)

print("Original validation shape:", X_val_tabular.shape)
print("Processed validation shape:", X_val_processed.shape)

print("Original test shape:", X_test_tabular.shape)
print("Processed test shape:", X_test_processed.shape)

print("\nProcessed dtype:", X_train_processed.dtype)

Original training shape: (16440, 10)
Processed training shape: (16440, 36)
Original validation shape: (3523, 10)
Processed validation shape: (3523, 36)
Original test shape: (3523, 10)
Processed test shape: (3523, 36)

Processed dtype: float64
```

Hình 9.19. Biến đổi dữ liệu

Nhận xét: Sau quá trình tiền xử lý, dữ liệu được chuyển từ các đặc trưng số và phân loại ban đầu thành ma trận số có 36 đặc trưng. Các tập Train, Validation và Test đều có cùng số chiều đầu vào, đảm bảo tính nhất quán khi đưa dữ liệu vào mô hình. Đặc biệt, `fit_transform()` chỉ được thực hiện trên tập Train, trong khi Validation và Test chỉ sử dụng `transform()`, giúp tránh việc thông tin từ dữ liệu đánh giá ảnh hưởng đến quá trình học và tiền xử lý mô hình.

9.7.3. Baseline và huấn luyện mô hình

```
baseline = DummyClassifier(  
    strategy="most_frequent"  
)  
  
baseline.fit(X_train_processed, y_train)  
  
baseline_pred = baseline.predict(X_val_processed)  
  
baseline_accuracy = accuracy_score(y_val, baseline_pred)  
baseline_precision = precision_score(  
    y_val,  
    baseline_pred,  
    zero_division=0  
)  
baseline_recall = recall_score(  
    y_val,  
    baseline_pred,  
    zero_division=0  
)  
baseline_f1 = f1_score(  
    y_val,  
    baseline_pred,  
    zero_division=0  
)  
  
print("Baseline strategy: Majority Class")  
print("Baseline class:", baseline.classes_[baseline.class_prior_.argmax()])  
  
print(f"\nValidation Accuracy: {baseline_accuracy:.4f}")  
print(f"Validation Precision: {baseline_precision:.4f}")  
print(f"Validation Recall: {baseline_recall:.4f}")  
print(f"Validation F1-score: {baseline_f1:.4f}")  
  
Baseline strategy: Majority Class  
Baseline class: 1  
  
Validation Accuracy: 0.8223  
Validation Precision: 0.8223  
Validation Recall: 1.0000  
Validation F1-score: 0.9025
```

Hình 9.20. Xây dựng mô hình Baseline

```

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        random_state=42
    ),

    "Decision Tree": DecisionTreeClassifier(
        random_state=42
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),

    "SVM": SVC(
        probability=True,
        random_state=42
    ),

    "Gradient Boosting": GradientBoostingClassifier(
        random_state=42
    )
}

trained_models = {}

for name, model in models.items():
    print(f"Training {name}...")

    model.fit(X_train_processed, y_train)
    trained_models[name] = model

print("\nAll tabular models trained successfully.")

Training Logistic Regression...
Training Decision Tree...
Training Random Forest...
Training SVM...
Training Gradient Boosting...

All tabular models trained successfully.

```

Hình 9.21. Huấn luyện 5 mô hình

9.7.4. Xây dựng mô hình dựa trên nội dung đánh giá

Bên cạnh các đặc trưng dạng bảng, nội dung đánh giá của khách hàng được khai thác để kiểm tra khả năng cải thiện dự đoán khi bổ sung thông tin văn bản. Review Text được chuyển đổi thành vector số bằng phương pháp TF-IDF.

```

tfidf = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    max_features=5000,
    ngram_range=(1, 2)
)

X_train_text = tfidf.fit_transform(train_text)
X_val_text = tfidf.transform(val_text)
X_test_text = tfidf.transform(test_text)

print("Original text samples:", len(train_text))
print("TF-IDF training shape:", X_train_text.shape)
print("TF-IDF validation shape:", X_val_text.shape)
print("TF-IDF test shape:", X_test_text.shape)

Original text samples: 16440
TF-IDF training shape: (16440, 5000)
TF-IDF validation shape: (3523, 5000)
TF-IDF test shape: (3523, 5000)

```

Hình 9.22. Chuyển đổi nội dung đánh giá khách hàng bằng TF-IDF.

```

text_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

text_model.fit(X_train_text, y_train)

text_pred = text_model.predict(X_val_text)

trained_models["Text Logistic Regression"] = text_model

print("Text-based model trained successfully.")

Text-based model trained successfully.

```

Hình 9.23. Huấn luyện mô hình Logistic Regression trên dữ liệu văn bản

Mô hình TF-IDF được cấu hình với tối đa 5.000 đặc trưng và sử dụng cả unigram và bigram. Sau khi biến đổi, mỗi đánh giá được biểu diễn bằng một vector đặc trưng có 5.000 chiều. Logistic Regression được sử dụng làm mô hình phân loại trên biểu diễn văn bản này.

Kết quả biến đổi tạo ra ma trận TF-IDF có kích thước 16.440×5.000 trên tập Train, 3.523×5.000 trên tập Validation và 3.523×5.000 trên tập Test. Cách biểu diễn này cho phép mô hình khai thác thông tin từ nội dung đánh giá thay vì chỉ dựa trên các thuộc tính dạng bảng.

9.8. Đánh giá mô hình

	Model	Accuracy	Precision	Recall	F1
0	SVM	0.9395	0.9838	0.9420	0.9624
1	Gradient Boosting	0.9384	0.9762	0.9482	0.9620
2	Logistic Regression	0.9353	0.9701	0.9506	0.9603
3	Random Forest	0.9330	0.9647	0.9534	0.9590
4	Decision Tree	0.9160	0.9455	0.9527	0.9491
5	Text Logistic Regression	0.8910	0.8970	0.9800	0.9367
6	Baseline	0.8223	0.8223	1.0000	0.9025

Hình 9.24. So sánh hiệu quả các mô hình trên tập Validation

Observation: SVM đạt F1-score cao nhất trên tập Validation với 0.9624, đồng thời đạt Accuracy 0.9395 và Precision 0.9838. Text Logistic Regression có Recall cao nhất với 0.9800 nhưng F1-score chỉ đạt 0.9367.

Interpretation: Các mô hình sử dụng đặc trưng dạng bảng đều cho kết quả tốt hơn mô hình Baseline. SVM có sự cân bằng tốt nhất giữa Precision và Recall theo F1-score.

ML implication: SVM được lựa chọn làm mô hình để đánh giá cuối cùng trên tập Test.

9.9. Lựa chọn mô hình

	Selected Model	Validation F1	Baseline F1	F1 Improvement	Test F1	Test ROC-AUC
0	SVM	0.9624	0.9025	0.0599	0.9616	0.972

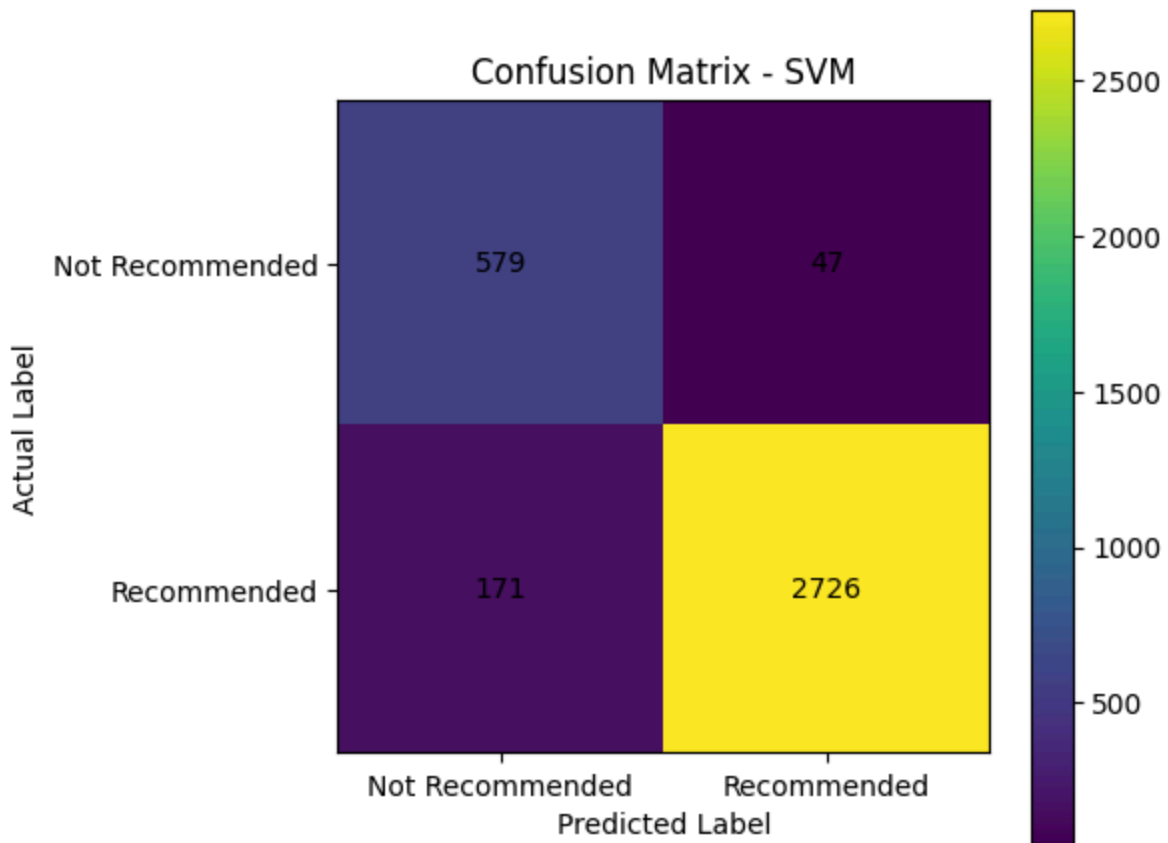
Hình 9.25. Kết quả đánh giá mô hình SVM trên tập Test

Sau khi so sánh trên tập Validation, SVM được lựa chọn để thực hiện đánh giá cuối cùng trên tập Test.

Observation: SVM đạt Accuracy 0.9381 và F1-score 0.9616 trên tập Test. Precision đạt 0.9831, trong khi Recall đạt 0.9410. ROC-AUC đạt 0.9720.

Interpretation: Mô hình có khả năng phân biệt tốt giữa hai lớp Recommended IND. Precision cao cho thấy khi mô hình dự đoán khách hàng có xu hướng đề xuất sản phẩm thì phần lớn dự đoán là chính xác.

ML implication: Kết quả trên tập Test cho thấy SVM duy trì hiệu năng cao trên dữ liệu chưa được sử dụng trong quá trình huấn luyện và lựa chọn mô hình, cho thấy khả năng tổng quát hóa tốt.



Hình 9.26. Confusion Matrix của mô hình SVM trên tập Test

Observation: Mô hình dự đoán đúng 579 mẫu thuộc lớp Not Recommended và 2.726 mẫu thuộc lớp Recommended. Có 47 mẫu bị dự đoán nhầm từ lớp 0 thành lớp 1 và 171 mẫu bị dự đoán nhầm từ lớp 1 thành lớp 0.

Interpretation: Số lượng dự đoán đúng chiếm phần lớn tổng số mẫu Test. Tuy nhiên, mô hình vẫn bỏ sót một số trường hợp Recommended, thể hiện qua 171 mẫu False Negative.

ML implication: Confusion Matrix cho thấy mô hình hoạt động tốt ở cả hai lớp, nhưng vẫn cần chú ý đến khả năng nhận diện lớp Not Recommended, đặc biệt trong trường hợp hệ thống được sử dụng để hỗ trợ đề xuất sản phẩm.


```
# 20. Error Analysis

error_analysis = X_test.copy()

error_analysis["Actual"] = y_test.values
error_analysis["Predicted"] = test_predictions
error_analysis["Correct"] = (
    error_analysis["Actual"] == error_analysis["Predicted"]
)

print("Total test samples:", len(error_analysis))
print("Correct predictions:", error_analysis["Correct"].sum())
print("Incorrect predictions:", (~error_analysis["Correct"]).sum())
```

Total test samples: 3523
 Correct predictions: 3305
 Incorrect predictions: 218

```
false_positive = (
    (error_analysis["Actual"] == 0) &
    (error_analysis["Predicted"] == 1)
).sum()

false_negative = (
    (error_analysis["Actual"] == 1) &
    (error_analysis["Predicted"] == 0)
).sum()

print("False Positives:", false_positive)
print("False Negatives:", false_negative)
```

False Positives: 47
 False Negatives: 171

```
error_rating = errors.groupby(
    ["Actual", "Predicted"]
)["Rating"].agg(
    ["count", "mean"]
)

display(error_rating.round(2))
```

		count	mean
Actual	Predicted		
0	1	47	3.81
1	0	171	2.93

Hình 9.27. Kết quả phân tích lỗi trên tập Test

Kết quả có 218 dự đoán sai trên tổng số 3.523 mẫu Test.

Trong đó:

- False Positive: 47 mẫu.
- False Negative: 171 mẫu.

Phân tích theo Rating cho thấy:

- False Positive có Rating trung bình 3.81.
- False Negative có Rating trung bình 2.93.

Nhận xét: Các lỗi dự đoán có sự khác biệt về mức Rating. Những trường hợp False Negative có Rating trung bình thấp hơn, cho thấy các đánh giá có mức độ hài lòng thấp hơn có thể khó được mô hình nhận diện chính xác trong một số trường hợp.

Ý nghĩa ML: Phân tích lỗi cho thấy Rating có liên quan đến khả năng dự đoán Recommended IND, đồng thời cho thấy vẫn tồn tại những trường hợp mà chỉ các đặc trưng hiện có chưa đủ để mô hình phân loại chính xác.

9.10. Phân tích ý nghĩa kinh doanh

9.10.1. Hành vi khách hàng được phát hiện

Kết quả phân tích cho thấy Rating có mối liên hệ đáng chú ý với **Recommended IND**. Những đánh giá có mức Rating cao có xu hướng đi kèm với khả năng khách hàng đề xuất sản phẩm cao hơn. Bên cạnh đó, độ dài nội dung đánh giá, thông tin về nhóm sản phẩm và các từ khóa thường xuất hiện trong Review Text cũng cung cấp thông tin bổ sung về hành vi và phản hồi của khách hàng.

Mô hình SVM đạt F1-score 0.9616 và ROC-AUC 0.9720 trên tập Test, cho thấy các đặc trưng được xây dựng có khả năng hỗ trợ tốt cho việc dự đoán khách hàng có xu hướng đề xuất sản phẩm.

9.10.2. Ứng dụng trong thương mại điện tử

Kết quả dự đoán có thể được sử dụng để:

- **Cá nhân hóa đề xuất sản phẩm:** xác định những khách hàng có khả năng đánh giá tích cực để ưu tiên các sản phẩm phù hợp.
- **Hỗ trợ chiến dịch marketing:** sử dụng thông tin về hành vi và nhóm sản phẩm để xây dựng các nội dung tiếp thị phù hợp.
- **Phân tích phản hồi khách hàng:** khai thác Rating và Review Text để phát hiện các xu hướng trong phản hồi về sản phẩm.
- **Hỗ trợ chăm sóc khách hàng:** chú ý đến các trường hợp có khả năng không đề xuất sản phẩm để tìm hiểu nguyên nhân và cải thiện trải nghiệm.

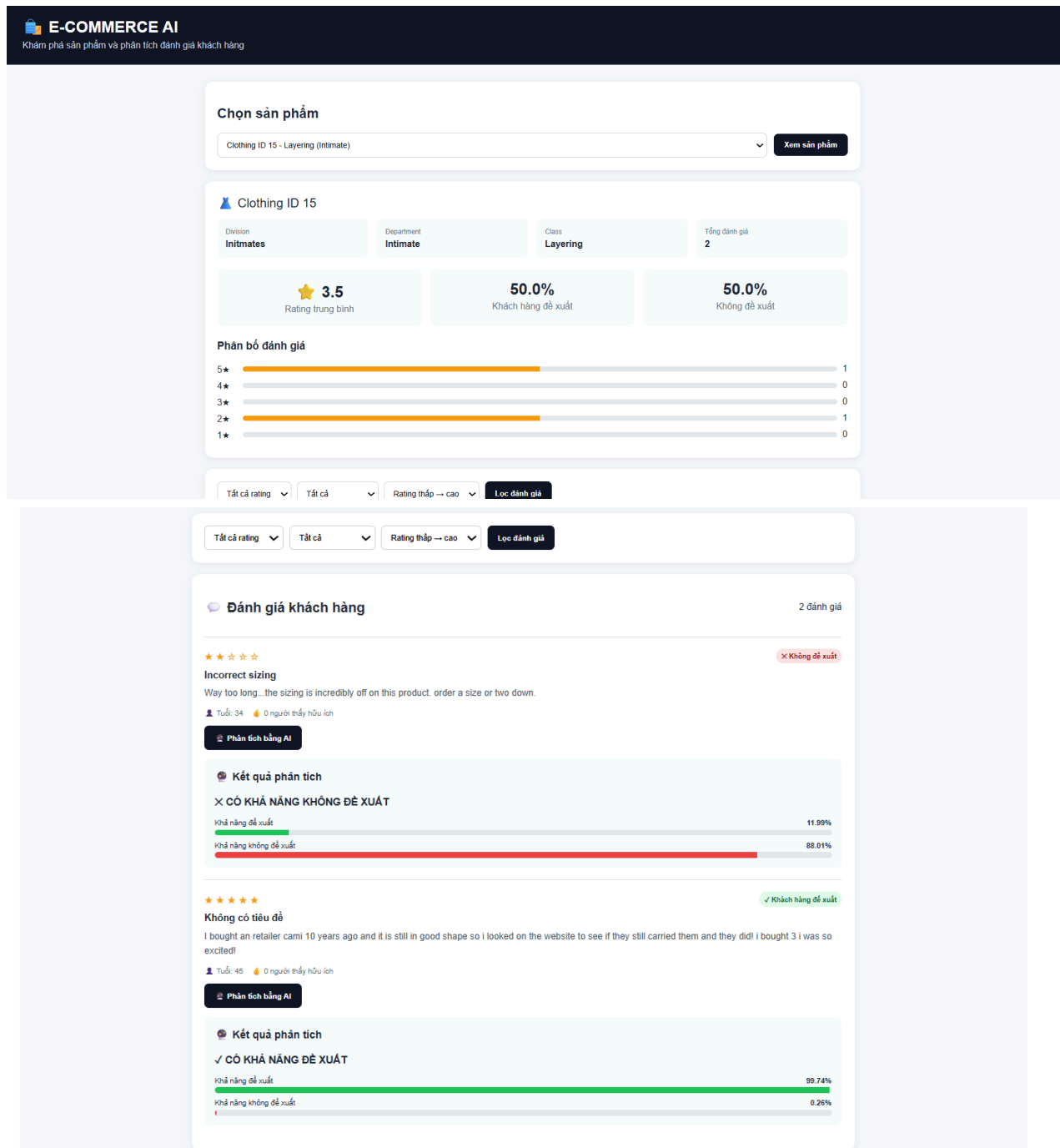
9.10.3. Hạn chế về dữ liệu

Tập dữ liệu hiện tại chủ yếu chứa thông tin đánh giá sản phẩm, không có lịch sử nhiều giao dịch của từng khách hàng, thời gian mua hàng, giá trị đơn hàng hoặc hành vi duyệt web và giỏ hàng. Vì vậy, hệ thống hiện tại tập trung vào dự đoán khả năng khách hàng đề xuất sản phẩm dựa trên thông tin đánh giá và đặc trưng sản phẩm, thay vì xây dựng hệ

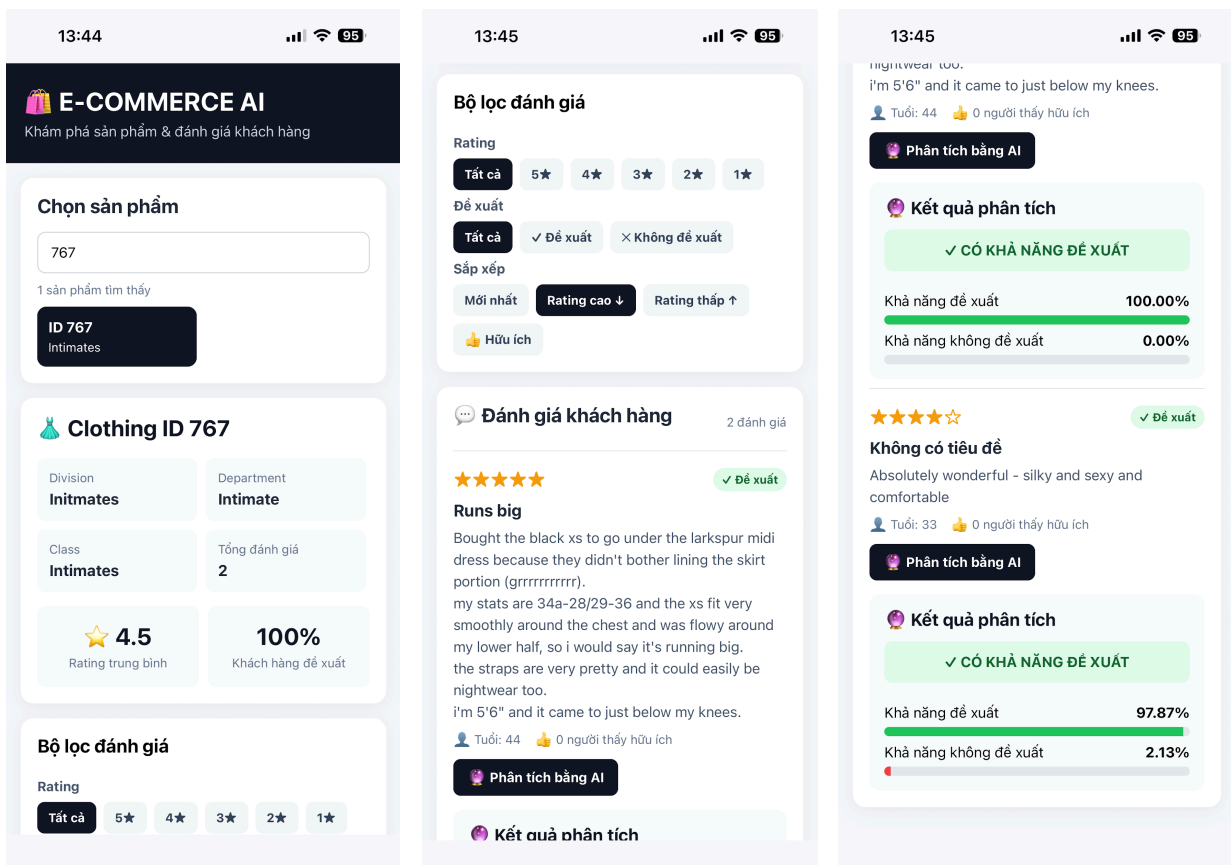
thống Recommendation theo lịch sử mua hàng thực tế.

Kết luận: Mô hình hiện tại phù hợp với mục tiêu phân tích phản hồi và dự đoán xu hướng đề xuất sản phẩm, nhưng nếu có thêm dữ liệu giao dịch và hành vi khách hàng, hệ thống có thể được mở rộng để phát hiện sở thích và xây dựng đề xuất sản phẩm cá nhân hóa sâu hơn.

9.11. Triển khai hệ thống



Hình 9.28. Deploy dạng web



Hình 9.29. Deploy dạng mobile bằng expo go

Chương X. So sánh ba hệ thống thông minh

10.1. So sánh dữ liệu và bài toán

Tiêu chí	Diabetes	House Price	E-commerce
Loại bài toán	Classification	Regression	Classification
Một observation	Một bệnh nhân	Một căn nhà	Một đánh giá sản phẩm
Target	Diabetes class	House price	Recommended IND
Dữ liệu	Tabular	Tabular	Tabular + Text
Đặc điểm chính	Thông tin sức khỏe	Đặc điểm bất động sản	Thông tin đánh giá khách hàng

Ba bộ dữ liệu có sự khác biệt rõ rệt về đối tượng quan sát và mục tiêu dự đoán. Trong Diabetes, một observation biểu diễn thông tin của một bệnh nhân và mục tiêu là xác định bệnh nhân thuộc lớp tiểu đường nào. Trong House Price, một observation biểu diễn đặc điểm của một căn nhà và mục tiêu là dự đoán giá nhà, do đó đây là bài toán hồi quy. Trong E-commerce, một observation biểu diễn một đánh giá sản phẩm của khách hàng và mục tiêu là dự đoán hành vi Recommended IND.

Sự khác biệt về biến mục tiêu dẫn đến sự khác biệt về loại bài toán. Diabetes và E-commerce được xử lý như các bài toán phân loại vì biến mục tiêu có các lớp rời rạc, trong khi House Price là bài toán hồi quy vì giá nhà là một giá trị số liên tục.

10.2. So sánh biểu diễn dữ liệu

Thành phần	Diabetes	House Price	E-commerce
Raw data	CSV / tabular	CSV / tabular	CSV / tabular + review text
Feature vector	Có	Có	Có
Feature matrix	Có	Có	Có
Categorical encoding	One-Hot Encoding	One-Hot Encoding	One-Hot Encoding
Numerical scaling	StandardScaler	StandardScaler	Theo pipeline đã sử dụng

Text representation	Không	Không	TF-IDF
Final representation	Feature matrix	Feature matrix	Tabular features + TF-IDF

Diabetes và House Price đều là dữ liệu dạng bảng. Mỗi observation sau khi làm sạch được biểu diễn thành một feature vector và toàn bộ dataset được biểu diễn thành feature matrix $X \in \mathbb{R}^{(N \times d)}$. Các biến số được giữ dưới dạng giá trị số và được chuẩn hóa bằng StandardScaler, trong khi các biến phân loại được chuyển đổi sang dạng số bằng One-Hot Encoding.

Đối với Diabetes, quá trình tiền xử lý làm tăng số chiều của không gian đặc trưng do các biến phân loại được mã hóa. Feature matrix cuối cùng được sử dụng làm đầu vào cho các mô hình phân loại.

Đối với House Price, các đặc điểm của căn nhà cũng được chuyển thành feature vector. Sau khi mã hóa các biến phân loại và chuẩn hóa các biến số, số chiều của feature matrix tăng từ không gian đặc trưng ban đầu lên không gian biểu diễn cuối cùng được sử dụng cho các mô hình hồi quy.

E-commerce có biểu diễn phức tạp hơn vì ngoài các đặc trưng dạng bảng còn có nội dung đánh giá của khách hàng. Các đặc trưng dạng bảng được mã hóa và chuyển đổi thành giá trị số, trong khi Review Text được chuyển đổi thành vector TF-IDF. Do đó, hệ thống có thể sử dụng đồng thời thông tin hành vi dạng bảng và thông tin từ nội dung văn bản.

10.3. So sánh chất lượng dữ liệu và tiền xử lý

10.3.1. Các bước tiền xử lý chung

Cả ba hệ thống đều thực hiện các bước kiểm tra chất lượng dữ liệu như missing values, duplicated records, invalid values và outliers. Các biến dạng số cần được đưa về không gian biểu diễn phù hợp với mô hình, trong khi các biến phân loại phải được chuyển đổi thành dạng số trước khi huấn luyện.

Một điểm chung quan trọng là preprocessing phải được học từ dữ liệu huấn luyện và sau đó được tái sử dụng trong validation, test và inference. Không được fit lại scaler, encoder hoặc các thành phần preprocessing bằng dữ liệu test hoặc dữ liệu người dùng trong quá trình triển khai, vì điều này có thể gây data leakage.

10.3.2. Các bước tiền xử lý riêng

Diabetes có đặc điểm riêng là dữ liệu mất cân bằng giữa các lớp, do đó việc đánh giá không thể chỉ dựa trên Accuracy mà cần chú ý đến Precision, Recall và F1-score.

House Price có đặc điểm riêng của bài toán hồi quy, trong đó outliers và phân bố lệch của

biến giá có thể ảnh hưởng đáng kể đến sai số dự đoán. Vì vậy, việc phân tích outlier và phân phối của biến mục tiêu có ý nghĩa quan trọng.

E-commerce có preprocessing phức tạp hơn do tồn tại dữ liệu văn bản. Review Text cần được làm sạch và chuyển đổi thành biểu diễn số bằng phương pháp TF-IDF trước khi đưa vào mô hình phân loại văn bản.

10.4. So sánh mô hình và đánh giá

Tiêu chí	Diabetes	House Price	E-commerce
Problem	Classification	Regression	Classification
Best model	Random Forest	Ridge Regression	Support Vector Machine
Main metric	F1-score	RMSE / R^2	F1-score
Test result	F1 = 0.8119	$R^2 = 0.7465$	F1 = 0.9624
Main reason	Cân bằng Precision/Recall	Sai số dự đoán	Cân bằng các lớp

10.5. So sánh triển khai

Tiêu chí	Diabetes	House Price	E-commerce
Saved model	Có	Có	Có
Saved preprocessing	Có	Có	Có
REST API	Có	Có	Có
Web	Có	Có	Có
Mobile	Có	Có	Có
Input	Patient features	House features	Customer/review features
Output	Diabetes prediction	Estimated price	Customer behavior

10.6. Trả lời các câu hỏi thảo luận

10.6.1. Ứng dụng dự đoán bệnh tiểu đường

Câu 1. What does one observation represent?

Một observation trong bộ dữ liệu Diabetes đại diện cho thông tin của một bệnh nhân. Mỗi dòng dữ liệu chứa các đặc trưng liên quan đến tình trạng sức khỏe của bệnh nhân và một biến mục tiêu biểu diễn lớp liên quan đến bệnh tiểu đường. Do đó, mỗi observation có thể được xem như một hồ sơ bệnh nhân được sử dụng để thực hiện dự đoán.

Câu 2. What is the raw data representation?

Dữ liệu ban đầu của ứng dụng Diabetes được lưu dưới dạng CSV và có cấu trúc dữ liệu dạng bảng. Mỗi hàng tương ứng với một bệnh nhân và mỗi cột tương ứng với một đặc trưng hoặc biến mục tiêu. Đây là dạng biểu diễn có cấu trúc, trong đó các giá trị ban đầu có thể bao gồm cả đặc trưng số và các biến cần được chuyển đổi trước khi đưa vào mô hình.

Câu 3. What is the final numerical representation?

Sau quá trình tiền xử lý, dữ liệu Diabetes được chuyển thành ma trận số để mô hình học máy có thể xử lý. Các đặc trưng được xử lý thông qua quá trình mã hóa đối với các biến phân loại và chuẩn hóa đối với các biến số. Trong quá trình xây dựng hệ thống, dữ liệu ban đầu có kích thước 96.146×8 và sau khi mã hóa, số lượng đặc trưng đầu vào tăng lên 15, tạo thành ma trận đặc trưng có kích thước 96.146×15 . Mỗi hàng của ma trận là một feature vector đại diện cho một bệnh nhân.

Câu 4. What do the dimensions of the feature matrix mean?

Ma trận đặc trưng của Diabetes có dạng $X \in \mathbb{R}^{N \times d}$ với N là số lượng observation trong tập dữ liệu và d là số lượng đặc trưng sau quá trình biểu diễn dữ liệu. Với dữ liệu của hệ thống, ma trận cuối cùng có 96.146 hàng và 15 cột, nghĩa là có 96.146 bệnh nhân và mỗi bệnh nhân được biểu diễn bằng một vector gồm 15 đặc trưng số.

Câu 5. Which features required encoding?

Các đặc trưng có kiểu dữ liệu phân loại hoặc không thể sử dụng trực tiếp dưới dạng số cần được mã hóa trước khi đưa vào mô hình. Trong hệ thống Diabetes, các biến phân loại được chuyển đổi thành dạng số bằng One Hot Encoding. Quá trình này tạo ra các cột nhị phân tương ứng với các giá trị có thể có của biến phân loại, giúp mô hình có thể xử lý chúng dưới dạng số mà không tạo ra thứ tự giả giữa các nhóm.

Câu 6. Which features required normalization?

Các đặc trưng số có thang đo khác nhau được chuẩn hóa bằng StandardScaler. Việc chuẩn hóa giúp đưa các đặc trưng về thang đo phù hợp hơn, đặc biệt quan trọng đối với các mô hình như Logistic Regression, SVM và KNN. Quá trình chuẩn hóa được thực hiện trong pipeline để đảm bảo cùng một phép biến đổi được sử dụng trong quá trình huấn luyện và triển khai.

Câu 7. What information was lost during representation?

Trong quá trình biểu diễn dữ liệu, một phần thông tin về dạng biểu diễn ban đầu có thể bị mất. Đối với các biến phân loại, thông tin về tên hoặc dạng biểu diễn ban đầu được chuyển thành các giá trị nhị phân sau One Hot Encoding. Ngoài ra, sau khi chuẩn hóa, giá trị tuyệt đối ban đầu của các đặc trưng số không còn được giữ nguyên mà được chuyển thành giá trị thể hiện vị trí tương đối của chúng trong phân phối dữ liệu. Tuy nhiên, những thay đổi này không làm mất thông tin cần thiết cho quá trình dự đoán mà chủ yếu thay đổi cách dữ liệu được biểu diễn.

Câu 8. What information was preserved?

Các thông tin quan trọng liên quan đến đặc điểm của bệnh nhân vẫn được bảo toàn trong feature vector cuối cùng. Các giá trị của những đặc trưng số được giữ lại dưới dạng giá trị đã chuẩn hóa, trong khi thông tin của các biến phân loại được giữ lại thông qua các biến nhị phân được tạo bởi One Hot Encoding. Biến mục tiêu cũng được giữ độc lập để phục vụ quá trình huấn luyện và đánh giá mô hình.

Câu 9. What preprocessing could cause data leakage?

Các thao tác như StandardScaler, One Hot Encoder hoặc Imputer có thể gây data leakage nếu được fit trên toàn bộ dữ liệu trước khi chia Train, Validation và Test. Đặc biệt, nếu scaler được học từ Test hoặc nếu các tham số xử lý missing value được tính từ Test thì thông tin của tập đánh giá sẽ ảnh hưởng đến mô hình. Trong hệ thống, pipeline được fit trên tập Train, trong khi Validation và Test chỉ sử dụng phép transform. Cách thực hiện này đảm bảo dữ liệu đánh giá không ảnh hưởng đến quá trình học và tiền xử lý mô hình.

Câu 10. Which model performed best?

Random Forest là mô hình có kết quả tốt nhất trong ứng dụng Diabetes. Trên tập Validation, Random Forest đạt Accuracy 0,9672 và F1-score 0,7825, trong đó F1-score cao nhất trong các mô hình được so sánh. Trên tập Test, mô hình đạt Accuracy 0,9710, Precision 0,9495, Recall 0,7091, F1-score 0,8119 và ROC-AUC 0,9617.

Câu 11. Why was that model selected?

Random Forest được lựa chọn vì đạt F1-score cao nhất trên tập Validation và tạo được sự cân bằng tốt hơn giữa Precision và Recall. Điều này đặc biệt quan trọng đối với dữ liệu Diabetes vì dữ liệu có sự mất cân bằng giữa các lớp. Nếu chỉ dựa vào Accuracy, mô hình có thể đạt điểm cao nhưng vẫn bỏ sót nhiều trường hợp thuộc lớp thiểu số. Random

Forest cũng có khả năng mô hình hóa các mối quan hệ phi tuyến giữa các đặc trưng sức khỏe của bệnh nhân.

Câu 12. Which evaluation metric is most important?

F1-score là metric quan trọng nhất trong quá trình lựa chọn mô hình Diabetes vì dữ liệu có sự mất cân bằng giữa các lớp. F1-score kết hợp Precision và Recall nên phản ánh tốt hơn khả năng mô hình vừa hạn chế dự đoán dương tính sai vừa hạn chế bỏ sót các trường hợp thuộc lớp cần quan tâm. Accuracy vẫn được báo cáo để cung cấp góc nhìn tổng quát, trong khi ROC-AUC được sử dụng để đánh giá khả năng phân biệt hai lớp của mô hình.

Câu 13. How is the model persisted?

Sau khi lựa chọn mô hình tốt nhất, mô hình và preprocessing pipeline được lưu lại để có thể sử dụng trong quá trình inference mà không cần huấn luyện lại. Việc lưu preprocessing cùng với model giúp đảm bảo dữ liệu đầu vào trong quá trình triển khai được biến đổi giống với dữ liệu đã sử dụng trong quá trình huấn luyện.

Câu 14. How does the Web service use the persisted model?

Web service nhận thông tin bệnh nhân từ giao diện người dùng, thực hiện kiểm tra dữ liệu đầu vào và đưa dữ liệu qua preprocessing pipeline đã được lưu. Sau đó, dữ liệu đã được chuyển đổi được đưa vào Random Forest để tạo ra kết quả dự đoán. Kết quả được API trả về cho giao diện web để hiển thị dự đoán và xác suất hoặc thông tin giải thích phù hợp.

Câu 15. How does the mobile application communicate with the prediction service?

Ứng dụng mobile đóng vai trò là client của REST API. Người dùng nhập các thông tin cần thiết trên mobile, ứng dụng gửi dữ liệu dưới dạng request đến API, API thực hiện preprocessing và inference bằng mô hình đã lưu, sau đó trả kết quả về dưới dạng response. Mobile nhận response và hiển thị kết quả dự đoán cho người dùng. Kiến trúc này đảm bảo quá trình huấn luyện không diễn ra trên thiết bị mobile mà chỉ thực hiện inference thông qua dịch vụ dự đoán.

10.6.2. Ứng dụng dự đoán giá nhà

Câu 1. What does one observation represent?

Một observation trong bộ dữ liệu House Price đại diện cho một căn nhà. Mỗi dòng chứa các đặc điểm của căn nhà như diện tích, số phòng, vị trí và các thuộc tính khác, cùng với giá nhà tương ứng. Vì vậy, observation có thể được xem là một hồ sơ mô tả một bất động sản cụ thể.

Câu 2. What is the raw data representation?

Dữ liệu ban đầu của House Price được lưu dưới dạng CSV và có cấu trúc bảng. Mỗi hàng

đại diện cho một căn nhà và mỗi cột đại diện cho một thuộc tính của căn nhà hoặc biến mục tiêu là giá nhà. Dữ liệu bao gồm cả các đặc trưng số và các đặc trưng phân loại, do đó cần được xử lý trước khi đưa vào mô hình hồi quy.

Câu 3. What is the final numerical representation?

Sau quá trình tiền xử lý, các đặc trưng của căn nhà được chuyển thành ma trận số. Các biến phân loại được mã hóa thành các biến số bằng One Hot Encoding và các đặc trưng số được chuẩn hóa bằng StandardScaler trong pipeline. Số lượng đặc trưng tăng từ 15 đặc trưng ban đầu lên 31 đặc trưng sau khi mã hóa. Vì vậy, mỗi căn nhà cuối cùng được biểu diễn bằng một feature vector gồm 31 giá trị số.

Câu 4. What do the dimensions of the feature matrix mean?

Feature matrix có dạng $X \in \mathbb{R}^{N \times d}$, trong đó N là số lượng căn nhà và d là số lượng đặc trưng sau tiền xử lý. Mỗi hàng tương ứng với một căn nhà và mỗi cột tương ứng với một đặc trưng trong biểu diễn số cuối cùng. Ví dụ, sau khi One Hot Encoding, một thuộc tính vị trí ban đầu có thể tạo ra nhiều cột nhị phân trong ma trận đặc trưng.

Câu 5. Which features required encoding?

Các đặc trưng phân loại như thông tin về vị trí hoặc các thuộc tính có dạng category cần được mã hóa trước khi đưa vào mô hình. One Hot Encoding được sử dụng để chuyển các giá trị phân loại thành các biến số nhị phân. Cách biểu diễn này giúp mô hình xử lý được dữ liệu phân loại mà không áp đặt thứ tự giả giữa các nhóm.

Câu 6. Which features required normalization?

Các đặc trưng số được chuẩn hóa bằng StandardScaler để giảm sự khác biệt về thang đo giữa các biến. Việc chuẩn hóa đặc biệt hữu ích đối với các mô hình hồi quy tuyến tính như Linear Regression và Ridge Regression. Tuy nhiên, các mô hình cây như Decision Tree và Random Forest không phụ thuộc nhiều vào scale của dữ liệu. Trong hệ thống, preprocessing được quản lý trong pipeline để đảm bảo tính nhất quán.

Câu 7. What information was lost during representation?

Một phần thông tin về cách dữ liệu được biểu diễn ban đầu bị thay đổi trong quá trình mã hóa và chuẩn hóa. Các biến phân loại không còn được lưu dưới dạng chuỗi ban đầu mà được biểu diễn bằng các cột số. Các giá trị số sau chuẩn hóa cũng không còn giữ nguyên đơn vị ban đầu. Tuy nhiên, thông tin cần thiết để mô tả đặc điểm của căn nhà vẫn được giữ lại trong feature vector.

Câu 8. What information was preserved?

Các thông tin chính về căn nhà vẫn được giữ lại trong biểu diễn cuối cùng. Diện tích, số

phòng, các thuộc tính vật lý và thông tin phân loại đều được chuyển thành những đặc trưng số tương ứng. Đặc biệt, One Hot Encoding vẫn giữ thông tin về nhóm phân loại, trong khi StandardScaler giữ nguyên mối quan hệ tương đối giữa các giá trị số.

Câu 9. What preprocessing could cause data leakage?

Việc tính toán các tham số chuẩn hóa hoặc mã hóa dựa trên toàn bộ dataset trước khi chia dữ liệu có thể gây data leakage. Nếu StandardScaler được fit trên cả Test hoặc nếu encoder sử dụng thông tin từ Test để xác định cách biểu diễn thì mô hình sẽ gián tiếp biết thông tin của dữ liệu đánh giá. Vì vậy, preprocessing phải được fit trên Train và chỉ transform Validation và Test.

Câu 10. Which model performed best?

Ridge Regression là mô hình có kết quả tốt nhất trong ứng dụng House Price. Trên tập Validation, Ridge đạt RMSE thấp nhất là 147127,42 và R^2 bằng 0,7563. Trên tập Test, Ridge đạt MAE bằng 117323,29, MSE bằng $2,126660 \times 10^{10}$, RMSE bằng 145830,73 và R^2 bằng 0,7465.

Câu 11. Why was that model selected?

Ridge Regression được lựa chọn vì đạt kết quả tốt nhất trên tập Validation theo tiêu chí RMSE. Mô hình có hiệu quả tốt hơn các mô hình Decision Tree, Random Forest và Gradient Boosting trên dữ liệu hiện tại. Việc sử dụng regularization của Ridge cũng giúp kiểm soát ảnh hưởng của các đặc trưng và cải thiện khả năng tổng quát hóa khi dữ liệu có nhiều đặc trưng sau quá trình mã hóa.

Câu 12. Which evaluation metric is most important?

RMSE là metric quan trọng trong việc lựa chọn mô hình House Price vì nó thể hiện mức độ sai lệch dự đoán theo cùng đơn vị với giá nhà và phạt mạnh hơn những sai số lớn. Bên cạnh RMSE, MAE được sử dụng để thể hiện sai số tuyệt đối trung bình và R^2 được sử dụng để đánh giá mức độ biến thiên của giá nhà được mô hình giải thích. MSE cũng được báo cáo theo yêu cầu của bài toán hồi quy.

Câu 13. How is the model persisted?

Mô hình Ridge Regression sau khi được lựa chọn được lưu cùng với preprocessing pipeline. Việc lưu cả mô hình và preprocessing giúp hệ thống có thể thực hiện inference trên dữ liệu mới mà không cần huấn luyện lại. Trong quá trình triển khai, dữ liệu đầu vào phải đi qua đúng pipeline đã được sử dụng trong quá trình huấn luyện.

Câu 14. How does the Web service use the persisted model?

Web application cung cấp form để người dùng nhập các thông tin của căn nhà. Dữ liệu được gửi đến REST API, sau đó API kiểm tra dữ liệu, sử dụng preprocessing pipeline đã

lưu để chuyển đổi dữ liệu thành feature vector và đưa vector này vào Ridge Regression. Kết quả trả về là giá nhà được dự đoán và được giao diện web hiển thị cho người dùng.

Câu 15. How does the mobile application communicate with the prediction service?

Ứng dụng mobile gửi thông tin căn nhà đến REST API thông qua HTTP request. API nhận dữ liệu, thực hiện cùng preprocessing như trong quá trình huấn luyện và sử dụng mô hình Ridge đã lưu để dự đoán giá nhà. Kết quả dự đoán được trả về mobile dưới dạng response và được hiển thị trên giao diện ứng dụng. Mobile không thực hiện huấn luyện mô hình mà chỉ thực hiện giao tiếp với dịch vụ dự đoán. Kiến trúc này phù hợp với yêu cầu Mobile → REST API → Preprocessing → Saved ML Model → Prediction → REST Response → Mobile.

10.6.3. Ứng dụng E-commerce Customer Behavior and Interest Discovery

Câu 1. What does one observation represent?

Một observation trong bộ dữ liệu E-commerce đại diện cho một đánh giá của khách hàng đối với một sản phẩm. Mỗi dòng chứa các thông tin liên quan đến khách hàng, sản phẩm, rating, nội dung đánh giá và biến Recommended IND. Vì vậy, observation trong hệ thống này được hiểu là một bản ghi phản ánh phản hồi và hành vi của khách hàng đối với sản phẩm.

Câu 2. What is the raw data representation?

Dữ liệu E-commerce được lưu dưới dạng CSV và có cấu trúc bảng, trong đó ngoài các đặc trưng dạng số và phân loại còn có trường văn bản Review Text. Do đó, dữ liệu ban đầu có hai dạng chính là dữ liệu bảng và dữ liệu văn bản. Đây là điểm khác biệt quan trọng so với Diabetes và House Price, vì nội dung đánh giá cần được chuyển đổi từ văn bản sang biểu diễn số trước khi mô hình có thể sử dụng.

Câu 3. What is the final numerical representation?

Đối với các đặc trưng dạng bảng, sau tiền xử lý dữ liệu được chuyển thành ma trận số gồm 36 đặc trưng. Các biến số được xử lý missing bằng median và chuẩn hóa bằng StandardScaler, trong khi các biến phân loại được mã hóa. Đối với Review Text, dữ liệu được chuyển thành ma trận TF-IDF với tối đa 5.000 đặc trưng sử dụng unigram và bigram. Ma trận TF-IDF có kích thước 16.440×5.000 trên Train, 3.523×5.000 trên Validation và 3.523×5.000 trên Test.

Câu 4. What do the dimensions of the feature matrix mean?

Đối với dữ liệu dạng bảng, ma trận 16.440×36 trên tập Train có nghĩa là có 16.440 observation và mỗi observation được biểu diễn bằng 36 đặc trưng số sau preprocessing. Đối với biểu diễn văn bản, ma trận 16.440×5.000 có nghĩa là có 16.440 review và mỗi

review được biểu diễn bằng một vector gồm tối đa 5.000 đặc trưng TF-IDF. Như vậy, số hàng biểu thị số observation còn số cột biểu thị số chiều của biểu diễn dữ liệu.

Câu 5. Which features required encoding?

Các biến phân loại trong dữ liệu E-commerce cần được mã hóa thành dạng số để mô hình có thể xử lý. One Hot Encoding được sử dụng để chuyển các nhóm phân loại thành các cột số. Đối với Review Text, cách xử lý khác với biến categorical vì văn bản được chuyển đổi thành các đặc trưng TF-IDF thay vì One Hot Encoding thông thường.

Câu 6. Which features required normalization?

Các đặc trưng số trong dữ liệu E-commerce có thang đo khác nhau nên được chuẩn hóa bằng StandardScaler. Trong quá trình xử lý, các giá trị thiếu của đặc trưng số được thay thế bằng median trước khi chuẩn hóa. Việc chuẩn hóa giúp các đặc trưng có thang đo khác nhau trở nên phù hợp hơn với các mô hình như Logistic Regression và SVM.

Câu 7. What information was lost during representation?

Trong quá trình TF-IDF, cấu trúc và ngữ nghĩa đầy đủ của câu trong Review Text không được giữ nguyên. TF-IDF chủ yếu biểu diễn mức độ quan trọng của các từ hoặc cụm từ trong tài liệu và không hiểu đầy đủ ngữ cảnh, thứ tự ngữ nghĩa hoặc mối quan hệ sâu giữa các từ. Ngoài ra, những từ không nằm trong vocabulary được xây dựng bởi TF-IDF sẽ không được biểu diễn trong vector cuối cùng. Vì vậy, TF-IDF cung cấp một biểu diễn hữu ích nhưng vẫn có giới hạn về khả năng hiểu ngữ nghĩa.

Câu 8. What information was preserved?

Biểu diễn TF-IDF vẫn giữ được thông tin về sự xuất hiện và mức độ quan trọng của các từ và cụm từ trong Review Text. Việc sử dụng cả unigram và bigram giúp mô hình khai thác không chỉ từng từ riêng lẻ mà còn một số cụm từ gồm hai từ. Đối với dữ liệu dạng bảng, các thông tin về Rating, Age, Positive Feedback Count và các thuộc tính phân loại vẫn được giữ lại dưới dạng các feature số sau preprocessing.

Câu 9. What preprocessing could cause data leakage?

Data leakage có thể xảy ra nếu TF-IDF vectorizer được fit trên toàn bộ dataset trước khi chia Train, Validation và Test. Khi đó vocabulary và IDF được tính từ cả dữ liệu Test, khiến thông tin của Test ảnh hưởng đến biểu diễn dữ liệu dùng cho huấn luyện. Tương tự, StandardScaler, Imputer và Encoder cũng không được fit trên Validation hoặc Test. Trong hệ thống, preprocessing được fit trên Train và Validation, Test chỉ sử dụng phép transform để đảm bảo tập Test vẫn độc lập.

Câu 10. Which model performed best?

SVM là mô hình có kết quả tốt nhất trong ứng dụng E-commerce trên biểu diễn dữ liệu

dạng bảng. Trên tập Validation, SVM đạt F1-score 0,9624, Accuracy 0,9395 và Precision 0,9838. Sau khi được lựa chọn, SVM đạt F1-score 0,9616 và ROC-AUC 0,9720 trên tập Test. Kết quả này cho thấy SVM có khả năng cân bằng tốt giữa Precision và Recall đối với bài toán dự đoán Recommended IND.

Câu 11. Why was that model selected?

SVM được lựa chọn vì đạt F1-score cao nhất trên tập Validation trong nhóm mô hình sử dụng đặc trưng dạng bảng. F1-score phản ánh sự cân bằng giữa Precision và Recall, phù hợp với dữ liệu E-commerce có sự mất cân bằng giữa hai lớp Recommended IND. Mặc dù mô hình Text Logistic Regression có Recall cao hơn với 0,9800, F1-score chỉ đạt 0,9367, thấp hơn SVM. Vì vậy, SVM được xem là lựa chọn phù hợp hơn dựa trên tiêu chí cân bằng giữa các chỉ số phân loại.

Câu 12. Which evaluation metric is most important?

F1-score là metric quan trọng nhất trong quá trình lựa chọn mô hình E-commerce vì biến Recommended IND bị mất cân bằng. Lớp 1 chiếm khoảng 82,24% dữ liệu trong khi lớp 0 chỉ chiếm khoảng 17,76%, do đó Accuracy đơn thuần có thể tạo ra đánh giá quá lạc quan về hiệu quả mô hình. Precision, Recall và F1-score cần được xem xét đồng thời, trong đó F1-score được sử dụng để đánh giá sự cân bằng giữa Precision và Recall. ROC-AUC cũng cung cấp thêm thông tin về khả năng phân biệt hai lớp.

Câu 13. How is the model persisted?

Mô hình SVM và preprocessing pipeline được lưu sau khi quá trình huấn luyện và lựa chọn mô hình hoàn tất. Pipeline chứa các bước xử lý cần thiết để biến đổi dữ liệu đầu vào về đúng dạng mà mô hình đã học. Nhờ đó, khi triển khai, hệ thống chỉ cần tải preprocessing và model đã lưu để thực hiện inference mà không cần huấn luyện lại.

Câu 14. How does the Web service use the persisted model?

Web application nhận thông tin đầu vào từ người dùng và gửi dữ liệu đến REST API. API thực hiện validation, áp dụng preprocessing pipeline đã lưu và đưa feature vector sau biến đổi vào SVM. Mô hình trả về dự đoán Recommended IND, sau đó API gửi kết quả trở lại giao diện web để hiển thị cho người dùng. Toàn bộ quá trình triển khai sử dụng mô hình đã được huấn luyện và không thực hiện lại quá trình training.

Câu 15. How does the mobile application communicate with the prediction service?

Mobile application hoạt động như một client của REST API. Người dùng nhập thông tin trên giao diện mobile, ứng dụng tạo HTTP request và gửi đến prediction API. API sử dụng preprocessing pipeline và mô hình SVM đã lưu để thực hiện inference, sau đó trả kết quả về mobile. Mobile nhận response và hiển thị dự đoán cho người dùng. Cách triển khai này tách biệt phần giao diện người dùng với mô hình học máy và phù hợp với kiến trúc Mobile → REST API → Preprocessing → Saved ML Model → Prediction → REST

Response → Mobile.

10.6.4. Câu hỏi bổ sung cho ứng dụng E-commerce

Câu 1. What information is contained in customer comments?

Customer comments chứa thông tin phản hồi trực tiếp của khách hàng về sản phẩm. Nội dung review có thể thể hiện mức độ hài lòng, đánh giá về chất lượng sản phẩm, trải nghiệm sử dụng, ưu điểm, nhược điểm và các vấn đề mà khách hàng gặp phải. Trong dữ liệu E-commerce, Review Text cung cấp một nguồn thông tin bổ sung bên cạnh các đặc trưng dạng bảng như Rating, Age, Department Name và Positive Feedback Count. Phân tích nội dung review cũng giúp phát hiện các từ khóa thường xuyên xuất hiện và các đặc điểm trong phản hồi của khách hàng.

Câu 2. How are comments transformed into numerical data?

Customer comments được làm sạch và chuyển đổi thành biểu diễn số bằng TF-IDF. TF-IDF xây dựng một vocabulary từ dữ liệu huấn luyện và biểu diễn mỗi review bằng một vector số thể hiện mức độ quan trọng của các từ hoặc cụm từ trong review. Hệ thống sử dụng tối đa 5.000 đặc trưng và kết hợp unigram với bigram. Vì vậy, mỗi review cuối cùng được biểu diễn bằng một vector có tối đa 5.000 chiều và có thể được đưa vào Logistic Regression để thực hiện phân loại.

Câu 3. What do token IDs represent?

Token ID là một số nguyên được sử dụng để đại diện cho một token trong vocabulary của một hệ thống xử lý văn bản. Mỗi token, chẳng hạn một từ hoặc một đơn vị văn bản, được ánh xạ tới một ID tương ứng. Token ID không trực tiếp biểu diễn ý nghĩa của từ mà chỉ là chỉ số dùng để xác định token trong vocabulary. Trong triển khai hiện tại của báo cáo, phần text classification sử dụng TF-IDF trực tiếp thay vì xây dựng đầy đủ pipeline Token → Token ID → Embedding, vì vậy token ID chưa phải là biểu diễn đầu vào cuối cùng của mô hình text hiện tại.

Câu 4. What do embedding vectors represent?

Embedding vector là một vector số dùng để biểu diễn đặc điểm và ngữ nghĩa của token, từ, câu hoặc tài liệu trong không gian vector. Những đơn vị văn bản có ý nghĩa hoặc ngữ cảnh tương tự có xu hướng được biểu diễn gần nhau trong không gian embedding. Embedding có khả năng biểu diễn thông tin ngữ nghĩa phong phú hơn so với biểu diễn TF-IDF. Tuy nhiên, trong hệ thống hiện tại, báo cáo đang sử dụng TF-IDF với 5.000 đặc trưng cho phần text classification thay vì embedding.

Câu 5. Which customer interests can be discovered?

Từ dữ liệu E-commerce có thể phát hiện một số thông tin về sở thích và hành vi của

khách hàng thông qua Rating, Recommended IND, nhóm sản phẩm, độ dài review và các từ khóa thường xuyên xuất hiện trong Review Text. Kết quả hiện tại cho thấy Rating có mối liên hệ đáng chú ý với Recommended IND, trong đó các đánh giá có Rating cao có xu hướng đi kèm với khả năng khách hàng đề xuất sản phẩm cao hơn. Các nhóm sản phẩm và nội dung review cũng có thể được sử dụng để nhận diện những chủ đề hoặc loại sản phẩm mà khách hàng quan tâm.

Câu 6. Does text improve prediction compared with tabular features alone?

Dựa trên kết quả hiện tại, chưa thể khẳng định rằng text cải thiện prediction so với tabular features alone một cách tuyệt đối. Mô hình Text Logistic Regression đạt Recall cao nhất là 0,9800 nhưng F1-score chỉ đạt 0,9367, trong khi SVM trên các đặc trưng dạng bảng đạt F1-score 0,9624 trên Validation và 0,9616 trên Test. Điều này cho thấy trong thí nghiệm hiện tại, biểu diễn dạng bảng kết hợp với SVM cho kết quả cân bằng hơn so với mô hình Logistic Regression sử dụng TF-IDF riêng biệt. Do đó, text có cung cấp thêm thông tin nhưng chưa chứng minh được rằng việc sử dụng text riêng biệt mang lại hiệu quả tốt hơn biểu diễn dạng bảng.

Chương XI. Mở rộng - chatbot với neo4j

11.1. Luồng hoạt động

Hệ thống chatbot được xây dựng theo mô hình Web UI – Flask Backend – Knowledge Graph/REST API, trong đó Flask đóng vai trò trung gian tiếp nhận câu hỏi, xác định loại yêu cầu và truy vấn nguồn dữ liệu phù hợp.

Luồng hoạt động tổng quát:

Người dùng → Web Chat UI → Flask Chatbot Backend → Neo4j / REST API → Xử lý kết quả → Flask Chatbot Backend → Web Chat UI → Người dùng.

Trong đó:

- Web Chat UI: Giao diện cho phép người dùng nhập câu hỏi và hiển thị câu trả lời từ chatbot.
- Flask Chatbot Backend: Tiếp nhận câu hỏi, xử lý và xác định nguồn dữ liệu cần sử dụng.
- Neo4j Knowledge Graph: Lưu trữ và truy vấn các mối quan hệ giữa các thực thể. Trong hệ thống, Neo4j được sử dụng cho dữ liệu bệnh tiểu đường và E-commerce.
- REST API: Cung cấp các chức năng dự đoán của các mô hình ML, bao gồm Diabetes API, House Price API và E-commerce API.
- Xử lý kết quả: Chatbot nhận dữ liệu từ Neo4j hoặc API, định dạng kết quả thành câu trả lời ngắn gọn và gửi lại giao diện.

Luồng xử lý chi tiết:

Bước 1. Người dùng nhập câu hỏi trên Web Chat UI.

Bước 2. Câu hỏi được gửi đến Flask Chatbot Backend thông qua endpoint `/chat`.

Bước 3. Backend phân tích nội dung câu hỏi và xác định loại yêu cầu.

Bước 4. Tùy vào loại câu hỏi, chatbot lựa chọn nguồn dữ liệu phù hợp:

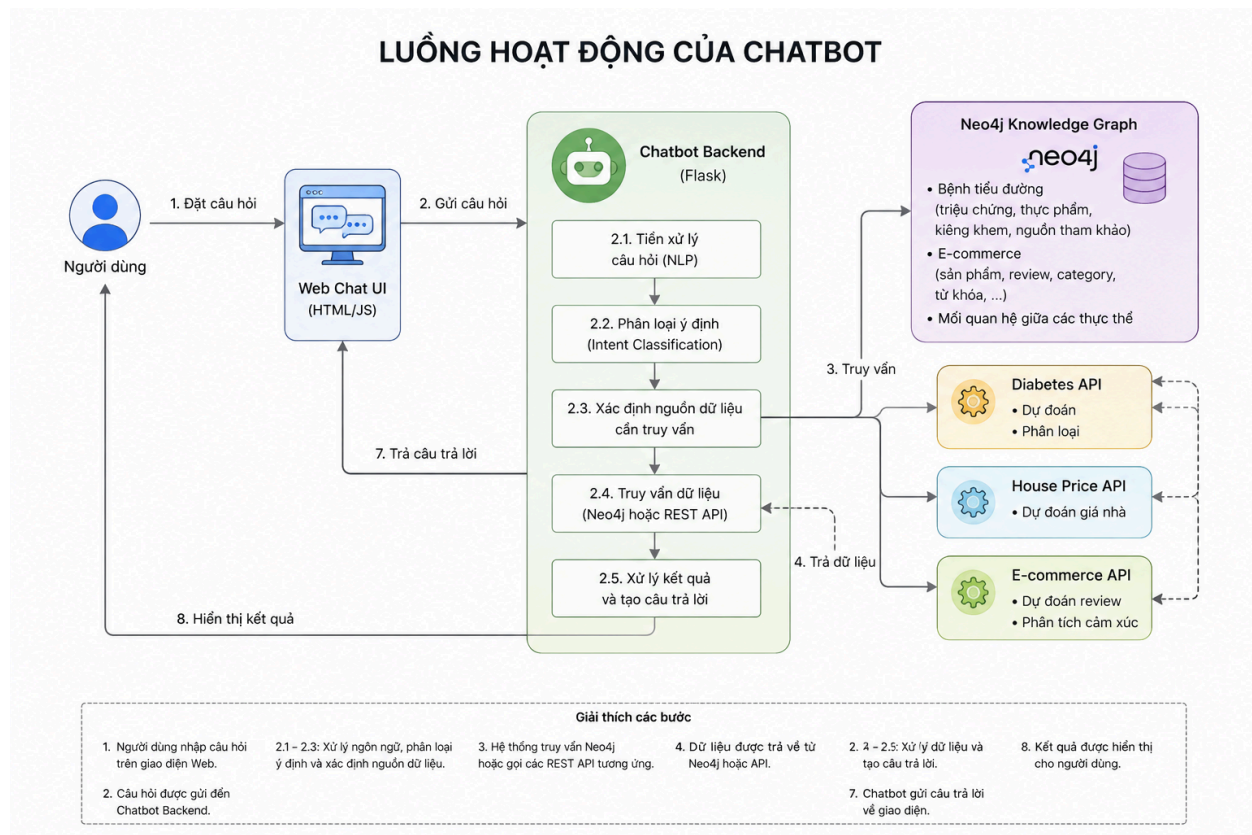
- Câu hỏi về bệnh tiểu đường → truy vấn Neo4j.
- Câu hỏi về sản phẩm và review E-commerce → truy vấn Neo4j hoặc gọi E-commerce API.
- Câu hỏi dự đoán giá nhà → gọi House Price API.
- Câu hỏi dự đoán review → gọi E-commerce API.
- Câu hỏi liên quan đến dự đoán tiểu đường → gọi Diabetes API.

Bước 5. Neo4j hoặc REST API trả dữ liệu về Flask Backend.

Bước 6. Backend xử lý và định dạng dữ liệu thành câu trả lời.

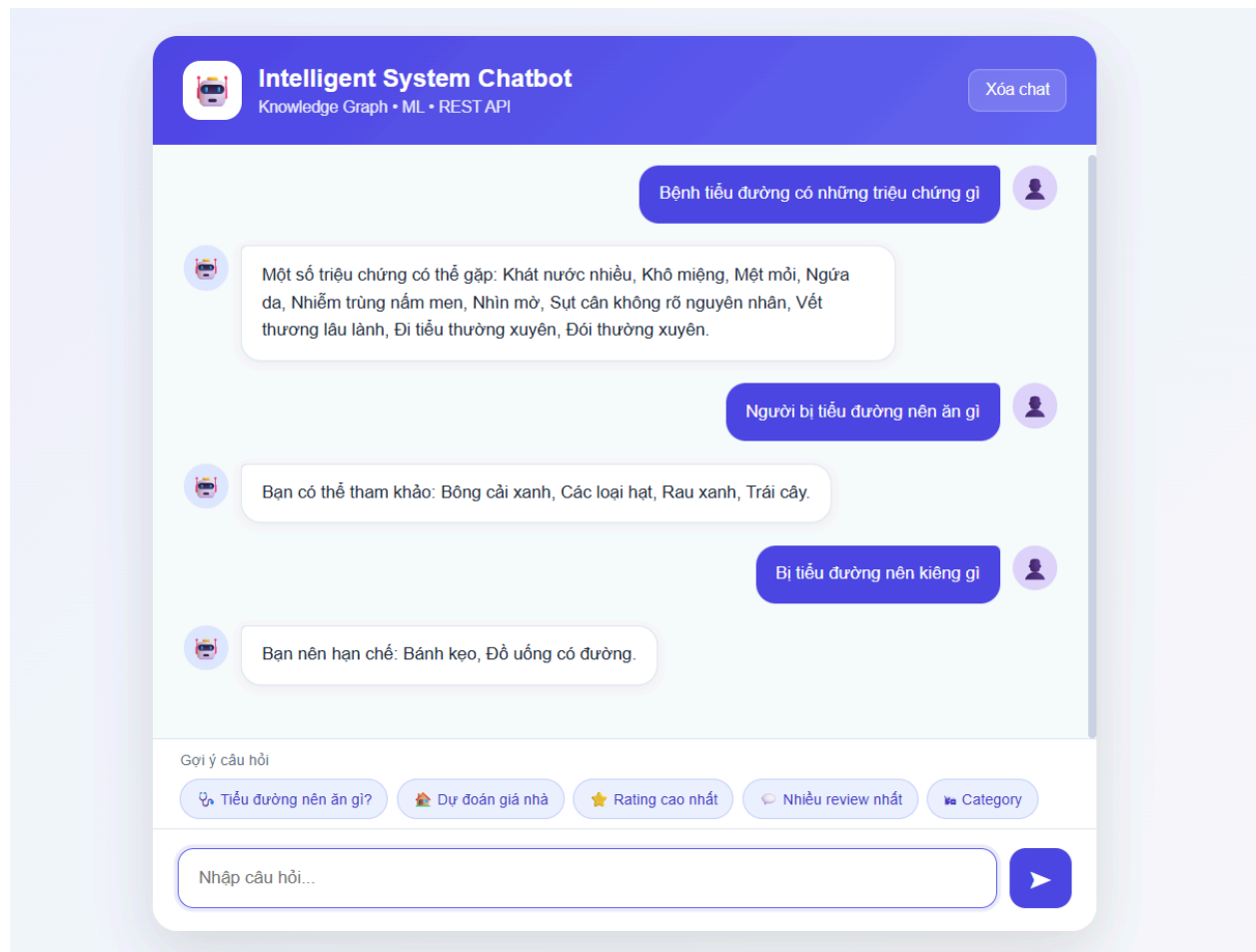
Bước 7. Câu trả lời được gửi về Web Chat UI.

Bước 8. Người dùng nhận kết quả trên giao diện chatbot.



Hình 11.1. Kiến trúc và luồng hoạt động của hệ thống Chatbot

11.2. Kết quả thực nghiệm



Hình 11.2. Kết quả thử nghiệm chatbot